

No Honor Among Crooks: Non-transferable Anonymous Tokens from Betrayability

David Kretzler
Huawei Technologies
Düsseldorf, Germany
david.kretzler@huawei.com

Yong Li
Huawei Technologies
Düsseldorf, Germany
yong.li1@huawei.com

Abstract—In anonymous token protocols, clients obtain access tokens by proving eligibility for the usage of a resource and later get access to the resource by redeeming the token. The server verifying eligibility and providing the resource cannot link the token issuance to its redemption. To prevent ineligible clients from accessing resources, it is crucial to prevent the transfer or sale of tokens. Durak et al. (CCS’24) propose binding tokens to valuable insurance secrets, which must be known to redeem the tokens. The value of the insurance secret deters the vendor from transferring the secret to the token buyer, who cannot redeem the token without the secret. However, the authors do not consider scenarios, where the token vendor assists the buyer during the token redemption. Their construction, therefore, falls short to guarantee non-transferability when facing a vendor-aided token redemption.

We address this gap by introducing the concept of *anonymous tokens with betrayability*. Our notion ensures that a token buyer, that is able to redeem a bought token, either knows the insurance secret or is able to betray the vendor in a vendor-aided redemption. The betrayal allows the buyer to steal the insurance secret without being detected. This way, we make the support in the token redemption equivalent to a transfer of the insurance secret and, hence, inherit the transfer deterrence of the insurance secret even when considering a vendor-aided token redemption. We formalize our new security notion, present a protocol for anonymous tokens with betrayability, prove its security, and provide an implementation and experimental evaluation.

1. Introduction

In recent years, anonymous tokens attracted a lot of attention, both from academia [1]–[11] and industry [12]–[14]. An anonymous token scheme is a protocol executed between three parties, the client, the issuer and the verifier. A client first requests a token from the issuer via an interactive issuance protocol, in which the issuer verifies the client’s eligibility to access a resource and then issues a token to the client. A prominent example for eligibility is humanhood, which can be proven by solving a CAPTCHA. At a later point in time, the client presents the token to a verifier, which verifies authenticity of the token and grants access to a service or resource if the verification is successful.

This process is called *token redemption*. Usually, the verifier checks whether the token has already been redeemed to prevent double spending. The basic security properties required by anonymous tokens are accountability and anonymity. The former states that only the issuer can issue valid tokens while the latter requires that the issuer and verifier cannot link the token issuance to its redemption.

Various publications [1]–[11] introduced a variety of protocols for anonymous tokens offering different trade-offs and additional properties. While some constructions [1], [2], [6] require the token issuance key for the verification of tokens and, hence, require the issuer and verifier to be the same entity, others [3]–[5], [7]–[11] enable tokens to be verified publicly and, hence, allow the issuer and verifier to be different entities. Further, there are proposals allowing issuers to include public [4], [10] or private [2], [3], [6], [11] metadata into tokens. Public meta data allows to add a label to tokens such as the revocation information or a usage restriction. Private meta data, on the other hand, allows the issuer to embed an invalidity flag into tokens instead of rejecting token requests. As the meta data is private, the flag can only be checked by the verifier but not by the client. This makes it harder for adversaries to derive information about the eligibility verification process.

One aspect that has received only little attention so far is the prevention of the transfer of tokens. Consider a scenario in which one client, the vendor, receives a token and sells it to another client, the buyer. Such an undesired flow of tokens can grant buyers access to resources for which they are not eligible. Unfortunately, preventing the transfer of tokens while still guaranteeing anonymity is notoriously hard. If the vendor transfers its whole state to the buyer, the only difference between the vendor and the buyer is the identity. A verifier capable of rejecting transferred tokens, hence, needs to know something about the client’s identity, which violates anonymity. The only work so far addressing non-transferability of tokens is by Durak et al. [7]. In their work, the authors utilize a technique previously known from [15]–[17] to address the described difficulty. They bind each token to a valuable insurance secret, such as the secret key of a cryptocurrency wallet. The insurance secret needs to be known to redeem the token. However, the client does not present the insurance secret in clear but provides some evidence proving knowledge of the secret. The intuition

behind this approach is that even though a malicious token vendor might be willing to transfer a token it will not transfer its insurance secret due to its value.

One downside of the solution of [7] is that the authors assume clients to redeem tokens on their own, i.e., the vendor and the buyer cannot communicate during the token redemption. Without this assumption, it would be possible for the vendor to assist the buyer in the creation of the evidence, that proves knowledge of the insurance secret, without actually transferring the insurance secret to the buyer. Such an attack undermines the non-transferability properties of their construction. We aim to address this shortcoming. In particular, we allow the token vendor to support the token buyer during the token redemption but enable the token buyer to betray the vendor during the vendor-aided evidence creation. More precisely, we enable the buyer to embed a trapdoor into the evidence creation, which allows it to retrieve the insurance secret from the evidence without being detected. As a vendor cannot know if it will be betrayed by a buyer, the transfer of a token including the aid in the redemption of said token becomes equivalent to transferring the insurance secret itself.

1.1. Contribution

In this work, we propose the notion of *anonymous tokens with betrayability (ATB)*, provide a formal definition, present an instantiation based on BBS signatures [18], prove security of our instantiation, and provide an experimental evaluation. Both, our security notion and our constructions constitute extensions of *non-transferable anonymous tokens (NTAT)* introduced by [7].

Similar to NTAT, anonymous tokens with betrayability bind tokens to valuable insurance secrets and require clients redeeming a token to prove ownership of the associated insurance secret. However, unlike prior work, we explicitly consider the threat of a collusion between the token vendor and the token buyer during redemption. To address this threat, we require betrayability, which allows the token buyer to betray the token vendor by including a trapdoor into the vendor-aided redemption. The trapdoor enables the extraction of the insurance secret without being detected. This way, we make the support in the redemption of a token equivalent to the transfer of the associated insurance secret.

We observe that it is impossible to achieve betrayability in the model proposed by [7] (cf. Appendix A). Therefore, we make a minor sacrifice in terms of unlinkability, i.e., we identify clients during redemption via a pseudonym realized as long-time identification public key. This means that we restrict our scope to applications in which the provided resource or service is bound to an identity, e.g., a user account, but a foreign-issued token is still required to prove claims about the client, e.g., human-hood. However, our scheme still guarantees that the token redemption cannot be linked to its issuance and allows the utilization of multiple pseudonyms for the same or different services. Technically, the introduction of identification keys is necessary as a buyer without a secret, that is capable of betraying the vendor,

could be emulated by a malicious verifier to extract insurance secrets from honest clients. By binding the trapdoor to the client's identification secret key, we can prevent the verifier from exploiting the trapdoor. Conceptually, we assess the identification of clients before accessing a resource to fit well into the setting of non-transferable tokens, as any token-authorized access to resources not bound to the client's identity could also be performed by the token vendor on behalf of the buyer, which makes the transfer of the token redundant.

We present an instantiation of our ATB definition and prove its security. Our construction aims to push most of the runtime to the issuance phase, as we envision this phase to be less efficiency critical. Furthermore, we have implemented and evaluated the presented instantiation as well as three alternative constructions offering different trade-offs, i.e., runtime versus communication overhead and issuance versus redemption efficiency. Our evaluation shows that all constructions provide high efficiency in the phase, they are optimized for (1 - 34 % of the overall runtime). Furthermore, the worst-case runtime of a whole protocol execution without network delays in the runtime-optimized versions of our construction is below 200 ms.

1.2. Technical Overview

In the following, we provide a technical overview over our construction.

BBS signatures. Analogously to [7], [19], tokens in our work constitute BBS signatures [18], [20], [21]. Let $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ be a tuple of groups of prime order p with generators $G_0, H_0, H_1, \dots, H_n \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$ and map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a bilinear pairing. A BBS signature on a message array $\{x_i\}_{i \in [0, L]}$ under a secret signing key $z \in \mathbb{Z}_p$ is a tuple (σ, e) with $\sigma = \frac{1}{z+e}(G_0 + \sum_{i \in [0, L]} x_i H_i)$ and $e \xleftarrow{\$} \mathbb{Z}_p$. The signature is verified under public verification key $Z = zG_2$ by checking whether $e(\sigma, Z + eG_2) = e(G_0 + \sum_{i \in [0, L]} x_i H_i, G_2)$.

Insurance secret and blind issuance. The insurance secret is a secret key sk^{in} associated with a public insurance key $\text{pk}^{\text{in}} = \text{sk}^{\text{in}} G_1$. In order to bind sk^{in} to a token, it is embedded into the token's signature together with a mask $r \xleftarrow{\$} \mathbb{Z}_p$, i.e., $\sigma = \frac{1}{z+e}(G_0 + rH_0 + \text{sk}^{\text{in}} H_1)$. We call σ the token and (r, e) the witness denoted as ω .

To prevent an issuer from learning the token, which would harm the unlinkability guarantees of the construction, issuers are required to sign tokens blindly. Clients send a signing request $T = \delta(G_0 + rH_0 + \text{sk}^{\text{in}} H_1)$ for $\delta \xleftarrow{\$} \mathbb{Z}_p$ and issuers provide a blind signature $S = \frac{1}{z+e}T$ which is unblinded to token $\sigma = \frac{1}{\delta}S$. To prevent maliciously crafted requests, we require that clients append zero-knowledge proofs of correct request creation to their message.

Naive token redemption. The naive way to realize the token redemption is to send the token and the public identification key pk^{ld} together with a zero-knowledge proof of knowledge of the witnesses $(e, r, \text{sk}^{\text{in}}, \text{sk}^{\text{ld}})$ to the verifier.

While this naive approach is sufficient for schemes not considering non-transferability, it does not ensure that the client submitting the proof, the buyer, indeed knows the witnesses. The proof can also be generated by another client, the vendor, and be transferred to the buyer together with the token. Even if the zero-knowledge proof is interactive and needs to be generated based on a challenge issued by the verifier, as done by [7], it is possible for the buyer to still redeem the token without knowledge of the witnesses by forwarding the challenge to the vendor and sending the vendor's response to the verifier.

Token redemption with betrayability. We mitigate the problems of the naive approach by including a trapdoor into the token redemption that can be used by a token buyer to betray a vendor and steal the vendor's insurance key. In particular, we design a three-message redemption protocol as follows. The redemption is initiated by the client by sending the token σ and its identity public key pk^{ld} . The verifier responds with an encryption key K . The client encrypts the secret insurance key sk^{ln} under K and sends the ciphertexts together with a zero-knowledge proof ensuring the verifier that the client knows the witnesses $(e, r, \text{sk}^{\text{ln}}, \text{sk}^{\text{ld}})$ and that the ciphertexts have been created correctly. If a buyer redeeming a token wants to steal the associated insurance key from a token vendor assisting in the proof creation, the buyer sends an encryption key K' , for which it knows the secret decryption key k' , to the vendor. If the vendor creates a valid proof, the client can use the decryption key k' to extract the insurance secret sk^{ln} .

To ensure that the verifier cannot retrieve the insurance secret sk^{ln} while still guaranteeing that a buyer can do so, we require that the verifier proves that K is useless, i.e., that it does not know the corresponding decryption key, in a way that can only be verified by the client redeeming a token. Technically, we require the verifier to prove that either K is useless or it knows the secret identification key sk^{ld} ($\text{pk}^{\text{ld}} = \text{sk}^{\text{ld}} G_1$). As the verifier does not know sk^{ld} , it can only send a useless key K to the client. However, the buyer knowing its identification key sk^{ld} , can replace K with K' and forge a valid proof. Although the vendor could prevent the betrayal by specifying the identity key for which the token should be redeemed, this would prevent the token buyer from redeeming the token for one of its own identities and, hence, render the bought token useless.

Zero-knowledge friendly trapdoor. We utilize the exponential ElGamal cryptosystem [22] for the encryption of the insurance secret as it harmonizes well with the Schnorr-style [23] zero-knowledge proofs we utilize in our construction. The client computes a ciphertext of a message x under public key K as $(A = r'G_1, B = r'K + xG_1)$. Further, we realize the proof of uselessness of K by sampling K as valid ElGamal key but with respect to generator G_0 , i.e., $K = kG_0$, and proving knowledge of $k = \text{dlog}_{G_0}(K)$. As ciphertexts are created with respect to G_1 , the adversary would have to know $\text{dlog}_{G_1}(G_0)$ to compute the decryption key associated with K .

Unfortunately, exponential ElGamal does not allow for direct decryption of high entropy secrets. The buyer can

compute xG_1 but would need to be able to compute $\text{dlog}_{G_1}(xG_1)$ to retrieve x . If x has low entropy it can be retrieved with brute-forcing techniques. This is not possible for sk^{ln} . To address this problem, we split the bit representation of sk^{ln} into chunks and encrypt the chunks individually, i.e., we split sk^{ln} into $x_1, \dots, x_L$ such that $x_i \in [0, 2^\ell - 1]$ and $\text{sk}^{\text{ln}} = \sum_{i \in [L]} 2^{(i-1)\ell} x_i$ and pick ℓ such that $\text{dlog}_{G_1}(x_i G_i)$ can be computed efficiently. To ensure that ciphertexts can be decrypted, the client does not only prove correct creation of the ciphertexts but also that the chunks are in the correct range and add up to the insurance key sk^{ln} . In our evaluation, we consider instantiations of the range proof based on Bulletproof++ [24] and Groth16 general-purpose SNARKs [25].

Pushing the bottleneck to the issuance phase. We assess the issuance phase to be less efficiency critical than the redemption phase. This is due to the fact that the issuance phase usually includes the verification of the client's eligibility to receive a token such as the solving of a CAPTCHA. Therefore, we aim to push the major protocol overhead of our construction, the creation of the range proofs, to the issuance phase. We do so by including the insurance key's chunk decomposition instead of the key itself into the token. BBS signatures support the efficient creation of signatures on message arrays without an increase in the signature size and, hence, the token size. When sending a token request, the client proves in zero-knowledge that the request contains the chunk decomposition of the insurance secret corresponding to its public insurance key and that all chunks are in the correct range. During redemption, the client only needs to prove that the provided ciphertexts match the secret key chunks embedded into its token.

1.3. Related Work

The concept of anonymous tokens is closely related to the long-studied concept of anonymous credentials. In the following, we provide an overview of the literature on anonymous credentials and anonymous tokens.

Anonymous credentials. Anonymous credentials have first been introduced by Chaum [26]–[28] in the form of blind signatures and subsequently refined by a long line of work including [17], [19], [29]–[39]. An anonymous credential can be used to prove statements about the credential holder based on attributes encoded into the credential. In contrast to anonymous tokens, anonymous credentials are designed to be used multiple times. The credential ensures that a utilization of the credential does not leak any information about the holder or the credential's attributes beyond the proven statement. Furthermore, it is guaranteed that a utilization of a credential cannot be linked to its issuance or any other utilization of the same credential. Anonymous credentials are usually realized as blindly issued signatures on lists of attributes. A popular signature scheme for building anonymous credentials is the BBS(+) signature scheme studied in [18], [20], [21], [40]–[42]. The features making BBS(+) signatures appealing for anonymous credentials are the capability to sign arrays of messages without increasing

the signature size, the support of blind signature issuance with minimal overhead, and the existence of efficient zero-knowledge proofs for the disclosure of individual attributes.

Non-transferability in the context of anonymous credentials has been studied by [17], [30], [38], [39]. In [17], the authors introduced the concept of insurance secrets previously known from [15], [16] to the domain of anonymous credentials. Subsequently, [30] proposed an alternative notion of non-transferability, in which a client can only share one of its credentials by sharing all of its credentials. The credentials the client is not willing to share act as an insurance disincentivizing the transfer of the credentials it is willing to share. UProve [39], is an industrial instantiation of anonymous credentials that adapts the insurance secret approach of [17] to achieve non-transferability. While UProve is highly efficient, it only supports one-show credentials (similar to tokens) and lacks a formal security proof. An alternative to insurance secrets is to store the credential within a secure hardware device. In [38], the authors proposed an anonymous credential scheme tailored to resource constrained secure hardware devices. In their protocol, the secure hardware device generates the proofs to use the credential, but relies on the unprotected hardware to improve efficiency. None of the works on non-transferable credentials considers the risk of a collusion between the credential vendor and buyer during the credential's utilization. While we assess that a vendor-aided utilization of credentials can be prevented with the use of secure hardware, we are interested in a solution that can be implemented in software.

Anonymous tokens. Anonymous tokens have been studied by [1]–[11]. One approach adopted by [1], [2], [4], [9] is to construct anonymous tokens based on Oblivious Pseudo Random Functions (OPRFs) [43]. While [1] formally introduced the concept of anonymous tokens, [2] extended their work with private metadata bits and [4] added public metadata and public verifiability. Subsequently, [9] proposed an OPRF-based token scheme providing public verifiability without requiring bilinear pairings. An alternative approach to OPRFs proposed by [3], [8], [10], [11] is to instantiate publicly verifiable anonymous tokens based on blind signatures [26], [27]. In particular, [3] utilizes blind Schnorr signatures [23], [44] and supports private metadata bits, [8] presents constructions based on blind BLS multi-signatures [45]–[47] and Snowblind [48] supporting a decentralized issuance of tokens, [10] proposes a construction supporting public metadata that is based on RSA signatures [49], and [11] utilizes so-called non-interactive blind signatures [50] to enable the issuance of tokens independently of a specific client request.

Another line of work [6], [7] instantiates anonymous tokens without public verifiability based on algebraic message authentication codes (MACs). Technically, the MACs are realized as blindly issued BBS signatures in a group without a pairing and, hence, without public verifiability of signatures. In [6], the authors identified weaknesses in the secrecy of the metadata bit in [2], [4] and proposed a MAC-based construction with a private metadata bit providing stronger secrecy guarantees. Subsequently, Durak et

al. [7] introduced the notion of non-transferability to the domain of anonymous tokens by adapting the insurance secret technique from [17], [39] to an algebraic MAC-based anonymous token construction derived from [19].

As [7] is the work most closely related to ours, we will discuss it in more detail. The authors extend prior definitions of anonymous tokens with a non-transferability property that requires clients to embed an insurance secret into tokens during issuance, which must also be known during redemption. To formalize this property, they require the existence of an extractor such that, for every client capable of successfully redeeming a token, the extractor—given black-box access to the client—can recover the corresponding insurance secret. The authors instantiate non-transferable anonymous tokens by realizing tokens as blindly issued BBS-based MACs on the insurance secret and including an interactive zero-knowledge proof of knowledge of the redeemed token's secret into the redemption protocol. The interactivity ensures freshness of the proof, making it impossible for a vendor to precompute a proof and transfer it together with the token. While the primary construction of [7] requires the issuer and the verifier to be the same party, the authors propose an extension with public verifiability that replaces the BBS-based MACs with BBS signatures. Furthermore, they show how their scheme can be extended to support public metadata.

Unfortunately, Durak et al. [7] do not consider a vendor-aided token redemption. If a vendor assists a buyer during the redemption of a transferred token, it can help generate the proof without revealing the insurance secret to the buyer, despite the proof being interactive. When considering such an attack, the extractor in the security definition effectively extracts the secret from the vendor rather than the buyer. Our work builds on [7] but addresses this limitation. Conceptually, we replace the non-transferability property with a betrayability property, which explicitly models collusion between a token vendor and a token buyer during redemption and requires the buyer to be able to steal the insurance secret from the vendor. Technically, we adopt the approach of the public verifiable protocol variant of [7], realizing tokens as blindly issued BBS signatures. However, we extend the token structure and embed a trapdoor into the redemption protocol that can be exploited only in a vendor-aided token redemption and enables the buyer to steal the vendor's secret. As in [7], our scheme can be extended to support public metadata by encoding additional data into tokens, as explained in Appendix E.

2. Preliminaries

We denote the security parameter by $\lambda \in \mathbb{N}$, the set $\{1, \dots, n\}$ as $[n]$, and the set $\{a, a+1, \dots, b-1, b\}$ as $[a, b]$. All considered sets are ordered multisets. We denote a negligible function by negl and computational indistinguishability of two distributions by $\approx_c$.

We use bilinear pairings as defined by [51]. A bilinear pairing group is described by three cyclic groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ of prime order p , generators $G_1 \in \mathbb{G}_1, G_2 \in$

$\mathbb{G}_2$ and a mapping $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$. The map e is called a bilinear map iff it can be computed efficiently, $e(aU, bV) = (ab)e(U, V)$ for all $(U, V, a, b) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{Z}_p \times \mathbb{Z}_p$, and $e(G_1, G_2) \neq 1$ for all generators G_1 and G_2 . To generate a bilinear pairing group, we make use of a group generation algorithm GGen that on input 1^λ outputs a group description $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, where p is a λ -bit prime and the groups $\mathbb{G}_1, \mathbb{G}_2$ and $\mathbb{G}_T$ are of order p . We denote by $\mathbb{G}^* = \mathbb{G} \setminus \{1_{\mathbb{G}}\}$ the set of all generators of $\mathbb{G}$ for $\mathbb{G} \in \{\mathbb{G}_1, \mathbb{G}_2\}$. In this work, we consider Type-2 or Type-3 pairings, i.e., pairings, in which no efficient homomorphism from $\mathbb{G}_1$ to $\mathbb{G}_2$ is known. For every bilinear pairing group $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \text{GGen}(1^\lambda)$ we assume the discrete logarithm to be hard in $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ and the external Diffie-Hellman (XDH) assumption to hold in $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$. The latter requires the decisional Diffie-Hellman (DDH) assumption to hold in $\mathbb{G}_1$. We denote an algorithm that attempts to compute the discrete logarithm of element $H \in \mathbb{G}_1$ to the basis G with a brute-forcing strategy over exponents $x \in [0, 2^i]$ by $\text{dlog}_G(h)[0, 2^i]$.¹ We abbreviate probabilistic polynomial time algorithms by PPT and denote the execution of a PPT algorithm on input b by $a \leftarrow \mathcal{A}(b)$. To fix the random tape r of algorithm $\mathcal{A}$, we write $a \leftarrow \mathcal{A}(b; r)$. Finally, we treat $\frac{1}{0}$ as 0.

2.1. Zero-Knowledge Proofs of Knowledge

Zero-knowledge proofs of knowledge allow a prover to convince a verifier that it knows a witness w for a statement s of a NP-language $\mathcal{L}_{\mathcal{R}}$ with associated relation $\mathcal{R}$ such that $(s, w) \in \mathcal{R}$. We use the Camenisch-Stadler notation [54] $\text{PK}_{\mathcal{R}} = [w : \mathcal{R}(s, w) = 1]$ to denote the relation of a proof system proving knowledge of a witness w such that $\mathcal{R}(s, w) = 1$. All components of the condition $\mathcal{R}(s, w) = 1$ that are not explicitly declared as witnesses are assumed to be part of the statement. We define non-interactive zero-knowledge proofs in the random oracle model as follows.

Definition 1. Let $\mathcal{R}$ be a NP-relation, $\mathcal{L}_{\mathcal{R}} = \{s : (s, w) \in \mathcal{R}\}$ be the associated NP-language and $\mathcal{H}$ be a global random oracle. A non-interactive zero-knowledge proof of knowledge for relation $\mathcal{R}$ consists of the following two polynomial time algorithms (Prove, Verify) with access to a random oracle $\mathcal{H}$.

- $\text{Prove}(s, w) \rightarrow \pi$: a probabilistic algorithm that takes as input a statement s and a witness w and outputs a proof π .
- $\text{Verify}(s, \pi) \rightarrow 0/1$: a deterministic algorithm that takes as input a statement s and a proof π and outputs either 1 accepting the proof or 0 rejecting it.

The algorithms satisfy the following properties:

- **Completeness:** For all $(s, w) \in \mathcal{R}$, it holds that

$$\Pr [\text{Verify}(s, \pi) = 1 : \pi \leftarrow \text{Prove}(s, w)] = 1.$$

1. There are algorithms such as baby-step giant-step [52] or Pollard's Kangaroo [53] that offer better efficiency than the naive approach of testing all exponents in the range individually.

Experiment: $\text{Exp}_{\mathcal{A}, L}^{\text{P-BBS-SUF}}(\lambda)$

```

1:  $\text{pp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \text{GGen}(1^\lambda), G_0, G_1 \xleftarrow{\$} \mathbb{G}_1^*$ 
2:  $\{H_i\}_{i \in [L]} \xleftarrow{\$} \mathbb{G}_1^L, G_2 \leftarrow \mathbb{G}_2^*, z \xleftarrow{\$} \mathbb{Z}_p, Z \leftarrow zG_2, \mathcal{S} \leftarrow \emptyset$ 
3:  $(\{m_i^*\}_{i \in [L]}, A^*, e^*) \leftarrow \mathcal{A}^{\text{Sign}}(\text{pp}, G_0, G_1, \{H_i\}_{i \in [L]}, G_2, Z)$ 
4: if  $e(A^*, eG_2 + Z) \neq e(G_0 + \sum_{i \in [L]} m_i^* H_i, G_2)$  then return 0
5: if  $(\{m_i^*\}_{i \in [L]}, A^*, e^*) \in \mathcal{S}$  then return 0 else return 1

 $\mathcal{O}^{\text{Sign}}(\{m_i\}_{i \in [L]}):$ 
6:  $e \xleftarrow{\$} \mathbb{Z}_p, A \leftarrow \frac{1}{z+e}(G_0 + \sum_{i \in [L]} m_i H_i)$ 
7:  $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\{m_i\}_{i \in [L]}, A, e)\}$ , return  $(A, e)$ 

```

Figure 1: Security game for strong unforgeability of BBS signatures.

- **Knowledge soundness:** For every $\lambda \in \mathbb{N}$, every auxiliary input $\text{aux} \in \{0, 1\}^*$ of size polynomial in λ , every PPT adversary $\mathcal{A}$, and every random tape r , there exists a PPT extractor $\mathcal{E}$ with oracle access to $\mathcal{A}$ and control over random oracle $\mathcal{H}$, as well as a negligible function negl , such that

$$\Pr \left[\begin{array}{l} \text{Verify}(s, \pi) = 1 \\ \wedge (s, w) \notin \mathcal{R} \end{array} : \begin{array}{l} (s, \pi) \leftarrow \mathcal{A}(\text{aux}; r) \\ w \leftarrow \mathcal{E}^{\mathcal{H}, \mathcal{A}}(\text{aux}, r) \end{array} \right] \leq \text{negl}(\lambda).$$

Since $\mathcal{A}$ is polynomial-time and its input size is polynomial in the security parameter λ , the definition implicitly requires the size of statement s and proof π to be polynomial in λ .

- **Zero-knowledge:** There exists a PPT simulator $\mathcal{S}$ controlling random oracle $\mathcal{H}$ such that for every $\lambda \in \mathbb{N}$ and for every $(s, w) \in \mathcal{R}$ of size polynomial in λ , it holds that

$$\{\text{Prove}(s, w)\} \approx_c \{\mathcal{S}^{\mathcal{H}}(1^\lambda, s)\}.$$

Remarks. Formally, a proof requires perfect soundness while an argument tolerates a negligible soundness error. However, in this work, we will utilize the term proof for both arguments and proofs. As all the proofs of knowledge, we utilize in our constructions, rely on the random oracle model, we choose to incorporate the random oracle into the formalization in order to ease the definition. One of the proof systems of our construction incorporates a range proof, which we will realize with the Bulletproof++ [24] proof system. This proof system requires a dedicated setup. We formalize zero-knowledge proofs of knowledge with setup in Appendix C.

2.2. BBS unforgeability

We base our construction on BBS signatures, as discussed in the technical overview (Section 1.2). However, in order to efficiently instantiate our zero-knowledge proofs, we make non-black box use of BBS signatures. Therefore, we do not need a formal definition of the abstract primitive instantiated by BBS signatures. Instead, we state Theorem 1.

Theorem 1. Let $GGen$ be as above. For every $\lambda \in \mathbb{N}$, $L = L(\lambda)$ ($L \in \mathbb{N} : L > 1$) and PPT adversary $\mathcal{A}$ there exists a negligible function negl such that

$$\Pr[\text{Exp}_{\mathcal{A},L}^{\text{P-BBS-SUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

where $\text{Exp}_{\mathcal{A},L}^{\text{P-BBS-SUF}}$ is as defined in Figure 1.

Theorem 1 is straightforwardly implied by Theorem 1 in [18].

3. Anonymous Tokens with Betrayability

Analogously to [7], *anonymous tokens with betrayability* (ATB) deter clients from transferring tokens by binding each token to a valuable insurance secret that needs to be known during both token issuance and token redemption. Even though a token vendor might be willing to sell a token, it would not sell the corresponding secret due to its value, which renders transferred tokens useless. While [7] assumed that the token buyer and the vendor cannot communicate during the token redemption, we get rid of this assumption. Instead, we guarantee that a client redeeming a token either knows the corresponding insurance secret or is able to include a trapdoor into a vendor-aided token redemption that allows it to steal the vendor's insurance secret without being detected.

Formally, we define ATB as below. In the following we denote both the token issuer and token verifier as server even though they can be instantiated by different parties. As our definition extends the one of [7], we take parts of their definition (almost) verbatim. We discuss the differences in the notions in Appendix B.

Definition 2. An anonymous token with betrayability scheme is defined as a tuple of the following PPT algorithms and protocols:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: a randomized algorithm that takes as input the security parameter λ and returns public parameters pp .
- $\text{S.KeyGen}(\text{pp}) \rightarrow (\text{sk}_s, \text{pk}_s)$: a randomized algorithm that takes as input the public parameters and generates a key pair for the server.
- $\text{C.KeyGen}(\text{pp}) \rightarrow (\text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}})$: a randomized algorithm that takes as input the public parameters and generates two key pairs for the client, one for insurance and one for identification.
- $\text{C.Query}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{pk}_s) \rightarrow (\text{query}, \text{st})$: a randomized algorithm constituting the first step of an interactive issuance protocol between client and server. The client uses the public parameters, its insurance keys and the server's public key to generate a token request and saves its state st for the token finalization.
- $\text{S.Issue}(\text{pp}, \text{sk}_s, \text{pk}_c^{\text{In}}, \text{query}) \rightarrow \text{resp}/\perp$: a randomized algorithm constituting the second step of the issuance protocol. The server takes the public parameters, its key pair, the client's public insurance key and the client query as input and produces a response resp . If the query is not valid, the algorithm returns $\perp$.

- $\text{C.Final}(\text{pp}, \text{st}, \text{resp}) \rightarrow (\sigma, \omega)$: a deterministic algorithm constituting the third and final step of the issuance protocol. The client takes the public parameters, the previous state and the server's response as input and generates a token σ as well as a witness ω .
- $\langle \text{C.Red}(\text{pp}, \sigma, \omega, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}, \text{pk}_s), \text{S.Red}(\text{pp}, \text{pk}_s) \rangle \rightarrow 1/0$: An interactive protocol executed between the client and the server, in which the client, identified by its identification public key pk_c^{Id} , shows a token σ to the server and proves that it knows a corresponding witness ω . We assume that the protocol is initiated by the client by sending $(\sigma, \text{pk}_c^{\text{Id}})$ to the server.² Formally, the computations of each party are modeled via a next message randomized PPT algorithm $\text{S.NextMsg resp. C.NextMsg}$ that takes an instance specific state of the party and an incoming message as input and produces a new state, an outgoing message, and a flag. If not required, we neglect the flag. The initial state is the input to the protocol. If the flag indicates a protocol termination, i.e., $\text{flag} = 1$, the protocol ends. The output 0 or 1 is the new state and the final message (if any) is the new message. To simplify the security games, we assume that the intermediate states cannot be 0 or 1 and that the server sends its output as final message.
- $\text{KeyV}(\text{pp}, \text{sk}, \text{pk}) \rightarrow 1/0$: a deterministic algorithm that takes as input the public parameters and a key pair and outputs 1 iff the key pair is valid.

An ATB scheme needs to satisfy correctness (Definition 3), accountability (Definition 4), unlinkability (Definition 5) and betrayability (Definition 6).

We define the correctness property as follows:

Definition 3 (Correctness). We call an ATB scheme correct, if for all $\lambda \in \mathbb{N}$, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $(\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp})$, $(\text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}) \leftarrow \text{C.KeyGen}(\text{pp})$ it holds that if $(\sigma, \omega) \leftarrow \text{C.Final}(\text{st}, \text{resp})$ for $(\text{query}, \text{st}) \leftarrow \text{C.Query}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{pk}_s)$ and $\text{resp} \leftarrow \text{S.Issue}(\text{pp}, \text{sk}_s, \text{pk}_c^{\text{In}}, \text{query})$ then

$$\begin{aligned} \text{KeyV}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}) &= \text{KeyV}(\text{pp}, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}) = 1 \\ \langle \text{C.Red}(\text{pp}, \sigma, \omega, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}, \text{pk}_s), \text{S.Red}(\text{pp}, \text{pk}_s) \rangle &= 1. \end{aligned}$$

3.1. Accountability

Accountability requires three properties, *one-more unforgeability* (OMUF), *request unforgeability* (RUF), and *identity unforgeability* (IUF). One-more unforgeability states that it is impossible for an adversary to redeem q tokens without making q successful token queries. Request unforgeability states that an adversary cannot request a token under the insurance key of an honest client. Identity unforgeability requires it to be impossible for an adversary to redeem a token under the identity of an honest client. Note that these

2. The protocol does not take any additional states as input or alter the parties' long-time states. If such effects need to be captured, e.g., to prevent tokens from being used multiple times, they need to be captured by the programs surrounding the anonymous token scheme.

properties imply that secret keys of honest parties cannot be leaked. This is due to the fact that any adversary capable of stealing a secret key from an honest party can either forge tokens or request or redeem a token in the name of another honest party. Formally, we define accountability as follows.

Definition 4. An ATB scheme is accountable if it satisfies one-more unforgeability, token consistency, request unforgeability, and identity unforgeability, i.e., if for every PPT adversary $\mathcal{A}$ there exists a negligible function negl such that $\Pr[\text{Exp}_{\mathcal{A}}^*(\lambda) = 1] \leq \text{negl}(\lambda)$ for $\text{Exp}^* \in \{\text{Exp}_{\mathcal{A}}^{\text{OMUF}}, \text{Exp}_{\mathcal{A}}^{\text{RUF}}, \text{Exp}_{\mathcal{A}}^{\text{IUF}}\}$ where $\text{Exp}_{\mathcal{A}}^{\text{OMUF}}, \text{Exp}_{\mathcal{A}}^{\text{RUF}}$ and $\text{Exp}_{\mathcal{A}}^{\text{IUF}}$ are as defined in Figure 2.

3.2. Unlinkability

Intuitively, *unlinkability (UL)* requires that a malicious server is not able to link the issuance of a token to its redemption, even if it controls both the verifier and the issuer. Formally, we define a security game in which the adversary has full control over both the issuer and the verifier. The adversary can initiate issuance sessions, each of which returns a unique identifier, and can later initiate redemption sessions for tokens using the identifiers of the corresponding issuance sessions. We require that the adversary cannot determine whether two tokens—selected based on their identifiers—have been swapped.

Definition 5. An ATB scheme is unlinkable if for every PPT adversary $\mathcal{A}$, there exists a negligible function negl such that $|\Pr[\text{Exp}_{\mathcal{A}}^{\text{UL}}(\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda)$ where $\text{Exp}_{\mathcal{A}}^{\text{UL}}$ is defined as in Figure 3.

3.3. Betrayability

Intuitively, *betrayability (BT)* requires that a client redeeming a bought token, potentially with the help of the token vendor, can *betray* the vendor and steal the insurance secret associated with the token.

Formally, we define a security game with two adversaries, the token vendor and the token buyer. The vendor first specifies a particular token it aims to sell, (σ^*) , and the token buyer an identification key pair it aims to redeem the token with, $(\text{sk}_*, \text{pk}_*)$. Afterwards, the adversaries engage in an interactive protocol. Whenever, the adversaries are executed, they also have access to a token request and redemption oracle representing concurrent interaction of the parties with the server. The security game has two modes. In the colluding mode ($b = 0$), the buyer does not betray the vendor and the adversaries win if the buyer successfully redeems σ^* with identification key pk_*^{ld} . In the betraying mode ($b = 1$), we replace the token buyer with an extractor that attempts to steal the insurance secret embedded into σ^* from the vendor. Since the game cannot directly verify whether a given insurance secret belongs to σ^* , we keep track of all the insurance public keys used by the vendor prior to specifying σ^* and consider the extractor to be successful if it extracts an insurance secret corresponding to

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{OMUF}}(\lambda)$
1: $q_{\text{Rq}}, q_{\text{Rd}} \leftarrow 0, \Gamma \leftarrow \emptyset, \text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp})$ 2: $\mathcal{A}^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, \text{pk}_s)$, if $q_{\text{Rq}} < q_{\text{Rd}}$ return 1 else return 0 $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query})$: 3: $q_{\text{Rq}} \leftarrow q_{\text{Rq}} + 1$, return $\text{S.Issue}(\text{pp}, \text{sk}_s, \text{pk}, \text{query})$ $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 4: if $\text{st}_{\text{sid}} \in \{0, 1\}$ then return $\perp$ 5: if $\text{st}_{\text{sid}} = \text{null}$ then $\text{st}_{\text{sid}} \leftarrow (\text{pp}, \text{pk}_s), (\sigma_{\text{sid}}, \cdot) \leftarrow m$ 6: $(\text{st}, m', \text{flag}) \leftarrow \text{S.NextMsg}(\text{st}_{\text{sid}}, m)$ 7: if $\text{flag} = \text{st} = 1 \wedge \sigma_{\text{sid}} \notin \Gamma$ then $q_{\text{Rd}} \leftarrow q_{\text{Rd}} + 1, \Gamma \leftarrow \Gamma \cup \{\sigma_{\text{sid}}\}$ 8: $\text{st}_{\text{sid}} \leftarrow \text{st}$, return m'
Experiment: $\text{Exp}_{\mathcal{A}}^{\text{RUF}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 2: $(\text{query}, \text{sk}_s) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, \text{pk}_c^{\text{In}}, \text{pk}_c^{\text{ld}})$ 3: $\text{resp} \leftarrow \text{S.Issue}(\text{pp}, \text{sk}_s, \text{pk}_c^{\text{In}}, \text{query})$ 4: if $\exists i$ s.t. $\text{query} = \text{query}_i \vee \text{resp} = \perp$ then return 0 5: return 1 $\mathcal{O}^{\text{Qu}}(i, \text{pk}_s)$: 6: if $\text{query}_i \neq \text{null}$ then abort 7: $(\text{query}_i, \text{st}_i^{\text{Q}}) \leftarrow \text{C.Query}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{ld}}, \text{pk}_s)$, $\text{pk}_i \leftarrow \text{pk}_s$ 8: return query_i $\mathcal{O}^{\text{Fin}}(i, \text{resp})$: 9: if $\text{query}_i = \text{null}$ or $\sigma_i \neq \text{null}$ then abort 10: $(\sigma_i, \omega_i) \leftarrow \text{C.Final}(\text{pp}, \text{st}_i^{\text{Q}}, \text{resp})$, return σ_i $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 11: if $\text{st}_{\text{sid}}^{\text{R}} = \text{null}$ then 12: $i \leftarrow m$, if $\sigma_i \in \{\text{null}, \perp\}$ then return $\perp$ 13: $\text{st}_{\text{sid}}^{\text{R}} \leftarrow (\text{pp}, \sigma_i, \omega_i, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}, \text{pk}_i)$, return $(\sigma_i, \text{pk}_c^{\text{ld}})$ 14: else $(\text{st}_{\text{sid}}^{\text{R}}, m') \leftarrow \text{C.NextMsg}(\text{st}_{\text{sid}}^{\text{R}}, m)$, return m'
Experiment: $\text{Exp}_{\mathcal{A}}^{\text{IUF}}(\lambda)$
1: $\text{res} \leftarrow 0, \text{pp} \leftarrow \text{Setup}(1^\lambda), (\cdot, \cdot, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 2: $(\text{pk}_s, \text{st}^{\text{A}}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(1, \text{pp}, \text{pk}_c^{\text{ld}}), \mathcal{A}^{\mathcal{O}^*}(2, \text{st}^{\text{A}})$ 3: return res $\mathcal{O}^{\text{Qu}}(i, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{In}}, \text{pk})$: 4: if $\text{query}_i \neq \text{null} \vee \text{KeyV}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}) = 0$ then return $\perp$ 5: $(\text{query}_i, \text{st}_i^{\text{Q}}) \leftarrow \text{C.Query}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{ld}}, \text{pk})$, $\text{pk}_i \leftarrow \text{pk}$ 6: return query_i $\mathcal{O}^{\text{Fin}}(i, \text{resp})$: 7: if $\text{query}_i = \text{null}$ or $\sigma_i \neq \text{null}$ then abort 8: $(\sigma_i, \omega_i) \leftarrow \text{C.Final}(\text{pp}, \text{st}_i^{\text{Q}}, \text{resp})$, return σ_i $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 9: if $\text{st}_{\text{sid}}^{\text{R}} = \text{null}$ then 10: $i \leftarrow m$, if $\sigma_i \in \{\text{null}, \perp\}$ then return $\perp$ 11: $\text{st}_{\text{sid}}^{\text{R}} \leftarrow (\text{pp}, \sigma_i, \omega_i, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}, \text{pk}_i)$, return $(\sigma_i, \text{pk}_c^{\text{ld}})$ 12: else $(\text{st}_{\text{sid}}^{\text{R}}, m') \leftarrow \text{C.NextMsg}(\text{st}_{\text{sid}}^{\text{R}}, m)$, return m' $\mathcal{O}^*(m)$: 13: if $\text{st} \in \{0, 1\}$ then return $\perp$ 14: if $\text{st} = \text{null}$ then $\text{st} \leftarrow (\text{pp}, \text{pk}_s)$, replace pk^* in m by pk_c^{ld} 15: $(\text{st}, m', \text{flag}) \leftarrow \text{S.NextMsg}(\text{st}, m)$ 16: if $\text{flag} = \text{st} = 1$ then $\text{res} \leftarrow 1$, return 1, else return m'

Figure 2: One-more unforgeability, request unforgeability and identity unforgeability game.

any of the vendor's insurance public keys. For betrayability, we require that the extractor's success probability in the betraying mode is at least as high as the adversaries' winning probability in the colluding mode.

Definition 6. An ATB scheme is betrayable if for every pair of PPT adversaries $\mathcal{A}_B$ and $\mathcal{A}_V$, there exists an extractor $\mathcal{E}$ with black-box access to $\mathcal{A}_B$, integer $n = n(\lambda)$ ($n \in \mathbb{N}$) and negligible function negl such that

$$\Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 1}^{\text{BT}}(\lambda) = 1] \geq \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 0}^{\text{BT}}(\lambda) = 1] - \text{negl}(\lambda)$$

where $\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, b}^{\text{BT}}$ is defined as in Figure 3.

4. Our ATB scheme

This section presents our construction for anonymous tokens with betrayability. We refer the reader to the technical overview (cf. Section 1.2) for a high-level description of our construction and focus on the technical presentation in the following.

4.1. Zero-Knowledge Proofs

We make use of three different zero-knowledge proof systems. We introduce the proof systems in the following and present instantiations in Appendix D. We slightly abuse the notation and omit statement components that are part of the protocol's public parameters pp when invoking algorithms $\text{PK}_*. \text{Prove}$ and $\text{PK}_*. \text{Verify}$.

All of the proof systems but the range proofs used within the request proof are instantiated with generalized Schnorr proofs [23]. The range proofs are instantiated with the Bulletproof++ proof system [24]. In the evaluation, we also consider instantiations of the range proofs based on Groth16 generalized purpose SNARKs [25].

Request proof. When sending a request, the client has to prove that the request is well-formed, i.e., that the request's blinded commitment T contains the chunk decomposition $\{x_i\}_{i \in [L]}$ of the secret insurance key sk^{In} and that all chunks are in the expected range. Formally, we define the request proof relation as

$$\text{PK}_{\text{Rq}} = \text{PK} \left[\begin{array}{l} (\{x_i\}_{i \in [L]}, r, \delta) : \\ \text{pk}_c^{\text{In}} = (\sum_{i \in [L]} 2^{(i-1)\ell} x_i) G_1 \\ T = \delta(G_0 + rH_0 + \sum_{i \in [L]} x_i H_i) \\ x_i \in [0, 2^\ell - 1] \text{ for } i \in [L] \end{array} \right].$$

As we will instantiate the range proofs with the Bulletproof++ system, we require an explicit setup, which fixes the statement components $(p, G_1, G_0, H_0, L, \ell)$. The setup returns proof parameters rpp that are input to both $\text{PK}_{\text{Rq}}. \text{Prove}$ and $\text{PK}_{\text{Rq}}. \text{Verify}$. We assume $(\text{PK}_{\text{Rq}}. \text{Setup}, \text{PK}_{\text{Rq}}. \text{Prove}, \text{PK}_{\text{Rq}}. \text{Verify})$ to be a zero-knowledge proof of knowledge with setup for relation PK_{Rq} .

Challenge proof. During redemption, the server sends an encryption key $K = kG_0$ to the client and proves that

Experiment: $\text{Exp}_{\mathcal{A}}^{\text{UL}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda), j' \leftarrow 0, b \xleftarrow{\$} \{0, 1\}, \Gamma \leftarrow \emptyset$ 
2:  $(\text{pk}_s, \text{st}_0^A) \leftarrow \mathcal{A}^{\text{OKG}}(0, \text{pp})$ 
3:  $(\alpha, \beta, \text{st}_1^A) \leftarrow \mathcal{A}^{\text{OKG}, \mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(1, \text{st}_0^A)$ 
4: if  $\exists k \in \{\alpha, \beta\}$  s.t.  $\sigma_k \in \{\text{null}, \perp\} \vee k \in \Gamma$  then abort
5: if  $b = 1$  then swap values of  $(\sigma_\alpha, \omega_\alpha)$  and  $(\sigma_\beta, \omega_\beta)$ 
6:  $b' \leftarrow \mathcal{A}^{\text{OKG}, \mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(2, \text{st}_1^A)$  if  $b = b'$  return 1 else 0
 $\mathcal{O}^{\text{KG}}()$ :
7:  $j' \leftarrow j' + 1, (\text{sk}_{j'}^{\text{ld}}, \text{pk}_{j'}^{\text{ld}}, \text{sk}_{j'}^{\text{ln}}, \text{pk}_{j'}^{\text{ln}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 
8: return  $(\text{pk}_{j'}^{\text{ld}}, \text{sk}_{j'}^{\text{ln}}, \text{pk}_{j'}^{\text{ln}})$ 
 $\mathcal{O}^{\text{Rq}}(i, j)$ :
9: if  $\text{query}_i \neq \text{null}$  or  $\text{sk}_j^{\text{ln}} = \text{null}$  then abort
10:  $(\text{query}_i, \text{st}_i^Q) \leftarrow \text{C.Query}(\text{pp}, \text{sk}_j^{\text{ln}}, \text{pk}_j^{\text{ln}}, \text{pk}_s)$ , return  $\text{query}_i$ 
 $\mathcal{O}^{\text{Fin}}(i, \text{resp})$ :
11: if  $\text{query}_i = \text{null} \vee \sigma_i \neq \text{null}$  then return  $\perp$ 
12:  $(\sigma_i, \omega_i) \leftarrow \text{C.Final}(\text{pp}, \text{st}_i^Q, \text{resp})$ , if  $\sigma_i = \perp$  return 0 else 1
 $\mathcal{O}^{\text{Rd}}(\text{sid}, \text{m})$ :
13: if  $\text{st}_{\text{sid}}^{\text{R}} = \text{null}$  then
14:    $(i, j) \leftarrow \text{m}$ , if  $\sigma_i \in \{\text{null}, \perp\} \vee \text{pk}_j^{\text{ld}} = \text{null}$  then abort
15:    $\Gamma \leftarrow \Gamma \cup \{i\}$ 
16:    $\text{st}_{\text{sid}}^{\text{R}} \leftarrow (\text{pp}, \sigma_i, \omega_i, \text{sk}_j^{\text{ld}}, \text{pk}_j^{\text{ld}}, \text{pk}_s)$ , return  $(\sigma_j, \text{pk}_j^{\text{ld}})$ 
17: else  $(\text{st}_{\text{sid}}^{\text{R}}, \text{m}') \leftarrow \text{C.NextMsg}(\text{st}_{\text{sid}}^{\text{R}}, \text{m})$ , return  $\text{m}'$ 

```

Experiment: $\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, b}^{\text{BT}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp})$ 
2:  $(\mathcal{K}, \mathcal{T}, \text{m}_0^V) \leftarrow \emptyset^3, \text{st}_1 = \{\text{pp}, n, \text{pk}_s\}$ 
3: if  $b = 0$  then  $\tilde{\mathcal{A}}_B \leftarrow \mathcal{A}_B$  else  $\tilde{\mathcal{A}}_B \leftarrow \mathcal{E}^{\mathcal{A}_B}$ 
4:  $(\sigma^*, \text{st}_{(2,0)}^B) \leftarrow \tilde{\mathcal{A}}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(1, \text{st}_1), \mathcal{K}^* \leftarrow \mathcal{K}$ 
5:  $(\text{pk}_{*}^{\text{ld}}, \text{sk}_{*}^{\text{ld}}, \text{st}_{(2,0)}^V) \leftarrow \tilde{\mathcal{A}}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(1, \sigma^*, \text{st}_1)$ 
6: if  $\text{KeyV}(\text{pp}, \text{sk}_{*}^{\text{ld}}, \text{pk}_{*}^{\text{ld}}) = 0$  then return 0
7: for  $i = 1$  until  $i = n$  do
8:    $(\text{m}_i^B, \text{st}_{(2,i)}^B) \leftarrow \tilde{\mathcal{A}}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(2, i, \text{m}_{i-1}^V, \text{st}_{(2,i-1)}^B)$ 
9:    $(\text{m}_i^V, \text{st}_{(2,i)}^V) \leftarrow \tilde{\mathcal{A}}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(2, i, \text{m}_i^B, \text{st}_{(2,i-1)}^V)$ 
10: if  $b = 0 \wedge \text{success} = \text{true}$  then return 1
11: if  $b = 1$ 
12:    $\text{sk}^* \leftarrow \tilde{\mathcal{A}}_B(2, \text{st}_{(3,n)}^B)$ 
13:   if  $\exists \text{pk} \in \mathcal{K}^* : \text{KeyV}(\text{pp}, \text{sk}^*, \text{pk}) = 1$  return 1
14: return 0
 $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query})$ :
19:  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{pk}\}$ 
20: return  $\text{S.Issue}(\text{pp}, \text{sk}_s, \text{pk}, \text{query})$ 
 $\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, \text{m})$ :
21: if  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} = \text{null}$  then
22:    $(\sigma, \text{pk}) \leftarrow \text{m}, \text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow (\text{pp}, \text{sk}_s)$ 
23:   if  $\mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_{*}^{\text{ld}})$  then  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{sid}\}$ 
24:    $(\text{st}_{\text{sid}}^{(S, \mathcal{P})}, \text{m}', \text{flag}) \leftarrow \text{S.NextMsg}(\text{st}_{\text{sid}}^{(S, \mathcal{P})}, \text{m})$ 
25: if  $\mathcal{P} = B \wedge \text{flag} = \text{st}_{\text{sid}}^{(S, \mathcal{P})} = 1 \wedge \text{sid} \in \mathcal{T}$  then success  $\leftarrow \text{true}$ 
26: return  $\text{m}'$ 

```

Figure 3: Unlinkability and betrayability game.

it either knows $k = \text{dlog}_{G_0}(K)$ or $\text{sk}^{\text{ld}} = \text{dlog}_{G_1}(\text{pk}^{\text{ld}})$. The first condition proves that encryption key K is useless as encryptions are created with respect to G_1 . The latter condition allows the holder of sk^{ld} to replace the useless key K with a useful K^* and forge a valid proof. We define the proof relation

$$\text{PK}_{\text{Ch}} = \text{PK}[(d) : K = dG_0 \vee \text{pk}_c^{\text{ld}} = dG_1],$$

where d represents either the decryption key k or the client's secret identification key sk^{ld} . We assume $(\text{PK}_{\text{Ch}}, \text{Prove}, \text{PK}_{\text{Ch}}, \text{Verify})$ to be a zero-knowledge proof of knowledge for relation PK_{Ch} .

Redemption proof. Via the redemption proof, the client proves that it knows the witness ω for the presented token σ , the secret key $y = \text{sk}^{\text{ld}}$ corresponding to the presented public identification key $Y = \text{pk}^{\text{ld}}$ and that the created ElGamal ciphertexts $\{(A_i, B_i)\}_{i \in [L]}$ encrypt the chunks $\{x_i\}_{i \in [L]}$ that have been committed to within σ . As the client does not know the secret BBS signing key z , we cannot directly prove knowledge of the witnesses $(z, r, e, \{x_i\}_{i \in [L]})$ such that $\sigma = \frac{1}{z+e}(G_0 + rH_0 + \sum_{i \in [L]} x_i H_i)$. Instead, we require the client to send $\sigma' = G_0 + rH_0 - e\sigma + \sum_{i \in [L]} x_i H_i$ and proof knowledge of the witnesses $(r, e, \{x_i\}_{i \in [L]})$ required to compute σ' . Note that if σ' is computed as supposed (which is proven by the zero-knowledge proof) and σ is valid it needs to hold that $\sigma' = z\sigma$ which can be verified by the server by checking whether $e(\sigma, Z) = e(\sigma', G_2)$. Therefore, it suffices to use the following proof system:

$$\text{PK}_{\text{Rd}} = \text{PK} \left[\begin{array}{l} (y, r, e, \{r_i, x_i\}_{i \in [L]}) : \\ Y = yG_1 \\ \sigma' = G_0 + rH_0 - e\sigma + \sum_{i \in [L]} x_i H_i \\ A_i = r_i G_1, B_i = r_i K + x_i G_1 \text{ for } i \in [L] \end{array} \right].$$

We assume $(\text{PK}_{\text{Rd}}, \text{Prove}, \text{PK}_{\text{Rd}}, \text{Verify})$ to be a zero-knowledge proof of knowledge for relation PK_{Rd} .

4.2. The construction

Our construction for anonymous tokens with betrayability is specified in Figure 4. In order to betray a token vendor during the token redemption, the buyer replaces the received encryption key K with $K^* = k^* G_1$ for a randomly sampled but known k^* . As the replacement of K invalidates the zero-knowledge proof π_{Ch} , the buyer also replaces π_{Ch} by $\pi_{\text{Ch}}^* = \text{PK}_{\text{Ch}}.\text{Prove}((K^*, \text{pk}_c^{\text{ld}}), \text{sk}_c^{\text{ld}})$. Even though the buyer does not know $\text{dlog}_{G_0}(K^*)$ it does know $\text{dlog}_{G_1}(\text{pk}_c^{\text{ld}})$ and, therefore, can create a valid zero-knowledge proof indistinguishable from the one created by the server. If the vendor (or some jointly executed protocol) returns the ciphertexts required for the token redemption, the buyer verifies the zero-knowledge proof and if successful uses k^* to decrypt the ciphertexts.

To facilitate the recovery of the insurance secret while employing exponential ElGamal for encryption, we split the insurance secret sk^{ln} into chunks $x_1, \dots, x_L \in [0, 2^\ell - 1]$, which are encrypted and decrypted individually. The betraying buyer recovers the secret chunks x_i from the decryptions

Construction 1: BBS-based ATB scheme

The scheme is parameterized with a complexity bound $\mathcal{C}$ describing a trade-off between protocol efficiency and runtime of betraying buyers. It is necessary that computations of complexity $\mathcal{C}$ are feasible for reasonably equipped clients.

Setup(1^λ):

- 1) Compute $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T) \leftarrow \text{GGen}(1^\lambda)$, sample $G_0, G_1 \xleftarrow{\$} \mathbb{G}_1^*, \mathbb{G}_2 \xleftarrow{\$} \mathbb{G}_2^*$ and $H_0, H_1, \dots, H_L \xleftarrow{\$} \mathbb{G}_1$, select ℓ such that $\text{dlog}_{G_1}(G)[0, 2^\ell - 1]$ can be computed with complexity of at most $\mathcal{C}$ for any $G \in \mathbb{G}_1$, and set $L \leftarrow \lceil \frac{p}{\ell} \rceil$.
- 2) Compute $\text{rpp} = \text{PK}_{\text{Rq}}.\text{Setup}(p, \mathbb{G}_1, G_0, H_0, L, \ell)$.
- 3) Return $\text{pp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, G_2, \{H_i\}_{i \in [0, L]}, L, \ell, \text{rpp})$.

C.KeyGen(pp):

- 4) Sample $x, y \xleftarrow{\$} \mathbb{Z}_p$, compute $X = xG_1, Y = yG_1$, and return $(\text{sk}_c^{\text{ln}}, \text{pk}_c^{\text{ln}}, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}) = (x, X, y, Y)$.

S.KeyGen(pp):

- 5) Sample $z \xleftarrow{\$} \mathbb{Z}_p$, compute $Z = zG_2$, and return $(\text{sk}_s, \text{pk}_s) = (z, Z)$.

C.Query(pp, $\text{sk}_c^{\text{ln}}, \text{pk}_c^{\text{ln}}, \text{pk}_s$):

- 6) Split the bit representation of sk_c^{ln} into chunks $x_1, \dots, x_L \in [0, 2^\ell - 1]$ such that $\text{sk}_c^{\text{ln}} = \sum_{i \in [L]} 2^{(i-1)\ell} x_i$ holds without modulo reduction.
- 7) Sample $r, \delta \xleftarrow{\$} \mathbb{Z}_p$.
- 8) Compute $C = G_0 + rH_0 + \sum_{i \in [L]} x_i H_i$ and $T = \delta C$.
- 9) Compute $\pi_{\text{Rq}} = \text{PK}_{\text{Rq}}.\text{Prove}(\text{rpp}, (\text{pk}_c^{\text{ln}}, T), (\{x_i\}_{i \in [L]}, r, \delta))$.
- 10) Return $(\text{query}, \text{st}) = ((T, \pi_{\text{Rq}}), (\text{pk}_s, r, \delta, \{x_i\}_{i \in [L]}, T))$.

S.Issue(pp, $\text{sk}_s, \text{pk}_c^{\text{ln}}, \text{query}$):

- 11) If $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}_c^{\text{ln}}, T), \pi_{\text{Rq}}) = 0$ return $\perp$.
- 12) Sample $e \xleftarrow{\$} \mathbb{Z}_p$ and compute $S = \frac{1}{\text{sk}_s + e} T$.
- 13) Return $\text{resp} = (e, S)$.

C.Final(pp, st, resp):

- 14) If $e(S, \text{pk}_s + eG_2) \neq e(T, G_2)$ return $\perp$.
- 15) Otherwise, return $(\sigma, \omega) = (\frac{1}{\delta} S, (r, e, \{x_i\}_{i \in [L]}))$.

(C.Red(pp, $\sigma, \omega, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}, \text{pk}_s$), S.Red(pp, pk_s)):

- 16) The protocol is initiated by C sending $(\text{pk}_c^{\text{ld}}, \sigma)$.
- 17) S samples $k \xleftarrow{\$} \mathbb{Z}_p$, computes $K = kG_0$ and $\pi_{\text{Ch}} = \text{PK}_{\text{Ch}}.\text{Prove}((K, \text{pk}_c^{\text{ld}}), k)$, and sends (K, π_{Ch}) to C .
- 18) If $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{ld}}), \pi_{\text{Ch}}) = 0$ or $K = 0$ then C outputs 0.
- 19) C samples $r_1, \dots, r_L \xleftarrow{\$} \mathbb{Z}_p$ and computes
 - $A_i = r_i G_1$ and $B_i = r_i K + x_i G_1$ for $i \in [L]$,
 - $\sigma' = G_0 + rH_0 - e\sigma + \sum_{i \in [L]} x_i H_i$ and
 - $\pi_{\text{Rd}} = \text{PK}_{\text{Rd}}.\text{Prove}((\text{pk}_c^{\text{ld}}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), (\text{sk}_c^{\text{ld}}, r, e, \{r_i, x_i\}_{i \in [L]}))$.
- 20) C sends $(\pi_{\text{Rd}}, \sigma', \{A_i, B_i\}_{i \in [L]})$ to S and outputs 1.
- 21) S outputs and sends 1 if $e(\sigma, Z) = e(\sigma', G_2)$ and $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}_c^{\text{ld}}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 1$ and 0 otherwise.

KeyV(pp, sk, pk):

- 22) If $\text{pk} = \text{sk}G_1$ return 1. Otherwise, return 0.

Figure 4: Our ATB scheme.

$D_i = x_i G_1$ by brute-forcing the discrete logarithm of D_i over the interval $[0, 2^\ell - 1]$, i.e., $x_i = \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$. Although there exists algorithms accelerating the computation of the discrete logarithm [52], [53], the complexity of that computation still grows exponentially in ℓ . This results in an inherent trade-off between the protocol's efficiency and the betraying buyer's computational complexity. Larger values of ℓ improve protocol efficiency, since they reduce L and thereby lower the complexity of the protocol. However, increasing ℓ also raises the cost of brute-forcing $x_i = \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$. We configure this trade-off by parametrizing the protocol with a complexity bound $\mathcal{C}$ and choosing ℓ in the setup such that $\text{dlog}_{G_1}(G)[0, 2^\ell - 1]$ can be computed with complexity below $\mathcal{C}$. Naturally, we require that computations of complexity $\mathcal{C}$ are feasible for reasonably equipped clients. As the complexity bound $\mathcal{C}$ is a fixed protocol parameter, it follows that $L = \lceil \frac{p}{\ell} \rceil$ scales with the security parameter λ (which is equal to the bit-size of p). The overall complexity of our protocol, therefore, scales quadratically in the security parameter.

We note that the betrayal is not completely for free, as the buyer will not necessarily be able to redeem the bought token if it decides to betray the vendor. The created redemption proof is with respect to a replaced encryption key K^* and, therefore, will not be accepted by the server. Furthermore, the buyer needs the complete token witness $(r, e, \{x_i\}_{i \in [L]})$ to redeem the token itself while it steals only $\{x_i\}_{i \in [L]}$. Nevertheless, we remark that our constructions can straightforwardly be extended with the same techniques used for the extraction of $\{x_i\}_{i \in [L]}$ to allow the extraction of r and e .

5. Security

In this section, we show security of the ATB construction presented in Section 4.

Theorem 2. *The construction described in Construction 1 (Figure 4) is a secure ATB scheme according to Definition 2.*

We prove the theorem by proving the necessary security properties individually.

5.1. Correctness

Theorem 3. *The construction described in Construction 1 satisfies the correctness property defined in Definition 3.*

From the definition of algorithms Setup, S.KeyGen, S.Issue, C.KeyGen, C.Final, and C.Query and the completeness of the request zero-knowledge proof system it follows that

$$\begin{aligned} \text{pk}_c^{\text{In}} &= \text{sk}_c^{\text{In}} G_1, \text{pk}_c^{\text{Id}} = \text{sk}_c^{\text{Id}} G_1, \text{pk}_s = \text{sk}_s G_2 \\ \text{sk}_c^{\text{In}} &= \sum_{i \in [L]} 2^{(i-1)\ell} x_i, x_i \in [0, 2^\ell - 1] \text{ for } i \in [L] \\ \sigma &= \frac{1}{\text{sk}_s + e} (G_0 + r H_0 + \sum_{i \in [L]} x_i H_i) \end{aligned}$$

for each $(\sigma, \omega) = (\sigma, (r, e, \{x_i\}_{i \in [L]}))$ produced by C.Final. We can conclude that $\text{KeyV}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}) = \text{KeyV}(\text{pp}, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}) = 1$. As σ is as defined as above and $\sigma' = G_0 + r H_0 - e \sigma + \sum_{i \in [L]} x_i H_i$ it needs to hold that $\sigma' = \text{sk}_s \sigma$ and, hence, $e(\sigma, \text{pk}_s) = e(\sigma', G_2)$. Finally, as all proof components of PK_{Rd} are computed as defined by the proof relation, it follows from the proof system's completeness property that $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}_c^{\text{Id}}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 1$. Therefore, we can conclude that $\langle \text{C.Red}(\text{pp}, \sigma, \omega, \text{sk}^{\text{Id}}, \text{pk}^{\text{Id}}, \text{pk}_s), \text{S.Red}(\text{pp}, \text{pk}_s) \rangle = 1$.

5.2. Accountability

Theorem 4. *The scheme described in Construction 1 satisfies the accountability property defined in Definition 4.*

To show accountability, we prove that for any PPT adversary $\mathcal{A}$, the probability of winning the one-more unforgeability game, the request unforgeability game, and the identity unforgeability game is at most negligible in the security parameter. We provide proof sketches in the following and defer the full proof to Appendix F.

One-more unforgeability. The goal of our proof is to simplify the security game such that the resulting game can only be won by an adversary providing a BBS forgery. In two game hops, we adapt the request oracle to employ the request proof's extractor to extract the witnesses $(\{x_i\}_{i \in [L]}, r, \delta)$ from the blinded request (T, π_{Rq}) and adapt the redemption oracle to employ the redemption proof's extractor to extract the redemption proof's witnesses $(y, r, e, \{r_i, x_i\}_{i \in [L]})$ from the final redemption message $(\pi_{\text{Rd}}, \sigma', \{A_i, B_i\}_{i \in [L]})$. Computational indistinguishability between the adapted and the original game follows from the knowledge soundness property of the request proof system and the redemption proof system.

In the resulting game, we can show that the adversary's success probability is negligible by reducing to the BBS unforgeability game (cf. Figure 1). First note that we can use the BBS signing oracle to answer token requests in the reduction without knowing the secret issuance key sk_s . As the experiment extracts the witnesses $(\{x_i\}_{i \in [L]}, r, \delta)$ within the request oracle, it can request signatures (σ, e) on $(\{x_i\}_{i \in [L]} \cup \{r\})$ from the BBS signing oracle and create the token responses as $(S = \delta \sigma, e)$. Next note that the adversary has to successfully redeem at least $q_{\text{Rq}} + 1$ distinct tokens while only querying q_{Rq} tokens. Therefore, it needs to hold that at least one of the redeemed tokens has not been issued by the request oracle and, hence, constitutes a valid BBS forgery (together with the extracted witnesses). As no PPT adversary can win the BBS unforgeability game with probability higher than negligible (cf. Theorem 1) and the difference in the success probability in the original and the simplified game is negligible, we conclude that every PPT adversary wins the original one-more unforgeability game with at most negligible probability.

Request unforgeability. The goal of the request unforgeability proof is to gradually simplify the security game until

we can show that the adversary's success probability is at most negligible by reducing to the discrete logarithm assumption.

In the first four game hops, we adapt the redemption oracle to (i) use the redemption proof's simulator instead of the proof system's witnesses to create redemption proofs, (ii) employ the challenge proof's extractor to extract k such that $K = kG_0 \vee \text{pk}_j^{\text{ld}} = kG_1$, (iii) randomly sample the ciphertexts if $K = kG_0$, and (iv) ignore the case that $\text{pk}_j^{\text{ld}} = kG_1$. All changes introduce only a negligible difference in the adversary's success probability; (i) due to the zero-knowledge property of the redemption proof, (ii) due to the knowledge soundness property of the challenge proof, (iii) due to DDH assumption in $\mathbb{G}_1$ (which also implies semantic security of the ElGamal ciphertexts), and (iv) due to the discrete logarithm assumption in $\mathbb{G}_1$ making it impossible for an adversary to find $\text{dlog}_{G_1}(\text{pk}_j^{\text{ld}})$.

Next, we adapt the request oracle to create request proofs with the proof system's simulator instead of the witnesses. Computational indistinguishability follows from the zero-knowledge property of the request proof system. Subsequently, we randomly sample $C_i \xleftarrow{\$} \mathbb{G}_1$ instead of computing $C_i \leftarrow G_0 + r_i H_0 + \sum_{j \in [L]} x_j H_j$. This change does not affect the adversary's view and, hence, the output distribution of the experiment as r_i acts as a one-time pad ensuring a uniform distribution of C_i . Finally, we adapt the experiment's main thread to employ the request proof's extractor to extract $(\{x_j\}_{j \in [L]}, \cdot, \cdot)$ such that $\text{pk}_c^{\text{ln}} = (\sum_{j \in [L]} 2^{(j-1)\ell} x_j) G_1$. Computational indistinguishability follows from the knowledge soundness property of the request proof system.

In the final game, we can show that any PPT adversary has at most a negligible success probability via a reduction to the discrete logarithm assumption. As the adversary has to provide $\{x_j\}_{j \in [L]}$ such that $\text{pk}_c^{\text{ln}} = (\sum_{j \in [L]} x_j 2^{(j-1)\ell}) G_1$ without getting any extra information about pk_c^{ln} beyond pk_c^{ln} itself (all oracles are simulated with randomly sampled dummy-values), a winning adversary can be used to compute $\text{dlog}_{G_1}(\text{pk}_c^{\text{ln}})$. By assumption it holds that no PPT adversary can compute the discrete logarithm in $\mathbb{G}_1$ with probability higher than negligible, which allows us to conclude that every PPT adversary wins the simplified request unforgeability game with at most negligible probability. As there is at most a negligible gap in the success probability between the final and the original game, we conclude that our construction satisfies request unforgeability.

Identity unforgeability. Analogously to our request unforgeability proof, the identity unforgeability proof gradually simplifies the identity unforgeability security game to enable a reduction to the discrete logarithm assumption.

First, we adopt the request unforgeability proof's first simplification of the redemption oracle and use the redemption proof's simulator instead of the proof system's witnesses to create redemption proofs within $\mathcal{O}^{\text{Rd}}$. Next, we use the redemption proof's extractor to extract the witnesses of the redemption proof submitted to $\mathcal{O}^*$. Computational indistinguishability follows from the knowledge soundness

property of the redemption proof system.

In the resulting game we can show that any PPT adversary has at most a negligible success probability by reducing to the discrete logarithm assumption. As the adversary has to provide a valid zero-knowledge proof proving knowledge of sk^* such that $\text{pk}_c^{\text{ld}} = \text{sk}^* G_1$ without getting any information about pk_c^{ld} beyond pk_c^{ld} itself a winning adversary can be used to compute $\text{dlog}_{G_1}(\text{pk}_c^{\text{ld}})$ – the discrete logarithm of pk_c^{ld} needs to be contained in the witnesses extracted from the redemption proof. As there is at most a negligible gap in the success probability between the simplified and the original identity unforgeability game and the adversary wins in the simplified game with at most negligible probability due to the discrete logarithm assumption, we conclude that our construction satisfies identity unforgeability.

5.3. Unlinkability

Theorem 5. *The scheme described in Construction 1 satisfies the unlinkability property defined in Definition 5.*

We provide a proof sketch in the following and defer the full proof to Appendix G. The goal of our unlinkability proof is to gradually simplify the unlinkability security game until the game no longer uses the challenge bit b . If the game does not use bit b , the adversary cannot do better than guessing.

Analogously to the request unforgeability proof we simplify the oracles, i.e., we employ the redemption proof's simulator to create redemption proofs, we use the challenge proof's extractor to extract k such that $K = kG_0$ or $\text{pk}_j^{\text{ld}} = kG_1$, we create ciphertexts of randomly sampled dummy insurance key chunks if $K = kG_0$, we ignore the case that $\text{pk}_j^{\text{ld}} = kG_1$, we employ the request proof's simulator to create request proofs, and finally sample all commitments randomly, i.e., $C_i \xleftarrow{\$} \mathbb{G}_1$.

In the resulting experiment, we can discard the storage of token witnesses from the experiment as they are no longer used after being stored. Subsequently, we resample the selected tokens σ_α and σ_β randomly from $\mathbb{G}_1$ after potentially swapping them. This change does not affect the view of the adversary and, hence, the experiment's output distribution because in both, the new and the previous experiment, all tokens are uniform random elements in $\mathbb{G}_1$ and the associated T_i values are distributed independently from the tokens due to the random masks δ_i . Finally, we get rid of the swap of the tokens, which does not influence the experiment anymore because the selected tokens are overwritten after potentially being swapped. This change yields a final experiment, in which the challenge bit b , that needs to be guessed, is not used and, hence, is guessed with probability of exactly $\frac{1}{2}$. As all game hops introduce at most a negligible difference in the success probability, we conclude that $|\Pr[\text{Exp}_A^{\text{UL}}(\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda)$ for every PPT adversary A .

5.4. Betrayability

We state the following theorem:

Theorem 6. *The scheme described in Construction 1 satisfies the betrayability property defined in Definition 6.*

To show betrayability, we define the following extractor $\mathcal{E}$ with access to the token buying adversary $\mathcal{A}_B$ and oracles $\mathcal{O}^{\text{Rq}}$ and $\mathcal{O}^{\text{Rd}}$. We denote the extractor as if it would have an internal storage even though it is required to store its internal state within the state returned by its execution. Formally, all stored variables are appended to the returned state and removed before being passed to the internally executed adversary. We provide a proof sketch in the following and defer the complete proof to Appendix H.

- 1) Define $\mathcal{O}_{\text{ext}}^{\text{Rq}}$ to answer queries $(\text{pk}, \text{query})$ by returning the output of $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query})$.
- 2) Define $\mathcal{O}_{\text{ext}}^{\text{Rd}}$ to answer queries (B, sid, m) as follows:
 - a) If $\text{st}_{\text{sid}}^{(S, B)}$ is not defined, and $m = (\sigma^*, \text{pk}_*^{\text{ld}})$, then store $k_{\text{sid}} \xleftarrow{\$} \mathbb{Z}_p$, compute $K_{\text{sid}} = k_{\text{sid}}G_1$ and $\pi_{\text{Ch}} = \text{PK}_{\text{Ch}}.\text{Prove}((K_{\text{sid}}, \text{pk}_*^{\text{ld}}), \text{sk}_*^{\text{ld}})$, add sid to $\mathcal{T}^*$, store $\text{st}_{\text{sid}}^{(S, B)} = (k_{\text{sid}}, K_{\text{sid}}, \sigma^*, \text{pk}_*^{\text{ld}})$ and respond with $(K_{\text{sid}}, \pi_{\text{Ch}})$.
 - b) If $\text{st}_{\text{sid}}^{(S, B)} \in \{0, 1\}$ then return $\emptyset$.
 - c) If $\text{sid} \in \mathcal{T}^*$, then parse $(\pi_{\text{Rd}}, \sigma', \{A_i, B_i\}_{i \in [L]}) = m$. If $e(\sigma^*, \text{pk}_s) \neq e(\sigma', G_2)$ or $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}_*^{\text{ld}}, K_{\text{sid}}, \sigma^*, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0$ then set $\text{st}_{\text{sid}}^{(S, B)} = 0$ and return 0. If x^* is undefined then compute $D_i = B_i - kA_i$, $x_i = \text{dlog}_{G_1}(D_i)[0; 2^\ell - 1]$. If $\nexists i \in [L]$ such that $x_i = \perp$ then define $x^* = \sum_{i \in [L]} 2^{(i-1)\ell} x_i$. Set $\text{st}_{\text{sid}}^{(S, B)} = 1$ and return 1.
 - d) Otherwise, return $\mathcal{O}^{\text{Rd}}(B, \text{sid}, m)$.
- 3) On input $(1, \sigma^*, \{\text{pp}, n, \text{pk}_s\})$, execute $(\text{pk}_*^{\text{ld}}, \text{sk}_*^{\text{ld}}, \text{st}_{(2,0)}^B) \leftarrow \mathcal{A}_B^{\mathcal{O}_{\text{ext}}^{\text{Rq}}, \mathcal{O}_{\text{ext}}^{\text{Rd}}}(1, \{\text{pp}, n, \text{pk}_s\})$, store $\text{pp}, \text{pk}_s, \text{pk}_*^{\text{ld}}, \text{sk}_*^{\text{ld}}, \sigma^*, \mathcal{T}^* \leftarrow \emptyset$ and return $(\text{pk}_*^{\text{ld}}, \text{sk}_*^{\text{ld}}, \text{st}_{(2,0)}^B)$.
- 4) On input $(2, i, m, \text{st})$, execute $(m', \text{st}') \leftarrow \mathcal{A}_B^{\mathcal{O}_{\text{ext}}^{\text{Rq}}, \mathcal{O}_{\text{ext}}^{\text{Rd}}}(2, i, m, \text{st})$ and return (m', st') .
- 5) On input $(3, \text{st}_{(3,n)}^B)$ return x^* if defined or $\perp$ otherwise.

The aim of the betrayability proof is to gradually simplify the betrayability security game until the game's only dependence on b is the win condition and the win condition for $b = 1$ is contained in the win condition for $b = 0$.

First, we inline the actions of the extractor $\mathcal{E}$ as defined above into the security game. Next, we make the whole redemption oracle independent from coin b . To do so, we use the challenge proof's simulator instead of the proof system's witnesses to create challenge proofs and adapt the actions inherent to the $(b = 1)$ -execution to the $(b = 0)$ -execution and vice versa. In particular, we also include a trapdoor into the challenge $(K \leftarrow kG_1)$ and attempt the decryption of the received ciphertexts to x^* if $(b = 0) \wedge \mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{ld}})$. In the other direction, we define the success variable if $(b = 1)$ analogously to how it is defined in the $(b = 0)$ -case. The simulation of the challenge proof introduces only a negligible difference to the output distribution due to the zero-knowledge property of the challenge proof system. The other changes do not influence the output distribution, as K is still distributed uniformly random and the subsequent steps do not influence the experiment (success is not used in the $(b = 1)$ -case and x^* is not used in the $(b = 0)$ -case).

In the next step, we use the request proof's extractor to extract the witnesses of the request proof and abort if the extraction fails. This step introduces only a negligible difference in the output distribution due to the knowledge soundness property of the request proof system. With the extracted witnesses, we compute the resulting token σ and store the token together with the witnesses in a set Γ as long as σ^* (or $\mathcal{K}^*$) has not been defined, yet.

Subsequently, we use the redemption proof's extractor to extract the witnesses of the redemption proof and abort if (i) the extraction fails or (ii) σ^* has not been issued by the request oracle (which can be checked via Γ). This step introduces only a negligible difference in the output distribution due to (i) the knowledge soundness property of the redemption proof system and (ii) the unforgeability property of the BBS signatures. To see the latter, note that the successful extraction of the redemption proof provides valid witnesses $\omega' = (\cdot, r', e', \{x_i\}_{i \in [L]})$ such that (σ^*, ω') constitutes a valid BBS signature. If σ^* has not been issued by the oracle, the tuple (σ^*, ω') needs to be a BBS forgery.

Next, we check if the token-witness pairs $(\sigma^*, \omega^* = (e^*, r^*, \{x_i\}_{i \in [L]}))$ stored in Γ match the witnesses ω' extracted from the redemption proof after the extraction of ω' . If this is not the case, we abort the experiment. Here, we show that the change introduces only a negligible difference in the output distribution via a reduction to the discrete logarithm assumption. In particular, we show that an adversary that is capable of computing two distinct but valid witnesses (ω^*, ω') for the same token σ^* can be used to compute the discrete logarithm in $\mathbb{G}_1$.

In a next step, we compute x^* based on ω^* instead of the ciphertexts included in the redemption message. This change does not influence the output distribution. To see this, note that $\omega' = \omega^*$ (when considering only the relevant components of ω') and $x'_i = x_i^* \in [0, 2^\ell - 1]$ for $i \in [L]$. It follows that the computation of $\text{dlog}_{G_1}(D_i)[0; 2^\ell - 1]$ succeeds with probability 1. Consequently, it needs to hold that both ways of computing x^* , based on the ciphertexts or based on ω^* , yield the same x^* .

Finally, we modify the win condition of the $(b = 0)$ -case to include the win condition of the $(b = 1)$ -case, i.e., $x^*G_1 \in \mathcal{K}^*$. This change does not influence the output distribution. Note that whenever success is set to **true** (the only win condition in the previous experiment), the experiment also defines x^* (based on ω^*). Further, observe that for every token-witness tuple in Γ , it needs to hold that a public key $\text{pk} = xG_1$ is added to $\mathcal{K}^*$. Therefore, it holds that if **success** = **true** then $x^*G_1 \in \mathcal{K}^*$, which shows that the additional win condition does not influence the output distribution.

As all game hops introduce at most a negligible difference in the success probability and the success probability of the $(b = 1)$ -case in the final experiment is clearly higher or equal to the one of the $(b = 0)$ -case (the win condition of the $(b = 1)$ -case is included into the one of the $(b = 0)$ -case), we can conclude that $\Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 1}^{\text{BT}}(\lambda) = 1] \geq \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 0}^{\text{BT}}(\lambda) = 1 - \text{negl}(\lambda)]$.

6. Evaluation

We implemented several versions of our construction, executed benchmarks and determined the communication complexity. For comparison, we have also implemented and benchmarked the publicly verifiable non-transferable anonymous token (PV-NTAT) construction of [7].

The versions of our construction differentiate in the instantiation of the range proofs and the phase, in which the range proofs are executed. We either execute the range proofs in the issuance phase as part of the client’s request proof or in the redemption phase as part of the client’s redemption proof. In the latter case, we also postpone the chunk decomposition of the insurance secret to the redemption phase. For the instantiation of the range proofs, we consider the Bulletproofs++ proof system [24] and Groth16 general-purpose zk-SNARKs [25]. Bulletproofs++ have the advantage that they plug-well into the generalized Schnorr proofs, we use for the remaining relations of the respective proof system, and support the efficient aggregation of proofs, which is desirable as we require one range proof for every insurance key chunk. Zk-SNARKS, on the other hand, have a constant proof size and, hence, are highly efficient in terms of communication. However, the proof creation and verification is much less efficient than the creation of Bulletproofs++ proofs. The Groth16 SNARK proofs fit well into our setting as there is one dedicated verifier for the range proofs, the server. Therefore, we do not need to deal with the toxic waste produced during the generation of the SNARK’s public parameters. In the SNARK-based variants, we realize the whole proof, not only the part concerning the range relation, with one SNARK.

We call the variants Req-BP++, Red-BP++, Req-SNARK, and Red-SNARK, based on the phase that includes the range proofs and the instantiation of the range proofs. The Req-BP++ variant is the one presented in Construction 1 in Section 4.

6.1. Benchmarks

Experimental setup. Our implementation and benchmarks are written in Rust (compiled with rustc 1.86.0-nightly (124cc9219)). We execute all our benchmarks on a Laptop with an Intel(R) Core(TM) i7-1360P @ 2.20 GHz processor and 16GB of RAM. We repeat all benchmarks 100 times and report the average result.

In our benchmarks, we measure the execution time of contract and client in both the request phase and the redemption phase. We bench the parties individually and add up non-continuously executed parts if necessary. The client’s execution during the request phase includes the computation of C.Query and C.Final and the server’s execution the computation of S.Issue. During the redemption phase, the client executes Step 18 - Step 20 while the server executes Step 17 and Step 21 of Construction 1. Our benchmarks do not consider the overhead of message transmission. Network affects can be incorporated based on the respective network conditions and the reported communication complexity.

All the variants are based on the *BLS12-377* curve. The SNARKs are computed over the *BW6-761* curve, as this curve’s scalar field is equal to the base field of the *BLS12-377* curve. We utilize Arkworks’ crates for curve operations and SNARK computation and the *bulletproofs-plus-plus* crate for the Bulletproof++ range proofs.

We repeat all benchmarks for $L \in \{4, 8, 16, 32\}$. By focusing on powers of 2 when considering 256-bit Scalar elements, we can ensure that the secret insurance key is split into equally sized chunks, which is convenient for implementation. We do not consider $L = 2$ as this would require a buyer betraying a token vendor to brute force 128-bit scalars, which is not feasible. Further, we note that an increase of the security parameter (which is equivalent to an increase of group order p) requires to also increase L in order to ensure that the chunks can still be brute-forced in reasonable time. The complexity of our protocol, therefore, scales quadratically with the security parameter.

Experimental Results. We report the results of the benchmarks of all our constructions in Figure 5, split in the runtime of the client and the server in the request phase and the redemption phase. In addition, we directly compare all variants in Figure 6.

The individual results show that the major overhead is due to the creation of the range proofs. In the Req-BP++ and Req-SNARK variant, the range proof is created by the client in the request phase. In the Red-BP++ and Red-SNARK construction, the range proof is created by the client in the redemption phase. In all variants, the phase including the creation of the range proof dominates the overall execution time of the variant. Further, the comparison highlights that the runtime of the variants based on Bulletproofs++ is much lower, which is expected to compensate the additional communication complexity in most network settings. When comparing the SNARK-based variants it is clearly evident that the Req-SNARK variant is much more efficient than the Red-SNARK.

Our implementation of the PV-NTAT construction of [7] yields 3.9 ms and 1 ms for the client in the request and redemption phase and 1.1 ms and 2.7 ms for the server in the request and redemption phase. This totals to a total execution time below 9 ms. Even though our extension comes with significant overhead, we still assess the variants based on Bulletproof++ with a total execution time below 50 ms for $L = 4$ to be feasible for real-world deployments.

6.2. Communication Complexity

Let L be the number of chunks, $|G|$ be the size of (encoded) group elements of $\mathbb{G}_1$, $|p|$ the size of field elements, $\text{RP}[|p|, L]$ the size of a Bulletproof++ range proof for L elements in range $[0, 2^{\frac{|p|}{2}} - 1]$, and $|\text{SNARK}|$ the size of a Groth16 SNARK. Based on these parameters, we report the communication complexity of the variants of our construction and compare them to the public verifiable protocol version of [7] in Table 1.

In the Bulletproof++ system instantiated over the BLS12-377 curve, four range proofs for interval $[0, 2^{64} - 1]$

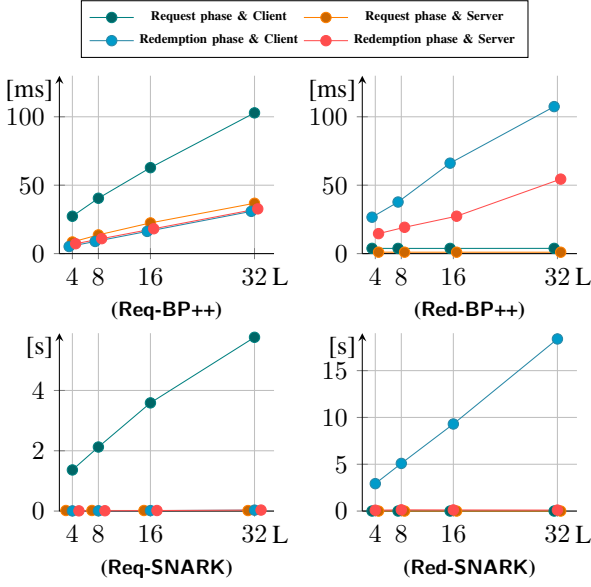


Figure 5: Execution cost of client and server in the request and redemption phase of our constructions dependent on the number of chunks L . To differentiate overlapping data points, we slightly shift them in x-direction, even though they still belong to $L = 4, 8, 16, 32$.

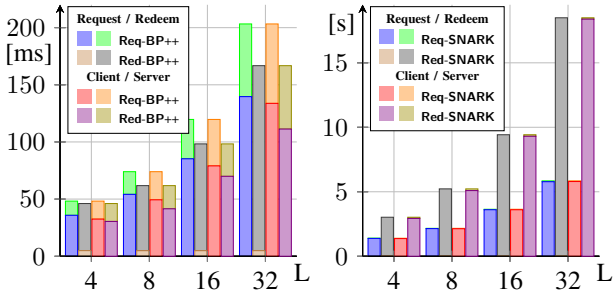


Figure 6: Comparison of the total runtime of all variants of our construction. The stacked bars differentiate between the request and redemption phase (the two left bars) and the client and server overhead (the two right bars).

require $\text{RP}[256, 4] = 736$ bytes ($12|G| + 5|p|$). One Groth16 proof instantiated over the BW6-761 curve has $|\text{SNARK}| = 288$ bytes (two elements of the curve pairing’s first group and one of its second group).

The overhead of our constructions when compared to [7] is mainly due to the chunk decomposition of the secret insurance key that allows decryption of exponential ElGamal ciphertexts of the insurance secret chunks, which in turn, enables us to perform efficient zero-knowledge proofs over ElGamal ciphertexts.

TABLE 1: Communication complexity of our Bulletproof++-based and SNARK-based instantiation compared to prior work on non-transferable tokens.

	Issuance		Redemption	
	Client	Server	Client	Server
PV-NTAT [7] ³	$2 G + 4 p $	$ G + p $	$2 G + 5 p $	$ p $
Req-BP++	$(2 + L) G + (3 + 2L) p + \text{RP}[p, L]$	$ G + p $	$(3 + 2L) G + (4 + 2L) p + \text{RP}[p, L]$	$3 G + p $
Red-BP++	$2 G + 4 p $	$ G + p $	$(3 + 2L) G + (4 + 2L) p + \text{RP}[p, L]$	$3 G + p $
Req-SNARK	$2 G + \text{SNARK} $	$ G + p $	$(3 + 2L) G + (4 + 2L) p $	$3 G + p $
Red-SNARK	$2 G + 4 p $	$ G + p $	$(3 + 2L) G + \text{SNARK} $	$3 G + p $

References

- [1] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, and F. Valsorda, “Privacy pass: Bypassing internet challenges anonymously,” *Proc. Priv. Enhancing Technol.*, vol. 2018, no. 3, pp. 164–180, 2018. [Online]. Available: <https://doi.org/10.1515/popets-2018-0026>
- [2] B. Kreuter, T. Lepoint, M. Orrù, and M. Raykova, “Anonymous tokens with private metadata bit,” in *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part I*, ser. Lecture Notes in Computer Science, D. Micciancio and T. Ristenpart, Eds., vol. 12170. Springer, 2020, pp. 308–336. [Online]. Available: https://doi.org/10.1007/978-3-030-56784-2_11
- [3] F. Benhamouda, T. Lepoint, M. Orrù, and M. Raykova, “Publicly verifiable anonymous tokens with private metadata bit,” *IACR Cryptol. ePrint Arch.*, p. 4, 2022. [Online]. Available: <https://eprint.iacr.org/2022/004>
- [4] T. Silde and M. Strand, “Anonymous tokens with public metadata and applications to private contact tracing,” in *Financial Cryptography and Data Security - 26th International Conference, FC 2022, Grenada, May 2-6, 2022, Revised Selected Papers*, ser. Lecture Notes in Computer Science, I. Eyal and J. A. Garay, Eds., vol. 13411. Springer, 2022, pp. 179–199. [Online]. Available: https://doi.org/10.1007/978-3-031-18283-9_9
- [5] F. Benhamouda, M. Raykova, and K. Seth, “Anonymous counting tokens,” in *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security, Guangzhou, China, December 4-8, 2023, Proceedings, Part II*, ser. Lecture Notes in Computer Science, J. Guo and R. Steinfield, Eds., vol. 14439. Springer, 2023, pp. 245–278. [Online]. Available: https://doi.org/10.1007/978-981-99-8724-5_8
- [6] M. Chase, F. B. Durak, and S. Vaudenay, “Anonymous tokens with stronger metadata bit hiding from algebraic macs,” in *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part II*, ser. Lecture Notes in Computer Science, H. Handschuh and A. Lysyanskaya, Eds., vol. 14082. Springer, 2023, pp. 418–449. [Online]. Available: https://doi.org/10.1007/978-3-031-38545-2_14

3. We add the transmission of the insurance public key during the request phase and the transmission of the token during the redemption phase. Further, we assume the hash-based commitment utilized in [7] to receive a field element in $\mathbb{Z}_p$ as random nonce and return a field element in $\mathbb{Z}_p$ as commitment.

- [7] F. B. Durak, L. Marco, A. Talayhan, and S. Vaudenay, "Non-transferable anonymous tokens by secret binding," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. ACM, 2024, pp. 2460–2474. [Online]. Available: <https://doi.org/10.1145/3658644.3670338>
- [8] I. Karantaidou, O. Renawi, F. Baldimtsi, N. Kamarinakis, J. Katz, and J. Loss, "Blind multisignatures for anonymous tokens with decentralized issuance," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. ACM, 2024, pp. 1508–1522. [Online]. Available: <https://doi.org/10.1145/3658644.3690364>
- [9] M. Orrù, S. Tessaro, G. Zaverucha, and C. Zhu, "Oblivious issuance of proofs," in *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IX*, ser. Lecture Notes in Computer Science, L. Reyzin and D. Stebila, Eds., vol. 14928. Springer, 2024, pp. 254–287. [Online]. Available: https://doi.org/10.1007/978-3-031-68400-5_8
- [10] G. Amjad, K. Yeo, and M. Yung, "RSA blind signatures with public metadata," *Proc. Priv. Enhancing Technol.*, vol. 2025, no. 1, pp. 37–57, 2025. [Online]. Available: <https://doi.org/10.56553/popets-2025-0004>
- [11] F. Baldimtsi, L. Hanzlik, Q. Nguyen, and A. Yadav, "Non-interactive anonymous tokens with private metadata bit," *IACR Cryptol. ePrint Arch.*, p. 430, 2025. [Online]. Available: <https://eprint.iacr.org/2025/430>
- [12] Cloudflare. (2025) Privacy pass. [Online]. Available: <https://developers.cloudflare.com/waf/tools/privacy-pass/>
- [13] Google. (2024) Private state tokens. [Online]. Available: <https://developers.google.com/privacy-sandbox/protectations/private-state-tokens>
- [14] IETF. (2025) Privacy pass (privacypass). [Online]. Available: <https://datatracker.ietf.org/wg/privacypass/about/>
- [15] C. Dwork, J. B. Lipton, and M. Naor, "Digital signets: Self-enforcing protection of digital information (preliminary version)," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, G. L. Miller, Ed. ACM, 1996, pp. 489–498. [Online]. Available: <https://doi.org/10.1145/237814.237997>
- [16] O. Goldreich, B. Pfitzmann, and R. L. Rivest, "Self-delegation with controlled propagation - or - what if you lose your laptop," in *Advances in Cryptology - CRYPTO '98, 18th Annual International Cryptology Conference, Santa Barbara, California, USA, August 23-27, 1998, Proceedings*, ser. Lecture Notes in Computer Science, H. Krawczyk, Ed., vol. 1462. Springer, 1998, pp. 153–168. [Online]. Available: <https://doi.org/10.1007/BFb0055726>
- [17] A. Lysyanskaya, R. L. Rivest, A. Sahai, and S. Wolf, "Pseudonym systems," in *Selected Areas in Cryptography, 6th Annual International Workshop, SAC'99, Kingston, Ontario, Canada, August 9-10, 1999, Proceedings*, ser. Lecture Notes in Computer Science, H. M. Heys and C. M. Adams, Eds., vol. 1758. Springer, 1999, pp. 184–199. [Online]. Available: https://doi.org/10.1007/3-540-46513-8_14
- [18] S. Tessaro and C. Zhu, "Revisiting BBS signatures," in *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, ser. Lecture Notes in Computer Science, C. Hazay and M. Stam, Eds., vol. 14008. Springer, 2023, pp. 691–721. [Online]. Available: https://doi.org/10.1007/978-3-031-30589-4_24
- [19] A. Barki, S. Brunet, N. Desmoulin, and J. Traoré, "Improved algebraic macs and practical keyed-verification anonymous credentials," in *Selected Areas in Cryptography - SAC 2016 - 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers*, ser. Lecture Notes in Computer Science, R. Avanzi and H. M. Heys, Eds., vol. 10532. Springer, 2016, pp. 360–380. [Online]. Available: https://doi.org/10.1007/978-3-319-69453-5_20
- [20] D. Boneh, X. Boyen, and H. Shacham, "Short group signatures," in *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, ser. Lecture Notes in Computer Science, M. K. Franklin, Ed., vol. 3152. Springer, 2004, pp. 41–55. [Online]. Available: https://doi.org/10.1007/978-3-540-28628-8_3
- [21] M. H. Au, W. Susilo, and Y. Mu, "Constant-size dynamic k -taa," in *Security and Cryptography for Networks, 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006, Proceedings*, ser. Lecture Notes in Computer Science, R. D. Prisco and M. Yung, Eds., vol. 4116. Springer, 2006, pp. 111–125. [Online]. Available: https://doi.org/10.1007/11832072_8
- [22] T. E. Gamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," in *Advances in Cryptology, Proceedings of CRYPTO '84, Santa Barbara, California, USA, August 19-22, 1984, Proceedings*, ser. Lecture Notes in Computer Science, G. R. Blakley and D. Chaum, Eds., vol. 196. Springer, 1984, pp. 10–18. [Online]. Available: https://doi.org/10.1007/3-540-39568-7_2
- [23] C. Schnorr, "Efficient identification and signatures for smart cards," in *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, ser. Lecture Notes in Computer Science, G. Brassard, Ed., vol. 435. Springer, 1989, pp. 239–252. [Online]. Available: https://doi.org/10.1007/0-387-34805-0_22
- [24] L. Eagen, S. Kanjalkar, T. Ruffing, and J. Nick, "Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments," in *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part V*, ser. Lecture Notes in Computer Science, M. Joye and G. Leander, Eds., vol. 14655. Springer, 2024, pp. 249–279. [Online]. Available: https://doi.org/10.1007/978-3-031-58740-5_9
- [25] J. Groth, "On the size of pairing-based non-interactive arguments," in *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, ser. Lecture Notes in Computer Science, M. Fischlin and J. Coron, Eds., vol. 9666. Springer, 2016, pp. 305–326. [Online]. Available: https://doi.org/10.1007/978-3-662-49896-5_11
- [26] D. Chaum, "Blind signatures for untraceable payments," in *Advances in Cryptology: Proceedings of CRYPTO '82, Santa Barbara, California, USA, August 23-25, 1982*, D. Chaum, R. L. Rivest, and A. T. Sherman, Eds. Plenum Press, New York, 1982, pp. 199–203. [Online]. Available: https://doi.org/10.1007/978-1-4757-0602-4_18
- [27] —, "Blind signature system," in *Advances in Cryptology, Proceedings of CRYPTO '83, Santa Barbara, California, USA, August 21-24, 1983*, D. Chaum, Ed. Plenum Press, New York, 1983, p. 153.
- [28] —, "Security without identification: Transaction systems to make big brother obsolete," *Commun. ACM*, vol. 28, no. 10, pp. 1030–1044, 1985. [Online]. Available: <https://doi.org/10.1145/4372.4373>
- [29] L. Chen, "Access with pseudonyms," in *Cryptography: Policy and Algorithms, International Conference, Brisbane, Queensland, Australia, July 3-5, 1995, Proceedings*, ser. Lecture Notes in Computer Science, E. Dawson and J. D. Golic, Eds., vol. 1029. Springer, 1995, pp. 232–243. [Online]. Available: <https://doi.org/10.1007/BFb0032362>
- [30] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology - EUROCRYPT 2001, International Conference on the Theory and Application of Cryptographic Techniques, Innsbruck, Austria, May 6-10, 2001, Proceedings*, ser. Lecture Notes in Computer Science, B. Pfitzmann, Ed., vol. 2045. Springer, 2001, pp. 93–118. [Online]. Available: https://doi.org/10.1007/3-540-44987-6_7

- [31] —, “Signature schemes and anonymous credentials from bilinear maps,” in *Advances in Cryptology - CRYPTO 2004, 24th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 2004, Proceedings*, ser. Lecture Notes in Computer Science, M. K. Franklin, Ed., vol. 3152. Springer, 2004, pp. 56–72. [Online]. Available: https://doi.org/10.1007/978-3-540-28628-8_4
- [32] J. Camenisch, “Anonymous credentials: Opportunities and challenges,” in *Security and Privacy in Dynamic Environments, Proceedings of the IFIP TC-11 21st International Information Security Conference (SEC 2006), 22-24 May 2006, Karlstad, Sweden*, ser. IFIP, S. Fischer-Hübner, K. Rannenberg, L. Yngström, and S. Lindskog, Eds., vol. 201. Springer, 2006, p. 460. [Online]. Available: https://doi.org/10.1007/0-387-33406-8_42
- [33] J. Camenisch, M. Dubovitskaya, K. Haralambiev, and M. Kohlweiss, “Composable and modular anonymous credentials: Definitions and practical constructions,” in *Advances in Cryptology - ASIACRYPT 2015 - 21st International Conference on the Theory and Application of Cryptology and Information Security, Auckland, New Zealand, November 29 - December 3, 2015, Proceedings, Part II*, ser. Lecture Notes in Computer Science, T. Iwata and J. H. Cheon, Eds., vol. 9453. Springer, 2015, pp. 262–288. [Online]. Available: https://doi.org/10.1007/978-3-662-48800-3_11
- [34] J. Camenisch, S. Krenn, A. Lehmann, G. L. Mikkelsen, G. Neven, and M. Ø. Pedersen, “Formal treatment of privacy-enhancing credential systems,” in *Selected Areas in Cryptography - SAC 2015 - 22nd International Conference, Sackville, NB, Canada, August 12-14, 2015, Revised Selected Papers*, ser. Lecture Notes in Computer Science, O. Dunkelman and L. Keliher, Eds., vol. 9566. Springer, 2015, pp. 3–24. [Online]. Available: https://doi.org/10.1007/978-3-319-31301-6_1
- [35] J. Blömer, J. Bobolz, D. Diemert, and F. Eidens, “Updatable anonymous credentials and applications to incentive systems,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, L. Cavallaro, J. Kinder, X. Wang, and J. Katz, Eds. ACM, 2019, pp. 1671–1685. [Online]. Available: <https://doi.org/10.1145/3319535.3354223>
- [36] A. Sonnino, M. Al-Bassam, S. Bano, S. Meiklejohn, and G. Danezis, “Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers,” in *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society, 2019. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/coconut-threshold-issuance-selective-disclosure-credentials-with-applications-to-distributed-ledgers/>
- [37] Z. Yu, M. H. Au, and R. Yang, “Accountable anonymous credentials,” in *Advances in Cyber Security: Principles, Techniques, and Applications*, K. Li, X. Chen, and W. Susilo, Eds. Springer, 2019, pp. 49–68. [Online]. Available: https://doi.org/10.1007/978-981-13-1483-4_3
- [38] L. Hanzlik and D. Slamanig, “With a little help from my friends: Constructing practical anonymous credentials,” in *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds. ACM, 2021, pp. 2004–2023. [Online]. Available: <https://doi.org/10.1145/3460120.3484582>
- [39] C. Paquin and G. Zaverucha. (2023) U-prove cryptographic specification v1.1 (revision 5). [Online]. Available: <https://github.com/microsoft/uprove-node-reference/blob/main/doc/U-Prove%20Cryptographic%20Specification%20V1.1%20Revision%205.pdf>
- [40] J. Camenisch, M. Drijvers, and A. Lehmann, “Anonymous attestation using the strong diffie hellman assumption revisited,” in *Trust and Trustworthy Computing - 9th International Conference, TRUST 2016, Vienna, Austria, August 29-30, 2016, Proceedings*, ser. Lecture Notes in Computer Science, M. Franz and P. Papadimitratos, Eds., vol. 9824. Springer, 2016, pp. 1–20. [Online]. Available: https://doi.org/10.1007/978-3-319-45572-3_1
- [41] J. Doerner, Y. Kondi, E. Lee, A. Shelat, and L. Tyner, “Threshold BBS+ signatures for distributed anonymous credential issuance,” in *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 773–789. [Online]. Available: <https://doi.org/10.1109/SP46215.2023.10179470>
- [42] S. Faust, C. Hazay, D. Kretzler, L. Rometsch, and B. Schlosser, “Non-interactive threshold BBS+ from pseudorandom correlations,” *IACR Cryptol. ePrint Arch.*, p. 1076, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1076>
- [43] S. Jarecki, A. Kiayias, and H. Krawczyk, “Round-optimal password-protected secret sharing and T-PAKE in the password-only model,” in *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II*, ser. Lecture Notes in Computer Science, P. Sarkar and T. Iwata, Eds., vol. 8874. Springer, 2014, pp. 233–253. [Online]. Available: https://doi.org/10.1007/978-3-662-45608-8_13
- [44] G. Fuchsbauer, A. Plouviez, and Y. Seurin, “Blind schnorr signatures and signed elgamal encryption in the algebraic group model,” in *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II*, ser. Lecture Notes in Computer Science, A. Canteaut and Y. Ishai, Eds., vol. 12106. Springer, 2020, pp. 63–95. [Online]. Available: https://doi.org/10.1007/978-3-030-45724-2_3
- [45] D. Boneh, B. Lynn, and H. Shacham, “Short signatures from the weil pairing,” in *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings*, ser. Lecture Notes in Computer Science, C. Boyd, Ed., vol. 2248. Springer, 2001, pp. 514–532. [Online]. Available: https://doi.org/10.1007/3-540-45682-1_30
- [46] A. Boldyreva, “Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme,” in *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, ser. Lecture Notes in Computer Science, Y. Desmedt, Ed., vol. 2567. Springer, 2003, pp. 31–46. [Online]. Available: https://doi.org/10.1007/3-540-36288-6_3
- [47] D. Boneh, M. Drijvers, and G. Neven, “Compact multi-signatures for smaller blockchains,” in *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, ser. Lecture Notes in Computer Science, T. Peyrin and S. D. Galbraith, Eds., vol. 11273. Springer, 2018, pp. 435–464. [Online]. Available: https://doi.org/10.1007/978-3-030-03329-3_15
- [48] E. C. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu, “Snowblind: A threshold blind signature in pairing-free groups,” in *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part I*, ser. Lecture Notes in Computer Science, H. Handschuh and A. Lysyanskaya, Eds., vol. 14081. Springer, 2023, pp. 710–742. [Online]. Available: https://doi.org/10.1007/978-3-031-38557-5_23
- [49] R. L. Rivest, A. Shamir, and L. M. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978. [Online]. Available: <https://doi.org/10.1145/359340.359342>
- [50] L. Hanzlik, “Non-interactive blind signatures for random messages,” in *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, ser. Lecture Notes in Computer Science, C. Hazay and M. Stam, Eds., vol. 14008. Springer, 2023, pp. 722–752. [Online]. Available: https://doi.org/10.1007/978-3-031-30589-4_25

- [51] D. Boneh and M. K. Franklin, "Identity-based encryption from the weil pairing," in *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, ser. Lecture Notes in Computer Science, J. Kilian, Ed., vol. 2139. Springer, 2001, pp. 213–229. [Online]. Available: https://doi.org/10.1007/3-540-44647-8_13
- [52] D. Shanks, "Class number, a theory of factorization, and genera," in *Proceedings of Symposia in Pure Mathematics*, vol. 20. American Mathematical Society, 1971, pp. 415–440.
- [53] J. M. Pollard, "Monte carlo methods for index computation (mod p)," *Mathematics of computation*, vol. 32, no. 143, pp. 918–924, 1978.
- [54] J. Camenisch and M. Stadler, "Efficient group signature schemes for large groups (extended abstract)," in *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, ser. Lecture Notes in Computer Science, B. S. K. Jr., Ed., vol. 1294. Springer, 1997, pp. 410–424. [Online]. Available: <https://doi.org/10.1007/BFb0052252>
- [55] Full version. [Online]. Available: <https://anonymous.4open.science/r/AnonymousTokensWithBetrayability>

Appendix A. Necessity of Identification Keys

In this section, we show that our betrayability notion cannot be achieved in the model of [7], in which the redemption of a token requires nothing but the token and its witness including the corresponding insurance secret.

First, consider a scenario Exp_1 , in which a token vendor V with a valid token supports an honest buyer B (that has no private state and does not try to betray V) in the redemption of a token at a server S that constitutes both issuer and verifier. Client V only participates in the process if B does nothing but forwarding messages between V and S and responds to the messages received by B exactly as it would respond to messages by S in an honest token redemption. If the vendor detects that a message received from B does not originate from S , it aborts the redemption.

We observe that in Exp_1 , the redemption of the token is always successful. To see this, consider a second scenario Exp_2 , in which a client C^* internally executes both V and B and engages in a redemption with server S . Client C^* sends the messages produced by B to S and the messages received by S to B . It is obvious that the redemption in Exp_1 is successful if it is in Exp_2 . We simplify Exp_2 to Exp'_2 , in which C^* directly forwards messages between V and S without making use of B . Clearly, this does not affect the outcome of the scenario, as the messages produced by C^* are exactly the same in Exp_2 and Exp'_2 . As Exp'_2 constitutes an honestly executed token redemption, the correctness of the anonymous token scheme guarantees that the redemption in Exp'_2 is successful. Therefore, we can conclude that the redemption in Exp_1 is successful as well.

Next, we observe that the betrayability property requires that a buyer B' is able to betray any vendor V' in any joint redemption protocol without being detected and that the betrayal is successful with probability equal or higher than the success probability of the joint redemption protocol (minus negligible). Therefore, it is guaranteed that in an experiment

Exp'_1 , in which B is replaced by a betraying buyer B' , B' obtains the insurance secret with non-negligible probability without being detected.

Finally, we consider a scenario Exp_3 , in which an honest client C redeems a token from a malicious server S' . The malicious server internally executes an honest server S and a betraying buyer B' . It sends the messages produced by B' to C , inputs the message received from C to B' and forwards messages between B' and S . Note that client C is not able to differentiate if it talks to B' or S as otherwise V would be able to do so in experiment Exp'_1 . As B' extracts the client's secret with non-negligible probability in Exp'_1 , it holds that S' can do so in scenario Exp_3 . However, if S' is able to extract an honest client's insurance secret, S' can be used to attack the request unforgeability property of the token scheme by first extracting the insurance secret with S' and then forging requests with the extracted secret. Similarly, S' can be used to break the unlinkability property.

Consequently, we can conclude that in the model of [7], betrayability and request unforgeability as well as betrayability and unlinkability contradict each other. In our model, we circumvent this impossibility by providing an identification key to the buyer that needs to be known for the token redemption and the insertion of the trapdoor into the redemption. This makes it impossible for a malicious server to emulate a betraying buyer.

Appendix B. Comparison of our security notion to [7]

In our definition, we inherit the one-more unforgeability, request unforgeability and unlinkability property from [7] and adapt them to our definition. Furthermore, we replace the non-transferability and uniqueness property of [7] by our betrayability property.

A first difference to [7] is that we introduce identification keys to identify clients during redemption and enable token buyers to incorporate a trapdoor into a vendor-aided token redemption. We include the identification keys in the redemption protocols of the inherited security games. Furthermore, we need to ensure that a client does not redeem a token under the identity of another client. Therefore, we introduce the identity unforgeability property.

Secondly, we observed a weakness in the one-more unforgeability notion of [7]. In particular, the adversary is required to provide token-witnesses pairs that can be redeemed in an honest execution of the redemption protocol in order to win the experiment. However, we assess it to be important that the tokens cannot even be redeemed in a malicious execution of the redemption protocol. Therefore, we adapt the definition accordingly. In our definition, the adversary has access to a query and a redemption oracle and wins if it successfully redeems more distinct tokens than it queried. We note that this definition is much closer to the original one-more unforgeability definition of anonymous tokens introduced by [1].

Third, we explicitly consider a collusion between a token vendor and a token buyer during token redemption. Instead

of requiring that the client redeeming a token knows the corresponding insurance secret as done in [7], we require that the buyer either knows the secret or is able to extract it from the vendor, if the vendor-aided redemption would be successful. Therefore, we have to extend the security game to incorporate two different types of adversaries and allow one of them to betray the other. As our notion of betrayability implies token uniqueness, we do no longer need this property.

Finally, we allow the adversary in the unlinkability game to select the public and private key of the server.

Appendix C. Zero-Knowledge Proofs with Setup

For the range proofs utilized in our construction, we require a zero-knowledge proof with a dedicated setup creating public parameters that are necessary for the proof creation and verification. The setup fixes some parts of the proof statements, e.g. the underlying algebraic group. We formalize zero-knowledge proofs with setup in the random oracle model as follows.

Definition 7. Let $\mathcal{R}$ be a NP-relation, $\mathcal{L}_{\mathcal{R}} = \{(s_0, s_1) : ((s_0, s_1), w) \in \mathcal{R}\}$ be the associated NP-language, and $\mathcal{H}$ be a global random oracle. A non-interactive zero-knowledge proof of knowledge with setup for relation $\mathcal{R}$ consists of the following three polynomial time algorithms (Setup, Prove, Verify) with access to a random oracle $\mathcal{H}$.

- $\text{Setup}(s_0) \rightarrow \text{pp}$: a probabilistic algorithm that takes as input a statement constant s_0 and outputs the public proof parameters rpp .
- $\text{Prove}(\text{rpp}, s_1, w) \rightarrow \pi$: a probabilistic algorithm that takes as input the public proof parameters rpp , a statement component s_1 , and a witness w and outputs a proof π .
- $\text{Verify}(\text{rpp}, s_1, \pi) \rightarrow 0/1$: a deterministic algorithm that takes as input the public proof parameters, a statement component s_1 , and a proof π and outputs either 1 accepting the proof or 0 rejecting it.

The algorithms satisfy the following properties:

- **Completeness:** For all $((s_0, s_1), w) \in \mathcal{R}$ it holds that

$$\Pr \left[\text{Verify}(\text{rpp}, s_1, \pi) = 1 : \begin{array}{l} \text{rpp} \leftarrow \text{Setup}(s_0), \\ \pi \leftarrow \text{Prove}(\text{rpp}, s_1, w) \end{array} \right] = 1.$$

- **Knowledge soundness:** For every $\lambda \in \mathbb{N}$, every auxiliary input $\text{aux} \in \{0, 1\}^*$ of size polynomial in λ , every statement constant $s_0 \in \{0, 1\}^*$ of size polynomial in λ , every PPT adversary $\mathcal{A}$, and every random tape r , there exists a PPT extractor $\mathcal{E}$ with oracle access to $\mathcal{A}$ and control over random oracle $\mathcal{H}$, as well as a negligible function negl , such that

$$\Pr \left[\begin{array}{l} \text{Verify}((s_0, s_1), \pi) = 1 \\ \wedge ((s_0, s_1), w) \notin \mathcal{R} \end{array} : \begin{array}{l} \text{rpp} \leftarrow \text{Setup}(s_0) \\ (s_1, \pi) \leftarrow \mathcal{A}(\text{rpp}, \text{aux}; r) \\ w \leftarrow \mathcal{E}^{\mathcal{H}, \mathcal{A}}(\text{rpp}, \text{aux}, r) \end{array} \right] \leq \text{negl}(\lambda).$$

Construction 2: Partial request proof

PK_{Rq}¹.Prove((pp, X, T, {D_i}_{i∈[L]}), (r, δ, {r_i, x_i}_{i∈[L]}):

- 1) Sample $\alpha_\delta, \alpha_1, \dots, \alpha_L, \beta_0, \beta_1, \dots, \beta_L \xleftarrow{\$} \mathbb{Z}_p$.
- 2) Compute $U_T = \alpha_\delta T + \beta_0 H_0 + \sum_{i \in [L]} \alpha_i H_i$ and $U_X = \sum_{i \in [L]} (\alpha_i \cdot 2^{(i-1)\ell}) G_1$.
- 3) For $i \in [L]$ compute $U_i = \alpha_i G_0 + \beta_i H_0$.
- 4) Compute $c = \mathcal{H}(\text{pp}, X, T, \{D_i\}_{i \in [L]}, U_T, U_X, \{U_i\}_{i \in [L]})$.
- 5) Compute $\gamma_\delta = \alpha_\delta + c \cdot \frac{1}{\delta}$ and $\gamma_0^R = \alpha_0 - c \cdot r$.
- 6) For $i \in [L]$ compute $\gamma_i^X = \alpha_i - c \cdot x_i$ and $\gamma_i^R = \beta_i - c \cdot r_i$.
- 7) Return $\pi = (c, \gamma_\delta, \gamma_0^R, \{\gamma_i^X, \gamma_i^R\}_{i \in [L]})$.

PK_{Rq}¹.Verify((pp, X, T, {D_i}_{i∈[L]}), π):

- 8) Compute $W_T = -cG_0 + \gamma_\delta T + \gamma_0^R H_0 + \sum_{i \in [L]} \gamma_i^X H_i$.
- 9) Compute $W_X = cX + \sum_{i \in [L]} (\gamma_i^X \cdot 2^{(i-1)\ell}) \cdot G_1$.
- 10) For $i \in [L]$ compute $W_i = cD_i + \gamma_i^X G_0 + \gamma_i^R H_0$.
- 11) Compute $c' = \mathcal{H}(\text{pp}, X, T, \{D_i\}_{i \in [L]}, W_T, W_X, \{W_i\}_{i \in [L]})$.
- 12) Output 1 if $c = c'$ and 0 otherwise.

Construction 3: Request proof Wrapper

PK_{Rq}.Setup(p, $\mathbb{G}_1$, G_0 , H_0 , L, ℓ):

- 1) Compute $\text{rpp}' \leftarrow \text{PK}_{\text{Rq}}^2.\text{Setup}(p, \mathbb{G}_1, G_0, H_0, L, \ell)$.
- 2) Output $\text{rpp} \leftarrow (p, \mathbb{G}_1, G_0, H_0, L, \ell, \text{rpp}')$.

PK_{Rq}.Prove(rpp, (pp, pk_c^{In} , T), ({x_i}_{i∈[L]}, r, δ)):

- 3) For $i \in [L]$ sample $r_i \xleftarrow{\$} \mathbb{Z}_p$ and compute $D_i = x_i G_0 + r_i H_0$.
- 4) Compute $\pi_1 = \text{PK}_{\text{Rq}}^1.\text{Prove}((\text{pp}, \text{pk}_c^{\text{In}}, T, \{D_i\}_{i \in [L]}), (r, \delta, \{r_i, x_i\}_{i \in [L]}))$.
- 5) Compute $\pi_2 = \text{PK}_{\text{Rq}}^2.\text{Prove}(\text{rpp}, \{D_i\}_{i \in [L]}, \{x_i, r_i\}_{i \in [L]})$.
- 6) Output $\pi = (\pi_1, \pi_2, \{D_i\}_{i \in [L]})$.

PK_{Rq}.Verify(rpp, (pp, pk_c^{In} , T), π):

- 7) Output 1 if $\text{PK}_{\text{Rq}}^1.\text{Verify}((\text{pp}, \text{pk}_c^{\text{In}}, T, \{D_i\}_{i \in [L]}), \pi_1) = 1$ and $\text{PK}_{\text{Rq}}^2.\text{Verify}(\text{rpp}, \{D_i\}_{i \in [L]}, \pi_2) = 1$. Otherwise, output 0.

Figure 7: Instantiations of the request proof systems.

- **Zero-knowledge:** There exists a PPT simulator $\mathcal{S}$ controlling oracle $\mathcal{H}$ such that for every group $\lambda \in \mathbb{N}$ and for every $((s_0, s_1), w) \in \mathcal{R}$ of size polynomial in λ , it holds that

$$\{(\text{rpp}, \pi) : \text{rpp} \leftarrow \text{Setup}(s_0), \pi \leftarrow \text{Prove}(\text{rpp}, s_1, w)\} \approx_c \{\mathcal{S}^{\mathcal{H}}(\text{rpp}, (s_0, s_1))\}.$$

Appendix D. Instantiations of our Zero-Knowledge Proof Systems

This section presents instantiations for the proof systems introduced in Section 4.1. We slightly abuse the notation and assume the protocol's public parameters $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, G_2, \{H_i\}_{i \in [0, L]}, L, \ell, \text{rpp})$ to be part of every statement even though the statement only contains the relevant components of the public parameters.

Within the request proof system, we make use of two different proof types, a zero-knowledge proof of knowledge to prove knowledge of $(\{x_i\}_{i \in [L]}, r, \delta)$ as well as a range proof to prove that the chunks are in the correct range $(x_i \in [0, 2^\ell - 1] \text{ for } i \in [L])$. Therefore, we split the request

Construction 4: Challenge Proof
PK_{Ch}.Prove((pp, K, pk_c^{ld}), d): 1) Set $D_0 = K$ and $D_1 = \text{pk}_c^{\text{ld}}$. 2) Set $b = 0$ if $D_0 = dG_0$ and $b = 1$ otherwise. Set $\bar{b} = 1 - b$. 3) Sample $\alpha_b, \gamma_{\bar{b}}, c_{\bar{b}} \xleftarrow{\\$} \mathbb{Z}_p$ and compute $U_b = \alpha_b G_b$ and $U_{\bar{b}} = \gamma_{\bar{b}} G_{\bar{b}} - c_{\bar{b}} D_{\bar{b}}$. 4) Compute $c = \mathcal{H}(\text{pp}, D_0, D_1, U_0, U_1)$ and $c_b = c - c_{\bar{b}}$. 5) Compute $\gamma_b = \alpha_b + d \cdot c_b$ and output $\pi = (U_0, U_1, \gamma_0, \gamma_1, c_0)$. PK_{Ch}.Verify((pp, K, pk_c^{ld}), π): 6) Compute $c' = \mathcal{H}(\text{pp}, K, \text{pk}_c^{\text{ld}}, U_0, U_1)$ and $c'_1 = c' - c_0$. 7) Output 1 if $\gamma_0 G_0 = U_0 + c_0 K$ and $\gamma_1 G_1 = U_1 + c'_1 \text{pk}_c^{\text{ld}}$ and 0 otherwise.
Construction 5: Redemption Proof
PK_{Rd}.Prove((pp, Y, K, σ, σ', $\{A_i, B_i\}_{i \in [L]}$), (y, r, e, $\{r_i, x_i\}_{i \in [L]}$)): 1) Sample $\alpha_y, \alpha_r, \alpha_e, \alpha_1, \dots, \alpha_L, \beta_1, \dots, \beta_L \xleftarrow{\\$} \mathbb{Z}_p$. 2) Compute $U_Y = \alpha_y G_1$ and $U_T = \alpha_r H_0 + \sum_{i \in [L]} \alpha_i H_i - \alpha_e \sigma$. 3) For $i \in [L]$ compute $U_i^A = \beta_i G_1$ and $U_i^B = \beta_i K + \alpha_i G_1$. 4) Compute $c = \mathcal{H}(\text{pp}, Y, \sigma, \sigma', K, \{A_i, B_i\}_{i \in [L]}, U_Y, U_T, \{U_i^A, U_i^B\}_{i \in [L]})$. 5) Compute $\gamma_y = \alpha_y - c \cdot y$, $\gamma_0^R = \alpha_r - c \cdot r$ and $\gamma_e = \alpha_e - c \cdot e$. 6) For $i \in [L]$ compute $\gamma_i^X = \alpha_i - c \cdot x_i$ and $\gamma_i^R = \beta_i - c \cdot r_i$. 7) Output $\pi = (c, \gamma_y, \gamma_0^R, \gamma_e, \{\gamma_i^X, \gamma_i^R\}_{i \in [L]})$. PK_{Rd}.Verify((pp, Y, K, σ, σ', $\{A_i, B_i\}_{i \in [L]}$), π): 8) Compute $W_Y = cY + \gamma_y G_1$ and $W_T = c(\sigma - G_0) + \gamma_0^R H_0 - \gamma_e \sigma + \sum_{i \in [L]} \gamma_i^X H_i$. 9) For $i \in [L]$ compute $W_i^A = cA_i + \gamma_i^R G_1$ and $W_i^B = cB_i + \gamma_i^R K + \gamma_i^X G_1$. 10) Compute $c' = \mathcal{H}(\text{pp}, Y, \sigma, \sigma', K, \{A_i, B_i\}_{i \in [L]}, W_Y, W_T, \{W_i^A, W_i^B\}_{i \in [L]})$. 11) Output 1 if $c = c'$ and 0 otherwise.

Figure 8: Instantiations of the challenge and redemption proof systems.

proof into two separate components with the following proof relations:

$$\begin{aligned}
\text{PK}_{\text{Rq}}^1 &= \text{PK} \left[\begin{array}{l} (r, \delta, \{x_i, r_i\}_{i \in [L]}) : \\ X = (\sum_{i \in [L]} 2^{(i-1)\ell} x_i) G_1 \\ T = \delta(G_0 + rH_0 + \sum_{i \in [L]} x_i H_i) \\ D_i = x_i G_0 + r_i H_0 \text{ for } i \in [L] \end{array} \right] \\
\text{PK}_{\text{Rq}}^2 &= \text{PK} \left[\begin{array}{l} (\{x_i, r_i\}_{i \in [L]}) : \\ D_i = x_i G_0 + r_i H_0 \text{ for } i \in [L] \\ x_i \in [0, 2^\ell - 1] \text{ for } i \in [L] \end{array} \right].
\end{aligned}$$

As we instantiate the range proof with the Bulletproof++ proof system [24], we have to incorporate Pedersen commitments to the chunks into both relations. In order to instantiate a zero-knowledge proof for the poof relation PK_{Rq} , we define a wrapper proof system ($\text{PK}_{\text{Rq}}.\text{Setup}$, $\text{PK}_{\text{Rq}}.\text{Prove}$, $\text{PK}_{\text{Rq}}.\text{Verify}$) that creates the required commitments to the chunks and creates proofs for the two proof relations PK_{Rq}^1 and PK_{Rq}^2 individually. We assume ($\text{PK}_{\text{Rq}}.\text{Setup}$, $\text{PK}_{\text{Rq}}.\text{Prove}$, $\text{PK}_{\text{Rq}}.\text{Verify}$) to be a secure zero-knowledge proof of knowledge with setup for relation PK_{Rq}^2 , e.g., instantiated with the Bulletproof++ proof system, and instantiate the proof systems for proof relation

PK_{Rq}^1 and the wrapper as in Figure 8. The challenge proof system and the redemption proof system are instantiated as in Figure 8.

Appendix E. Incorporating Metadata

Given that our scheme builds on BBS signatures, it can easily be extended to support public metadata and client-side private metadata.

Public metadata can be added by requiring both client and server to replace G_0 by $G'_0 = G_0 + \mathcal{H}_{\text{pm}}(\text{md}) \cdot H_{\text{pm}}$ for additional public parameter $H_{\text{pm}} \xleftarrow{\$} \mathbb{G}_1$, collision resistant hash function $\mathcal{H}_{\text{pm}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p$, and metadata md per session. We note that public metadata limits the anonymity set of a redeemed token to all other tokens issued for the same metadata. Therefore, it is recommended to pick coarse grained metadata that is shared with sufficiently many other clients in order to sustain the anonymity guarantees of the scheme.

Client-side private metadata can be added by replacing the commitment $C_i = G_0 + \mathcal{H}_{\text{pm}}(\text{md}) \cdot H_{\text{pm}} + rH_0 + \sum_{i \in [L]} x_i H_i$ with $C'_i = C_i + \sum_{j \in [N]} s_j H_j^{\text{sm}}$ for N metadata elements s_j encoded as field elements and additional public parameters $H_1^{\text{sm}}, \dots, H_N^{\text{sm}} \xleftarrow{\$} \mathbb{G}_1$. Obviously, we have to extend the request and redemption proof such that the client additionally proves knowledge of the secret metadata. This can be done with the same technique used to prove knowledge of the insurance key chunks. If necessary, the scheme can be extended to require clients to prove statements about the private metadata with zero-knowledge proof techniques already known from BBS(+) signatures [18], [20], [21].

Appendix F.

Proof of Theorem 4

This section provides a detailed proof of Theorem 4. Let $\mathcal{A}$ be a generic PPT adversary. We prove one-more unforgeability

$$\Pr[\text{Exp}_{\mathcal{A}}^{\text{OMUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

request unforgeability

$$\Pr[\text{Exp}_{\mathcal{A}}^{\text{RUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

and identity unforgeability

$$\Pr[\text{Exp}_{\mathcal{A}}^{\text{IUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

separately. By combining the three, we can conclude that Theorem 4 holds.

F.1. One-more Unforgeability

We prove one-more unforgeability via a sequence of games. Let $\text{Exp}_0^{\text{OMUF}} = \text{Exp}_{\mathcal{A}}^{\text{OMUF}}$ be the initial security game. The subsequent games are defined as follows.

Inline construction. In the first game, we inline our concrete construction into the security game (where relevant). The game is defined as follows:

Experiment: $\text{Exp}_1^{\text{OMUF}}(\lambda)$
1: $q_{\text{Rq}}, q_{\text{Rd}} = 0, \Gamma \leftarrow \emptyset, \text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp})$ 2: $\mathcal{A}^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, \text{pk}_s)$, if $q_{\text{Rq}} < q_{\text{Rd}}$ return 1 else return 0 $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query})$: 3: $q_{\text{Rq}} \leftarrow q_{\text{Rq}} + 1, (T, \pi_{\text{Rq}}) \leftarrow \text{query}$ 4: if $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}^{\text{In}}, T), \pi_{\text{Rq}}) = 0$ then return $\emptyset$. 5: $e \xleftarrow{\$} \mathbb{Z}_p, S \leftarrow \frac{1}{\text{sk}_s + e} T$ 6: return (e, S) $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 7: if $\text{st}_{\text{sid}} \in \{0, 1\}$ then return $\perp$ 8: if $\text{st}_{\text{sid}} = \text{null}$ then 9: $(\sigma_{\text{sid}}, \text{pk}_{\text{sid}}^{\text{ld}}) \leftarrow m, k_{\text{sid}} \xleftarrow{\$} \mathbb{Z}_p, K_{\text{sid}} \leftarrow k_{\text{sid}} G_0$ 10: $\pi_{\text{sid}} \leftarrow \text{PK}_{\text{Ch}}.\text{Prove}((K_{\text{sid}}, \text{pk}_{\text{sid}}^{\text{ld}}), k_{\text{sid}})$ 11: $\text{st}_{\text{sid}} \leftarrow (\sigma_{\text{sid}}, \text{pk}_{\text{sid}}^{\text{ld}}, K_{\text{sid}})$ 12: return $(K_{\text{sid}}, \pi_{\text{sid}})$ 13: else 14: $(\pi_{\text{Rd}}, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 15: if $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}} \leftarrow 0$, return 0 16: if $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}_{\text{sid}}, K_{\text{sid}}, \sigma_{\text{sid}}, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0$ then $\text{st}_{\text{sid}} \leftarrow 0$, return 0 17: if $\sigma_{\text{sid}} \notin \Gamma$ then $q_{\text{Rd}} \leftarrow q_{\text{Rd}} + 1, \Gamma \leftarrow \Gamma \cup \{\sigma_{\text{sid}}\}$ 18: $\text{st}_{\text{sid}} \leftarrow 1$, return 1

This step does not change the output behavior. It follows that

$$\Pr[\text{Exp}_1^{\text{OMUF}}(\lambda) = 1] = \Pr[\text{Exp}_0^{\text{OMUF}}(\lambda) = 1]$$

Extract request proofs. In the next experiment, we adapt the request oracle. After successfully verifying the request proof, we employ the extractor $\mathcal{E}_{\text{Rq}}$ of the request proof system ($\text{PK}_{\text{Rq}}.\text{Setup}, \text{PK}_{\text{Rq}}.\text{Prove}, \text{PK}_{\text{Rq}}.\text{Verify}$) to extract

the witness of the request proof. The extractor has access to the adversary $\mathcal{A}$, receives the transcript, trans, of the experiment execution up to the extraction including the adversary's random tape, and has the power to program the random oracle. It extracts the witnesses $(\{x_i\}_{i \in [L]}, r, \delta)$ that are all either $\perp$ or satisfy the proof system relation PK_{Rq} . If the witnesses do not satisfy the proof relation, we abort the experiment. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_2^{\text{OMUF}}(\lambda)$
... $\mathcal{O}^{\text{Rq}}(\text{pk}^{\text{In}}, \text{query})$: 3: $q_{\text{Rq}} \leftarrow q_{\text{Rq}} + 1, (T, \pi_{\text{Rq}}) \leftarrow \text{query}$ 4: if $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}^{\text{In}}, T), \pi_{\text{Rq}}) = 0$ then return $\emptyset$. 5: $(\{x_i\}_{i \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}(\cdot)}(\text{trans})$, if $r = \perp$ then return $\perp$ 6: $e \xleftarrow{\$} \mathbb{Z}_p, S \leftarrow \frac{1}{\text{sk}_s + e} T$ 7: return (e, S) ...

It is obvious that $\text{Exp}_2^{\text{OMUF}}$ and $\text{Exp}_1^{\text{OMUF}}$ only differ if the extractor is not successful, which happens with negligible probability due to the knowledge soundness property of the request proof system. This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_2^{\text{OMUF}}(\lambda) = 1] - \Pr[\text{Exp}_1^{\text{OMUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Extract redemption proofs. Next, we adapt the redemption oracle to extract the witness of the redemption proof π_{Rd} after successfully verifying π_{Rd} . To do so, we use $\mathcal{E}_{\text{Rd}}$ of the redemption proof system, that succeeds expect with negligible probability. As before, the extractor has access to the adversary $\mathcal{A}$, receives the transcript, trans, of the experiment execution up to the extraction including the adversary's random tape, and has the power to program the random oracle. It extracts the witnesses $(\text{sk}^{\text{ld}}, r, e, \{r_i, x_i\}_{i \in [L]})$ that are either all $\perp$ or satisfy the redemption proof relation PK_{Rd} . The resulting experiment is defined as follows:

Experiment: $\text{Exp}_3^{\text{OMUF}}(\lambda)$
... $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 8: ... 14: else 15: $(\pi_{\text{Rd}}, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 16: if $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}} \leftarrow 0$, return 0 17: if $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}_{\text{sid}}, K_{\text{sid}}, \sigma_{\text{sid}}, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0$ then $\text{st}_{\text{sid}} \leftarrow 0$, return 0 18: $(\text{sk}^{\text{ld}}, r, e, \{r_i, x_i\}_{i \in [L]}) \leftarrow \mathcal{E}_{\text{Rd}}^{\mathcal{A}(\cdot)}(\text{trans})$ 19: if $e = \perp$ then $\text{st}_{\text{sid}} \leftarrow 0$, return 0 20: if $\sigma_{\text{sid}} \notin \Gamma$ then $q_{\text{Rd}} \leftarrow q_{\text{Rd}} + 1, \Gamma \leftarrow \Gamma \cup \{\sigma_{\text{sid}}\}$ 21: $\text{st}_{\text{sid}} \leftarrow 1$, return 1

It is obvious that $\text{Exp}_3^{\text{OMUF}}$ and $\text{Exp}_2^{\text{OMUF}}$ only differ if the extractor is not successful, which happens with negligible probability due to the knowledge soundness property of the redemption proof system. This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_3^{\text{OMUF}}(\lambda) = 1] - \Pr[\text{Exp}_2^{\text{OMUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Simplification. In a bridging step, we adapt the issuance oracle to compute the signatures based on the extracted

witnesses of the request proof. As both ways of computing the (blinded) token result in exactly the same token, this change does not influence the output distribution. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_4^{\text{OMUF}}(\lambda)$

```

...  $\mathcal{O}^{\text{Rq}}(\text{pk}^{\text{In}}, \text{query})$  :
3:  $q_{\text{Rq}} \leftarrow q_{\text{Rq}} + 1$ ,  $(T, \pi_{\text{Rq}}) \leftarrow \text{query}$ 
4: if  $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}^{\text{In}}, T), \pi_{\text{Rq}}) = 0$  then return  $\emptyset$ .
5:  $(\{x_i\}_{i \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}(\cdot)}(\text{trans})$ , if  $r = \perp$  then return  $\perp$ 
6:  $e \xleftarrow{\$} \mathbb{Z}_p$ ,  $\sigma \leftarrow \frac{1}{\text{sk}_s + e} (G_0 + rH_0 + \sum_{j \in [L]} x_j H_j)$ ,  $S \leftarrow \delta \cdot \sigma$ 
7: return  $(e, S)$ 
...

```

We conclude that

$$\Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1] = \Pr[\text{Exp}_3^{\text{OMUF}}(\lambda) = 1].$$

Reduction to BBS signatures. Note that in $\text{Exp}_4^{\text{OMUF}}$, the adversary can only receive q_{Rq} signatures but has to provide $q + 1$ signatures with distinct first signature components σ in order to win the experiment. Therefore, it follows that at least one of the submitted tokens is a fresh BBS signature. This allows us to prove that $\Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$ via a reduction to Theorem 1. Formally, we pick $L'(\lambda)$ such that, for $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \text{GGen}(1^\lambda)$, $G_1 \xleftarrow{\$} \mathbb{G}_1^*$, and $\ell = \lceil \frac{|p|}{L'} \rceil$, the computation of $\text{dlog}_{G_1}(G)[0, 2^\ell - 1]$ has complexity below $\mathcal{C}$ for all $G \in \mathbb{G}_1$. Then, we define an adversary $\mathcal{B}$ on security game $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}(\lambda)$ that internally executes $\mathcal{A}$ on security game $\text{Exp}_4^{\text{OMUF}}(\lambda)$ and wins with the same probability as $\mathcal{A}$. We define $\mathcal{B}$ as follows:

- Receive $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, \{H_i\}_{i \in [L'+1]}, G_2, Z)$ from the $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}$ security game.
- Pick ℓ such that $\text{dlog}_{G_1}(G)[0, 2^\ell - 1]$ can be computed with complexity of at most $\mathcal{C}$ for any $G \in \mathbb{G}_1$ and set $L \leftarrow \lceil \frac{|p|}{\ell} \rceil$. Based on the definition of L' it holds that $L \leq L'$.
- Define $H_0 \leftarrow H_{L+1}$ and compute $\text{rpp} = \text{RP.Setup}(p, \mathbb{G}_1, G_0, H_0, L, \ell)$.
- Define $\text{pp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, H_0, H_1, \dots, H_L, L, \ell, \text{rpp})$.
- Initialize $\Gamma, \Gamma_{\text{Rq}}, \Gamma_{\text{Rd}} \leftarrow \emptyset$.
- Execute $\mathcal{A}^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, Z)$ and answer oracle queries as follows:
 - To answer $\mathcal{O}^{\text{Rq}}$, execute the steps of the oracle as defined in $\text{Exp}_4^{\text{OMUF}}$ but instead of sampling e and calculating σ (which is not possible, as sk_s is not known), query $\mathcal{O}^{\text{Sign}}(\{x_i\}_{i \in [L]} \cup \{r\} \cup \{0\}_{i \in L' \setminus L})$ of the $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}$ game to receive (e, σ) and store $(\{x_i\}_{i \in [L]} \cup \{r\}, \sigma, e)$ in Γ_{Rq} .
 - $\mathcal{O}^{\text{Rd}}$ can be executed as defined in $\text{Exp}_4^{\text{OMUF}}$ since it does not need knowledge of the secret key. However, when adding σ_{sid} to Γ , $\mathcal{B}$ also adds $(\{x_i\}_{i \in [L]} \cup \{r\} \cup \{0\}_{i \in L' \setminus L}, \sigma_{\text{sid}}, e)$ to Γ_{Rd} .
- If $q_{\text{Rq}} \geq q_{\text{Rd}}$ return $\perp$.
- Otherwise, define $(\{x_i^*\}_{i \in [L]} \cup \{r^*\}, \sigma^*, e^*)$ to be the first element of $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ and return $(\{x_i^*\}_{i \in [L]} \cup \{r^*\}, \sigma^*, e^*)$.

First note, that the view of $\mathcal{A}$ in the experiment simulated by $\mathcal{B}$ is identical to the view of $\mathcal{A}$ in experiment $\text{Exp}_4^{\text{OMUF}}$. The public parameters are generated identically in both experiments and oracle queries are answered identically as well. To analyze the success probability of $\mathcal{B}$, we first note

that all elements in Γ_{Rd} are valid BBS message-signature tuples under public key $Z = zG_2$ (where z is unknown). Note that for each tuple $(\{x_i\}_{i \in [L]} \cup \{r\}, \sigma, e)$ (and associated σ') that is added to Γ_{Rd} it holds that $e(\sigma, Z) = e(\sigma', G_2)$ (explicitly checked) and $\sigma' = G_0 + rH_0 - e\sigma + \sum_{i \in [L]} x_i H_i$ (due to the success of the knowledge extractor). The former implies that $\sigma' = z\sigma$ which allows us to conclude that $\sigma = \frac{1}{z+e}(G_0 + rH_0 + \sum_{i \in [L]} x_i H_i)$ and, hence, that $(\{x_i\}_{i \in [L]} \cup \{r\}, \sigma, e)$ is a valid BBS message-signature tuple if added to Γ_{Rd} .

Next, we note that any element in $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ (if existent) is a fresh BBS signature. This follows straightforwardly from the fact that Γ_{Rq} contains all issued BBS signatures.

Finally, we note that $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ contains at least one element if $\mathcal{A}$ would win $\text{Exp}_4^{\text{OMUF}}$. In order for $\mathcal{A}$ to win, it needs to hold that $q_{\text{Rq}} \geq q_{\text{Rd}}$. Furthermore, the size of Γ_{Rq} is at most q_{Rq} (if the zero-knowledge proof is invalid, q_{Rq} is incremented but Γ_{Rq} remains untouched) and the size of Γ_{Rd} is equal to q_{Rd} . Therefore, it needs to hold that Γ_{Rd} is larger than Γ_{Rq} and, hence, that $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ contains at least one element, if $\mathcal{A}$ would win $\text{Exp}_4^{\text{OMUF}}$.

As we have shown that Γ_{Rd} contains only valid BBS signatures, $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ contains only fresh BBS signatures (if any), and that $\Gamma_{\text{Rd}} \setminus \Gamma_{\text{Rq}}$ contains at least one element if $\mathcal{A}$ would win in $\text{Exp}_4^{\text{OMUF}}$, we conclude that $\mathcal{B}$ outputs a valid forgery if $\mathcal{A}$ would win in $\text{Exp}_4^{\text{OMUF}}$ and, hence that

$$\Pr[\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}(\lambda) = 1] = \Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1].$$

Putting all steps together. Above, we have shown that

$$\begin{aligned} \Pr[\text{Exp}_1^{\text{OMUF}}(\lambda) = 1] &= \Pr[\text{Exp}_0^{\text{OMUF}}(\lambda) = 1] \\ |\Pr[\text{Exp}_2^{\text{OMUF}}(\lambda) = 1] - \Pr[\text{Exp}_1^{\text{OMUF}}(\lambda) = 1]| &\leq \text{negl}(\lambda). \\ |\Pr[\text{Exp}_3^{\text{OMUF}}(\lambda) = 1] - \Pr[\text{Exp}_2^{\text{OMUF}}(\lambda) = 1]| &\leq \text{negl}(\lambda). \\ \Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1] &= \Pr[\text{Exp}_3^{\text{OMUF}}(\lambda) = 1] \\ \Pr[\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}(\lambda) = 1] &= \Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1] \end{aligned}$$

Further Theorem 1 states that $\Pr[\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}(\lambda) = 1] \leq \text{negl}(\lambda)$. Therefore, we conclude that

$$\Pr[\text{Exp}_4^{\text{OMUF}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

F.2. Request Unforgeability

We prove request unforgeability via a sequence of games. Let $\text{Exp}_0^{\text{RUF}} = \text{Exp}_{\mathcal{A}}^{\text{RUF}}$ be the initial security game. The subsequent games are defined as follows.

Inline construction. In the first game, we inline our concrete construction into the security game's main thread. The game is defined as follows:

Experiment: $\text{Exp}_1^{\text{RUF}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ ,  $(\text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 
2:  $((T^*, \pi^*), \text{sk}_s) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, \text{pk}_c^{\text{In}}, \text{pk}_c^{\text{Id}})$ 
3: if  $\exists i$  s.t.  $(T^*, \pi^*) = (T_i, \pi_i)$  then return 0
4: if  $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}_c^{\text{In}}, T^*), \pi^*) = 0$  then return 0
5: return 1

```

```

 $\mathcal{O}^{\text{Qu}}(i, \text{pk}):$ 
6: if  $T_i \neq \text{null}$  then return  $\perp$ 
7: Select  $x_1, \dots, x_L \in [0, 2^\ell - 1]$  such that  $\text{sk}_c^{\text{In}} = \sum_{j \in [L]} 2^{(j-1)\ell} x_j$  holds without modulo reduction (This representation is unique and independent of  $i$ )
8:  $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p$ ,  $C_i = G_0 + r_i H_0 + \sum_{j \in [L]} x_j H_j$ ,  $\text{pk}_i \leftarrow \text{pk}$ 
9:  $T_i \leftarrow \delta_i C_i$ ,  $\pi_i \leftarrow \text{PK}_{\text{Rq}}.\text{Prove}(\text{rpp}, (\text{pk}_c^{\text{In}}, T_i), (\{x_j\}_{j \in [L]}, r_i, \delta_i))$ 
10: return  $(T_i, \pi_i)$ 

 $\mathcal{O}^{\text{Fin}}(i, \text{resp} = (e, S)):$ 
11: if  $T_i = \text{null} \vee \sigma_i \neq \text{null}$  then return  $\perp$ 
12: if  $e(S, \text{pk}_i + eG_2) \neq e(T_i, G_2)$  then  $\sigma_i \leftarrow \perp$ , return  $\sigma_i$ 
13: else  $\sigma_i \leftarrow \frac{1}{\delta_i} S$ ,  $e_i \leftarrow e$ , return  $\sigma_i$ 

 $\mathcal{O}^{\text{Rd}}(\text{sid}, m):$ 
14: if  $\text{st}_{\text{sid}} = \text{null}$  then
15:    $i \leftarrow m$ , if  $\sigma_i \in \{\text{null}, \perp\}$  then return  $\perp$ 
16:    $\text{st}_{\text{sid}} \leftarrow i$ , return  $(\text{pk}_c^{\text{Id}}, \sigma_i)$ 
17: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
18:    $i \leftarrow \text{st}_{\text{sid}}$ ,  $(K, \pi_{\text{Ch}}) \leftarrow m$ ,  $\text{st}_{\text{sid}} \leftarrow \perp$ 
19:   if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{Id}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
20:    $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma_i + \sum_{j \in [L]} x_j H_j$ 
21:    $\forall j \in [L] : r_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow r_j K + x_j G_1$ 
22:    $\pi_{\text{Rd}} \leftarrow \text{PK}_{\text{Rd}}.\text{Prove}((\text{pk}_c^{\text{Id}}, K, \sigma_i, \sigma', \{r_j, x_j\}_{j \in [L]}), (\text{sk}_c^{\text{Id}}, r_i, e_i, \{r_j, x_j\}_{j \in [L]}))$ 
23:   return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
24: else return  $\emptyset$ 

```

This step does not change the output distribution. It follows that

$$\Pr[\text{Exp}_1^{\text{RUF}}(\lambda) = 1] = \Pr[\text{Exp}_0^{\text{RUF}}(\lambda) = 1].$$

Simulate redemption proofs. First, we simplify the redemption oracle by using the redemption proof's simulator $\mathcal{S}_{\text{Rd}}$ instead of the proof system's witnesses to create redemption proofs. The simulator receives the proof's statement $(\text{pp}, \text{pk}_c^{\text{Id}}, K, \sigma_i, \{A_j, B_j\}_{j \in [L]})$ and has the capability of programming the random oracle. It creates a valid zero knowledge proof π_{Rd} , except with negligible probability. The resulting experiment is defined as follows:

```

Experiment:  $\text{Exp}_2^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Rd}}(\text{sid}, m):$ 
14: ...
17: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
18:    $i \leftarrow \text{st}_{\text{sid}}$ ,  $(K, \pi_{\text{Ch}}) \leftarrow m$ ,  $\text{st}_{\text{sid}} \leftarrow \perp$ 
19:   if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{Id}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
20:    $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma_i + \sum_{j \in [L]} x_j H_j$ 
21:    $\forall j \in [L] : r_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow r_j K + x_j G_1$ 
22:    $\pi_{\text{Rd}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_c^{\text{Id}}, K, \sigma_i, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
23:   return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
24: else return  $\emptyset$ 

```

Note that the redemption proofs in $\text{Exp}_1^{\text{RUF}}$ are always valid due to the completeness of the proof system and the fact that the proof relation always holds (signatures have been verified before defining $\sigma \neq \perp$, identity keys are generated honestly, and ciphertexts are created as expected). It follows that $\text{Exp}_2^{\text{RUF}}$ and $\text{Exp}_1^{\text{RUF}}$ only differ if the simulator is not successful, which happens with at most negligible probability due to the zero-knowledge property of

the redemption proof system. We use the difference lemma to conclude that

$$|\Pr[\text{Exp}_2^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_1^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Extract challenge proof. In the next game, we further adapt the redemption oracle to extract the witness of the challenge proof. After successfully verifying the challenge proof π_{Ch} , we employ extractor $\mathcal{E}_{\text{Ch}}$ of the challenge proof system to extract the witness of the challenge proof. The extractor has access to the adversary $\mathcal{A}$, receives the transcript trans of the experiment execution up to the extraction including the adversary's random tape, and has the power to program the random oracle. It extracts the witness k that is either $\perp$ or satisfies the proof system relation PK_{Ch} . If the witness does not satisfy the proof relation, we return an empty message. The resulting experiment is defined as follows:

```

Experiment:  $\text{Exp}_3^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Rd}}(\text{sid}, m, i):$ 
14: ...
17: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
18:    $i \leftarrow \text{st}_{\text{sid}}$ ,  $(K, \pi_{\text{Ch}}) \leftarrow m$ ,  $\text{st}_{\text{sid}} \leftarrow \perp$ 
19:   if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{Id}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
20:    $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma_i + \sum_{j \in [L]} x_j H_j$ 
21:    $k \leftarrow \mathcal{E}_{\text{Ch}}^{\mathcal{A}(\cdot)}(\text{trans})$ , if  $k = \perp$  then return  $\perp$ 
22:    $\forall j \in [L] : r_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow r_j K + x_j G_1$ 
23:    $\pi_{\text{Rd}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_c^{\text{Id}}, K, \sigma_i, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
24:   return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
25: else return  $\emptyset$ 

```

It is obvious that $\text{Exp}_3^{\text{RUF}}$ and $\text{Exp}_2^{\text{RUF}}$ only differ if the extractor is not successful, which happens with negligible probability due to the knowledge soundness property of the challenge proof system. This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_3^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_2^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Dummy ciphertexts if the decryption key is useless. Next, we further adapt the redemption oracle to randomly sample the second ciphertext component if the ElGamal key K is useless, i.e., if $K = kG_0$ (remember that ciphertexts are computed with respect to G_1). The resulting experiment is defined as follows:

```

Experiment:  $\text{Exp}_4^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Rd}}(\text{sid}, m, i):$ 
14: ...
17: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
18:    $i \leftarrow \text{st}_{\text{sid}}$ ,  $(K, \pi_{\text{Ch}}) \leftarrow m$ ,  $\text{st}_{\text{sid}} \leftarrow \perp$ 
19:   if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{Id}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
20:    $k \leftarrow \mathcal{E}_{\text{Ch}}^{\mathcal{A}(\cdot)}(\text{trans})$ , if  $k = \perp$  then return  $\perp$ 
21:    $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma_i + \sum_{j \in [L]} x_j H_j$ 
22:   if  $K = kG_0$  then
23:      $\forall j \in [L] : r_j, \bar{r}_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow \bar{r}_j G_1$ 
24:   else then (requires that  $\text{pk}_c^{\text{Id}} = kG_1$ )
25:      $\forall j \in [L] : r_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow r_j K + x_j G_1$ 

```

```

26:  $\pi_{\text{Rd}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_c^{\text{ld}}, K, \sigma_i, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
27: return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
28: else return  $\emptyset$ 

```

Indistinguishability between $\text{Exp}_4^{\text{RUF}}$ from $\text{Exp}_3^{\text{RUF}}$ follows intuitively from the semantic security of the ciphertexts and formally from the Decisional Diffie–Hellman assumption in $\mathbb{G}_1$.⁴ To see this, note that (G_1, A_*, G_0, B_*) can be constructed from a DDH tuple $(G_1, A = aG_1, B = bG_1, C = cG_1)$ with c being either random or equal to (ab) . To do so, we define $G_1 = G_1, A_* = A, G_0 = B, B_* = kC + x_*G_1$. If $c = ab$, it holds that $B_* = kaG_0 + \bar{x}_*G_1$, which makes the experiment derived from the DDH tuple equivalent to $\text{Exp}_3^{\text{RUF}}$. If c is random, it holds that B_* is distributed uniformly random, which makes the derived experiment equivalent to $\text{Exp}_4^{\text{RUF}}$. As we can only replace one ciphertext after another, we utilize a sequence of $n \cdot L$ hybrids, assuming that the adversary can query the redemption oracle at most n times. As the DDH assumption guarantees that the difference in the success probability between any two adjacent intermediate hybrids is at most negligible and there are only a polynomial number of hybrids between $\text{Exp}_3^{\text{RUF}}$ and $\text{Exp}_4^{\text{RUF}}$, we can conclude that the output distribution in $\text{Exp}_4^{\text{RUF}}$ and $\text{Exp}_3^{\text{RUF}}$ is computationally indistinguishable, i.e.,

$$|\Pr[\text{Exp}_4^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_3^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Assume encryption keys to be useless. In the next experiment, we get rid of the differentiation between $K = kG_0$ and $\text{pk}_c^{\text{ld}} = kG_1$ in the redemption oracle. We assume that $K = kG_0$ if the extractor for k is successful. The experiment is defined as follows:

```

Experiment:  $\text{Exp}_5^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Rd}}(\text{sid}, m, i)$ :
14: ...
68: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
69:  $(i, \text{sk}_c^{\text{ld}}, \text{pk}_c^{\text{ld}}) \leftarrow \text{st}_{\text{sid}}, (K, \pi_{\text{Ch}}) \leftarrow m, \text{st}_{\text{sid}} \leftarrow \perp$ 
70: if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{ld}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
71:  $k \leftarrow \mathcal{E}_{\text{Ch}}^{A(\cdot)}(\text{trans})$ , if  $k = \perp$  then return  $\perp$ 
72:  $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma_i + \sum_{j \in [L]} x_j H_j$ 
73:  $\forall j \in [L] : r_j, \bar{r}_j \xleftarrow{\$} \mathbb{Z}_p, A_j \leftarrow r_j G_1, B_j \leftarrow \bar{r}_j G_1$ 
74:  $\pi_{\text{Rd}}^{\text{sid}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_c^{\text{ld}}, K, \sigma_i, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
75: return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
76: else return  $\emptyset$ 

```

It is obvious that $\text{Exp}_5^{\text{RUF}}$ and $\text{Exp}_4^{\text{RUF}}$ only differ if $K \neq kG_0$ resp. if $\text{pk}_c^{\text{ld}} = kG_1$. We call this event F . We can show that event F cannot happen except with negligible probability via a reduction to the discrete logarithm problem. Note that, in case of event F , $k = \text{sk}_c^{\text{ld}}$ is the discrete logarithm of pk_c^{ld} to the basis of G_1 and that sk_c^{ld} is generated by the experiment but not used at all (redemption

4. A direct reduction to the ElGamal cryptosystem is cumbersome as the second ciphertext component B needs to be multiplied by k to transform the ciphertext component $B = r\text{pk} + M$ received from the security game into the component $B' = rkG_0 + M'$ ($\text{pk} = G_0$) compatible with our security game. This obviously tampers with the message M .

zero-knowledge proofs are simulated). The only information about sk^{ld} available to the adversary is pk_c^{ld} . This allows us to use an adversary from which we can extract k such that $\text{pk}_c^{\text{ld}} = kG_1$ to solve the discrete logarithm problem in $\mathbb{G}_1$. As we assume the discrete logarithm assumption to be hard in $\mathbb{G}_1$, we can apply the difference lemma to conclude that

$$|\Pr[\text{Exp}_5^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_4^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Simulate requests proofs. Next, we tackle the request oracle and create the request proofs returned by the oracle with simulator $\mathcal{S}_{\text{Rq}}$ instead of the proof system's witnesses. The simulator receives the proof's statement $(\text{pp}, \text{pk}_c^{\text{ld}}, T')$ and has the capability of programming the random oracle. It creates a valid zero knowledge proof π_{Rq} , except with negligible probability. The resulting experiment is defined as follows:

```

Experiment:  $\text{Exp}_6^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Qu}}(i, \text{pk})$ :
6: if  $T_i \neq \text{null}$  then return  $\perp$ 
7: Select  $x_1, \dots, x_L \in [0, 2^\ell - 1]$  such that  $\text{sk}^{\text{ld}} = \sum_{j \in [L]} x_j \cdot 2^{(j-1)\ell}$  holds without modulo reduction (This representation is unique and independent of  $i$ )
8:  $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p, C_i = G_0 + r_i H_0 + \sum_{j \in [L]} x_j H_j, \text{pk}_i \leftarrow \text{pk}$ 
9:  $T_i \leftarrow \delta_i C_i, \pi_i \leftarrow \mathcal{S}_{\text{Rq}}(\text{pp}, \text{pk}_c^{\text{ld}}, T_i)$ 
10: return  $(T_i, \pi_i)$ 
...

```

In $\text{Exp}_5^{\text{RUF}}$, the requests proofs are always valid due to the completeness property of the proof system. Therefore, it follows that the experiments $\text{Exp}_6^{\text{RUF}}$ and $\text{Exp}_5^{\text{RUF}}$ only differ if the simulator is not successful, which happens with at most negligible probability due to the zero-knowledge property of the request proof system. We use the difference lemma to conclude that

$$|\Pr[\text{Exp}_6^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_5^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Simplification. In a bridging step, we simplify the request oracle by sampling $C_i \xleftarrow{\$} \mathbb{G}_1$ instead of computing $C_i \leftarrow G_0 + r_i H_0 + \sum_{j \in [L]} x_j H_j$. As the uniformly randomly sampled mask r_i acts as a one-time pad and is not used further, C_i is distributed uniformly random in both experiments. The game is defined as follows:

```

Experiment:  $\text{Exp}_7^{\text{RUF}}(\lambda)$ 
...
 $\mathcal{O}^{\text{Qu}}(i, \text{pk}_s)$ :
6: if  $T_i \neq \text{null}$  then return  $\perp$ 
7:  $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p, C_i \xleftarrow{\$} \mathbb{G}_1, \text{pk}_i \leftarrow \text{pk}, T_i \leftarrow \delta_i C_i, \pi_i \leftarrow \mathcal{S}_{\text{Rq}}(\text{pp}, \text{pk}_c^{\text{ld}}, T_i)$ 
8: return  $(T_i, \pi_i)$ 
...

```

As the simplification does not affect the output distribution, we conclude that

$$\Pr[\text{Exp}_7^{\text{RUF}}(\lambda) = 1] = \Pr[\text{Exp}_6^{\text{RUF}}(\lambda) = 1].$$

Extract request proof. We adapt the main thread to extract the witness of the request proof π^* after successfully verifying the proof. To do so, we employ extractor $\mathcal{E}_{\text{Rq}}$ of the request proof system. The extractor has access to the adversary $\mathcal{A}$, receives the transcript trans of the experiment execution up to the extraction including the adversary's random tape, and has the power to program the random oracle. It extracts the witnesses $(\{x_j\}_{j \in [L]}, r, \delta)$ that are either all $\perp$ or satisfy the proof system's relation PK_{Rq} . The resulting game is defined as follows:

Experiment: $\text{Exp}_8^{\text{RUF}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ ,  $(\text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 
2:  $((T^*, \pi^*), \text{sk}_s) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, \text{pk}_c^{\text{In}}, \text{pk}_c^{\text{Id}})$ 
3: if  $\exists i$  s.t.  $(T^*, \pi^*) = (T_i, \pi_i)$  then return 0
4: if  $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}_c^{\text{In}}, T^*), \pi^*) = 0$  then return 0
5:  $(\{x_j\}_{j \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}(\cdot)}(\text{trans})$ , if  $r = \perp$  then return 0
6: return 1
...

```

It is obvious that $\text{Exp}_8^{\text{RUF}}$ and $\text{Exp}_7^{\text{RUF}}$ only differ if the extractor is not successful. Note that, we ensure that the request proof has not been issued by the request oracle. Therefore, it needs to originate from the adversary such that the proofs issued by the query oracle cannot tamper with the extraction. As the extractor is successful expect with negligible probability, due to the knowledge soundness property, we can apply the difference lemma to conclude that

$$|\Pr[\text{Exp}_8^{\text{RUF}}(\lambda) = 1] - \Pr[\text{Exp}_7^{\text{RUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Reduction to the discrete logarithm assumption. If the adversary $\mathcal{A}$ wins $\text{Exp}_8^{\text{RUF}}$, then the game extracted $\{x_j\}_{j \in [L]}$ from $\mathcal{A}$ such that $\text{pk}_c^{\text{In}} = (\sum_{j \in [L]} 2^{(j-1) \cdot \ell} x_j) G_1$. Therefore, we can use $\mathcal{A}$ to compute the discrete logarithm of pk_c^{In} to the basis of G_1 which contradicts the discrete logarithm assumption in $\mathbb{G}_1$. Formally, we define an adversary $\mathcal{B}$, that receives a discrete logarithm challenge $(G, H) \in \mathbb{G}_1^2$, internally executes $\mathcal{A}$, and returns $x = \text{dlog}_G(H)$ with probability $\Pr[\text{Exp}_8^{\text{RUF}}(\lambda) = 1]$. $\mathcal{B}$ is defined as follows:

- Execute $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and send $(\mathbb{G}_1, p)$ to the discrete logarithm challenger to receive (G, H) .
- Overwrite $G_1 \leftarrow G$ and sample $\text{pk}_c^{\text{Id}} \xleftarrow{\$} \mathbb{G}_1$.
- Execute $((T^*, \pi^*), \cdot) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(\text{pp}, H, \text{pk}_c^{\text{Id}})$.
– Answer oracle queries as defined by experiment $\text{Exp}_8^{\text{RUF}}$.
- If $\exists i$ s.t. $(T^*, \pi^*) = (T_i, \pi_i)$ or $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}_c^{\text{In}}, T^*), \pi^*) = 0$ then return $\perp$.
- Otherwise, employ the request proof extractor to compute $(\{x_j\}_{j \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}(\cdot)}(\text{trans})$.
- If $r = \perp$ then return $\perp$.
- Otherwise, return $x = \sum_{j \in [L]} 2^{(j-1) \cdot \ell} x_j$.

First note that the view of $\mathcal{A}$ in the experiment simulated by $\mathcal{B}$ is identical to the view of $\mathcal{A}$ in experiment $\text{Exp}_8^{\text{RUF}}$. The public parameters, the insurance public key, and the oracle responses are distributed identically in both experiments. By definition of the extractor $\mathcal{E}_{\text{Rq}}$ it needs to hold that either $r = \perp$ (extraction fails) or $\text{pk}_c^{\text{In}} = (\sum_{j \in [L]} 2^{(j-1) \cdot \ell} x_j) G_1$.

Further, if $\mathcal{A}$ wins $\text{Exp}_8^{\text{RUF}}$ it needs to hold that $r \neq \perp$ and, therefore, that $x = \sum_{j \in [L]} 2^{(j-1) \cdot \ell} x_j$ is the discrete logarithm of pk_c^{In} . It follows that $\mathcal{B}$ wins the discrete logarithm experiment if $\mathcal{A}$ would win $\text{Exp}_8^{\text{RUF}}$. As we assume that no PPT adversary wins the discrete logarithm experiment in $\mathbb{G}_1$ expect with negligible probability, it follows that

$$\Pr[\text{Exp}_8^{\text{RUF}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Putting all steps together. As we use a polynomial number of game hops (in the number of oracle queries and the chunk size L), the difference in the success probability of the adversary in any two adjacent experiments is at most negligible, and the adversary can win in the final game $\text{Exp}_{10}^{\text{RUF}}$ with at most negligible probability, we can conclude that

$$\Pr[\text{Exp}_A^{\text{RUF}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

F.3. Identity Unforgeability

We prove identity unforgeability via a sequence of games. Let $\text{Exp}_0^{\text{IUF}} = \text{Exp}_A^{\text{IUF}}$ be the initial security game. The subsequent games are defined as follows.

Inline construction. In the first game, we inline our concrete construction into the security game. The game is defined as follows:

Experiment: $\text{Exp}_1^{\text{IUF}}(\lambda)$

```

1:  $\text{res} \leftarrow 0$ ,  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ ,  $(\cdot, \cdot, \text{sk}_c^{\text{Id}}, \text{pk}_c^{\text{Id}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 
2:  $(\text{pk}_s, \text{st}^A) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(1, \text{pp}, \text{pk}_c^{\text{Id}}, \mathcal{A}^{\mathcal{O}^*}(2, \text{st}^A))$ 
3: return res

```

$\mathcal{O}^{\text{Qu}}(i, \text{pk}_c^{\text{In}}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{Id}})$:

```

4: if  $T_i \neq \text{null} \vee \text{KeyV}(\text{pp}, \text{sk}_c^{\text{In}}, \text{pk}_c^{\text{In}}) = 0$  then return  $\perp$ 
5: Select  $x_1^i, \dots, x_L^i \in [0, 2^\ell - 1]$  such that  $\text{sk}_c^{\text{In}} = \sum_{j \in [L]} x_j^i \cdot 2^{(j-1)\ell}$  holds without modulo reduction (This representation is unique and independent of  $i$ ).
6:  $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p$ ,  $C_i = G_0 + r_i H_0 + \sum_{j \in [L]} x_j^i H_j$ ,  $\text{pk}_i \leftarrow \text{pk}$ 
7:  $T_i \leftarrow \delta_i C_i$ ,  $\pi_i \leftarrow \text{PK}_{\text{Rq}}.\text{Prove}(\text{rpp}, (\text{pk}_c^{\text{In}}, T_i), (\{x_j^i\}_{j \in [L]}, r_i, \delta_i))$ 
8: return  $(T_i, \pi_i)$ 

```

$\mathcal{O}^{\text{Fin}}(i, \text{resp} = (e, S))$:

```

9: if  $T_i = \text{null} \vee \sigma_i \neq \text{null}$  then return  $\perp$ 
10: if  $e(S, \text{pk}_i + eG_2) \neq e(T_i, G_2)$  then  $\sigma_i \leftarrow \perp$ , return  $\sigma_i$ 
11: else  $\sigma_i \leftarrow \frac{1}{\delta_i} S$ ,  $e_i \leftarrow e$ , return  $\sigma_i$ 

```

$\mathcal{O}^{\text{Rd}}(\text{sid}, m)$:

```

12: if  $\text{st}_{\text{sid}} = \text{null}$  then
13:    $i \leftarrow m$ , if  $\sigma_i \in \{\text{null}, \perp\}$  then return  $\perp$ 
14:    $\text{st}_{\text{sid}} \leftarrow i$ , return  $(\text{pk}_c^{\text{Id}}, \sigma_i)$ 
15: elseif  $\text{st}_{\text{sid}} \neq \perp$  then
16:    $i \leftarrow \text{st}_{\text{sid}}$ ,  $(K, \pi_{\text{Ch}}) \leftarrow m$ ,  $\text{st}_{\text{sid}} \leftarrow \perp$ 
17:   if  $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_c^{\text{Id}}), \pi_{\text{Ch}}) = 0 \vee K = 0$  return  $\perp$ 
18:    $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma + \sum_{j \in [L]} x_j^i H_j$ 

```

```

19:  $\forall j \in [L] : r_j \xleftarrow{\$} \mathbb{Z}_p$ ,  $A_j \leftarrow r_j G_1$ ,  $B_j \leftarrow r_j K + x_j^i G_1$ 
20:  $\pi_{\text{Rd}} \leftarrow \text{PK}_{\text{Rd}}.\text{Prove}((\text{pk}_c^{\text{Id}}, K, \sigma_i, \sigma', \{A_j, B_j\}_{j \in [L]}), (r_i, e_i, \text{sk}_c^{\text{Id}}, \{r_j, x_j^i\}_{j \in [L]}))$ 
21: return  $(\pi_{\text{Rd}}, \sigma', \{A_j, B_j\}_{j \in [L]})$ 
22: else return  $\emptyset$ 

```

$\mathcal{O}^*(m)$:

```

23: if st ∈ {0, 1} then return ⊥
24: if st = null then
25:   (σ*, ·) ← m, k* ←  $\mathbb{Z}_p$ , K* ← k*G0, st ← (σ*, pks, K*)
26:   πCh ← PKCh.Prove((K*, pkcld), k*)
27:   return (K*, πCh)
28: else
29:   (π*, σ*, {Ai*, Bi*}i∈[L]) ← m
30:   if e(σ*, pks) ≠ e(σs*, G2) then stsid ← 0, return 0
31:   if PKRd.Verify((pkcld, K*, σ*, σs*, {Ai*, Bi*}i∈[L]), π*)
     = 0 then st ← 0, return 0
32:   res, st) ← (1, 1), return 1

```

This step does not affect the output distribution. It follows that

$$\Pr[\text{Exp}_1^{\text{IUF}}(\lambda) = 1] = \Pr[\text{Exp}_0^{\text{IUF}}(\lambda) = 1].$$

Simulate redemption proofs. Analogously to $\text{Exp}_2^{\text{RUF}}$, we use the redemption proof's simulator to create redemption proofs π_{Rd} within the redemption oracle. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_2^{\text{IUF}}(\lambda)$

```

...
 $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$ :
12: ...
15: elseif stsid ≠ ⊥ then
16:   i ← stsid, (K, πCh) ← m, stsid ← ⊥
17:   if PKCh.Verify((K, pkcld), πCh) = 0 then return ⊥
18:   σ' ← G0 + riH0 - eiσ +  $\sum_{j \in [L]} x_j^i H_j$ 
19:    $\forall j \in [L] : r_j \leftarrow \mathbb{Z}_p$ , Aj ← rjG1, Bj ← rjK + xjiG1
20:   πRd ←  $\mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_c^{\text{ld}}, K, \sigma_i, \sigma', \{A_j^{\text{sid}}, B_j^{\text{sid}}\}_{j \in [L]})$ 
21:   return (πRd, σ', {Aj, Bj}j∈[L])
22: else return ∅
...

```

Analogously to $\text{Exp}_2^{\text{RUF}}$ it can be shown that

$$|\Pr[\text{Exp}_2^{\text{IUF}}(\lambda) = 1] - \Pr[\text{Exp}_1^{\text{IUF}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

based on the zero-knowledge property of the redemption proof system.

Extraction of the redemption proof. We adapt $\mathcal{O}^*$ to extract the witnesses of the redemption proof analogously to $\text{Exp}_3^{\text{OMUF}}$. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_3^{\text{IUF}}(\lambda)$

```

...
 $\mathcal{O}^*(m)$ :
23: ...
28: else
29:   (π*, σ*, {Ai*, Bi*}i∈[L]) ← m
30:   if e(σ*, pks) ≠ e(σs*, G2) then stsid ← 0, return 0
31:   if PKRd.Verify((pkcld, K*, σ*, σs*, {Ai*, Bi*}i∈[L]), π*) = 0
     then st ← 0, return 0
32:   (sk*, r*, e*, {ri*, xi*}i∈[L]) ←  $\mathcal{E}_{\text{Rd}}^{\mathcal{A}(\cdot)}(\text{trans})$ 
33:   if e* = ⊥ then stsid ← 0, return 0
34:   (res, st) ← (1, 1), return 1

```

It is obvious that $\text{Exp}_3^{\text{IUF}}$ and $\text{Exp}_2^{\text{IUF}}$ only differ if the extractor is not successful. As the challenge K^* ensures that the zero-knowledge proof is fresh and cannot be one that has been sent from the oracle to the adversary, we conclude from the knowledge soundness property of the redemption

proof system that the extractor fails only with negligible probability. Therefore, we can apply the difference lemma to conclude that

$$|\Pr[\text{Exp}_3^{\text{IUF}}(\lambda) = 1] - \Pr[\text{Exp}_2^{\text{IUF}}(\lambda) = 1]| \leq \text{negl}(\lambda).$$

Reduction to the discrete logarithm assumption. In order to win the experiment, the adversary $\mathcal{A}$ has to provide a zero-knowledge proof of knowledge of secret key $\text{sk}^* = \text{dlog}_{G_1}(\text{pk}_c^{\text{ld}})$ to the oracle $\mathcal{O}^*$. However, it does not receive any information about sk^* beyond pk_c^{ld} . This is due to the fact that the redemption proofs in oracle $\mathcal{O}^{\text{Rd}}$ are simulated. Hence, we can use $\mathcal{A}$ to compute the discrete logarithm of pk_c^{ld} to the basis of G_1 which contradicts the discrete logarithm assumption in $\mathbb{G}_1$. Formally, we define an adversary $\mathcal{B}$, that receives a discrete logarithm challenge $(G, H) \in \mathbb{G}_1^2$, internally executes $\mathcal{A}$, and returns $x = \text{dlog}_G(H)$ with probability $\Pr[\text{Exp}_3^{\text{IUF}}(\lambda) = 1]$. $\mathcal{B}$ is defined as follows:

- Execute $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and send $(\mathbb{G}_1, p)$ to the discrete logarithm challenger to receive (G, H) .
- Overwrite $G_1 \leftarrow G$.
- Execute $(\text{pk}_s, \text{st}^{\mathcal{A}}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{Qu}}, \mathcal{O}^{\text{Fin}}, \mathcal{O}^{\text{Rd}}}(1, \text{pp}, H)$
 - Answer oracle queries as defined by experiment $\text{Exp}_3^{\text{IUF}}$.
- Execute $\mathcal{A}^{\mathcal{O}^*}(2, \text{st}^{\mathcal{A}})$.
 - Answer oracle queries as defined by experiment $\text{Exp}_3^{\text{IUF}}$.
- If sk^* is not defined or equals $\perp$ then return $\perp$.
- Otherwise return sk^* .

First note that the view of $\mathcal{A}$ in the experiment simulated by $\mathcal{B}$ is identical to the view of $\mathcal{A}$ in experiment $\text{Exp}_3^{\text{IUF}}$. The public parameters, the identification public key, and the oracle responses are distributed identically in both experiments. Further, note that $\mathcal{A}$ can only win in $\text{Exp}_3^{\text{IUF}}$ if extractor $\mathcal{E}_{\text{Rd}}$ successfully extracts a witness $(\text{sk}^*, \dots)$ from the redemption proof such that $H = \text{sk}^*G_1$. We conclude that $\mathcal{B}$ wins the discrete logarithm experiment exactly if $\mathcal{A}$ would win $\text{Exp}_3^{\text{IUF}}$. As no PPT adversary can win the discrete logarithm experiment except with negligible probability by assumption, it follows that

$$\Pr[\text{Exp}_3^{\text{IUF}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Putting all steps together. As we use a polynomial number of game hops (in the number of oracle queries), the difference in the success probability of the adversary in any two adjacent experiments is at most negligible, and the adversary can win in the final game $\text{Exp}_3^{\text{IUF}}$ with at most negligible probability, we can conclude that

$$\Pr[\text{Exp}_A^{\text{IUF}}(\lambda) = 1] \leq \text{negl}(\lambda).$$

Appendix G. Proof of Theorem 5

This section provides a detailed proof of Theorem 5. To show unlinkability, we have to show that for every PPT adversaries $\mathcal{A}$ it holds that

$$|\Pr[\text{Exp}_A^{\text{UL}}(\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda).$$

We do so via a sequence of games. Let $\mathcal{A}$ be an arbitrary PPT adversaries and $\text{Exp}_0^{\text{UL}} = \text{Exp}_{\mathcal{A}}^{\text{UL}}$ be the initial security game. The subsequent games are defined as follows.

Inline construction. In the first game, we inline our construction into the security game. The game is defined as follows:

Experiment: $\text{Exp}_1^{\text{UL}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), j' \leftarrow 0, b \xleftarrow{\$} \{0, 1\}, \Gamma \leftarrow \emptyset$ 2: $(\text{pk}_s, \text{st}_0^A) \leftarrow \mathcal{A}^{\text{KG}}(0, \text{pp})$ 3: $(\alpha, \beta, \text{st}_1^A) \leftarrow \mathcal{A}^{\text{KG}, \text{ORq}, \text{OFin}, \text{ORd}}(1, \text{st}_0^A)$ 4: if $\exists k \in \{\alpha, \beta\}$ s.t. $\sigma_k \in \{\text{null}, \perp\} \vee k \in \Gamma$ then abort 5: if $b = 1$ then swap values of $(\sigma_\alpha, \omega_\alpha)$ and $(\sigma_\beta, \omega_\beta)$ 6: $b' \leftarrow \mathcal{A}^{\text{KG}, \text{ORq}, \text{OFin}, \text{ORd}}(2, \text{st}_1^A)$ 7: if $b = b'$ then return 1 else return 0 <u>$\mathcal{O}^{\text{KG}}()$:</u> 8: $j' \leftarrow j' + 1, (\text{sk}_j^{\text{ld}}, \text{pk}_j^{\text{ld}}, \text{sk}_j^{\text{in}}, \text{pk}_j^{\text{in}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 9: return $(\text{pk}_j^{\text{ld}}, \text{sk}_j^{\text{in}}, \text{pk}_j^{\text{in}})$ <u>$\mathcal{O}^{\text{Rq}}(i, j)$:</u> 10: if $T_i \neq \text{null} \vee j > j'$ then abort 11: Split bit representation of sk_j^{in} into $x_1^i, \dots, x_L^i \in [0, 2^\ell - 1]$ such that $\text{sk}_j^{\text{in}} = \sum_{\varphi \in [L]} 2^{(\varphi-1)\ell} x_\varphi$ without modulo reduction. 12: $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p, C_i = G_0 + r_i H_0 + \sum_{\varphi \in [L]} x_\varphi H_\varphi, T_i \leftarrow \delta_i C_i$ 13: $\pi_i \leftarrow \text{PK}_{\text{Rq}}.\text{Prove}(\text{rpp}, (\text{pk}_j^{\text{in}}, T_i), \{x_\varphi\}_{\varphi \in [L]}, r_i, \delta_i)$ 14: return (T_i, π_i) <u>$\mathcal{O}^{\text{Fin}}(i, \text{resp} = (e, S))$:</u> 15: if $T_i = \text{null} \vee \sigma_i \neq \text{null}$ then return $\perp$ 16: if $e(S, \text{pk}_s + eG_2) \neq e(T_i, G_2)$ then $\sigma_i \leftarrow \perp$, return σ_i 17: else $\sigma_i \leftarrow \frac{1}{\delta_i} S, e_i \leftarrow e$, return 1 <u>$\mathcal{O}^{\text{Rd}}(\text{sid}, m)$:</u> 18: if $\text{st}_{\text{sid}} = \text{null}$ then 19: $(i, j) \leftarrow m$, if $\sigma_i \in \{\text{null}, \perp\} \vee \text{pk}_j^{\text{ld}} = \text{null}$ then return $\perp$ 20: $\Gamma \leftarrow \Gamma \cup \{i\}, \text{st}_{\text{sid}} \leftarrow (i, j)$, return $(\text{pk}_j^{\text{ld}}, \sigma_i)$ 21: elseif $\text{st}_{\text{sid}} \neq \perp$ then 22: $(i, j) \leftarrow \text{st}_{\text{sid}}, (K, \pi_{\text{Ch}}) \leftarrow m, \text{st}_{\text{sid}} \leftarrow \perp$ 23: if $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_j^{\text{ld}}), \pi_{\text{Ch}}) = 0 \vee K = 0$ return $\perp$ 24: $\sigma' \leftarrow G_0 + r_i H_0 - e_i \sigma + \sum_{\phi \in [L]} x_\phi^i H_i$ 25: $\forall \varphi \in [L] : r_\varphi \xleftarrow{\$} \mathbb{Z}_p, A_\varphi \leftarrow r_\varphi G_1, B_\varphi \leftarrow r_\varphi K + x_\varphi^i G_1$ 26: $\pi_{\text{Rd}} \leftarrow \text{PK}_{\text{Rd}}.\text{Prove}((\text{pk}_j^{\text{ld}}, K, \sigma_i, \{A_\varphi, B_\varphi\}_{\varphi \in [L]}), (\text{sk}_j^{\text{ld}}, r_i, e_i, \{r_\varphi, x_\varphi^i\}_{\varphi \in [L]}))$ 27: return $(\pi_{\text{Rd}}, \sigma', \{A_\varphi, B_\varphi\}_{\varphi \in [L]})$ 28: else return $\emptyset$

This step does not change the output distribution. It follows that

$$\Pr[\text{Exp}_1^{\text{UL}}(\lambda) = 1] = \Pr[\text{Exp}_0^{\text{UL}}(\lambda) = 1].$$

Simplify the redemption oracle. Analogously to $\text{Exp}_2^{\text{RUF}}, \text{Exp}_3^{\text{RUF}}, \text{Exp}_4^{\text{RUF}}$ and $\text{Exp}_5^{\text{RUF}}$, we define games $\text{Exp}_2^{\text{UL}}, \text{Exp}_3^{\text{UL}}, \text{Exp}_4^{\text{UL}}$, and Exp_5^{UL} that simplify the redemption oracle as follows:

- Exp_2^{UL} creates the redemption proofs using the proof system's simulator $\mathcal{S}_{\text{Rd}}$ instead of creating them honestly.
- Exp_3^{UL} employs an extractor for the challenge proof to extract k such that $K = kG_1$ or $\text{pk}_j^{\text{ld}} = kG_1$ after π_{Ch} has been verified successfully.

- Exp_4^{UL} randomly samples the second ciphertext component if $K = kG_1$. If $\text{pk}_j^{\text{ld}} = kG_1$, the ciphertexts are still created correctly.
- Exp_5^{UL} removes the $\text{pk}_j^{\text{ld}} = kG_1$ case, i.e., it always creates ciphertexts on randomly sampled dummy values.

Exp_5^{UL} is defined as follows:

Experiment: $\text{Exp}_5^{\text{UL}}(\lambda)$
... <u>$\mathcal{O}^{\text{Rd}}(\text{sid}, m)$:</u> 18: ... 21: elseif $\text{st}_{\text{sid}} \neq \perp$ then 22: $(i, j) \leftarrow \text{st}_{\text{sid}}, (K, \pi_{\text{Ch}}) \leftarrow m, \text{st}_{\text{sid}} \leftarrow \perp$ 23: if $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_j^{\text{ld}}), \pi_{\text{Ch}}) = 0 \vee K = 0$ return $\perp$ 24: $k \leftarrow \mathcal{E}_{\text{Ch}}^{\mathcal{A}(\cdot)}, \text{if } k = \perp$ then return $\perp$. 25: $\forall \varphi \in [L] : r_\varphi, \bar{r}_\varphi \xleftarrow{\$} \mathbb{Z}_p, A_\varphi \leftarrow r_\varphi G_1, B_\varphi \leftarrow \bar{r}_\varphi G_1$ 26: $\pi_{\text{Rd}}^{\text{sid}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_j^{\text{ld}}, K, \sigma_i, \{A_\varphi^{\text{sid}}, B_\varphi^{\text{sid}}\}_{\varphi \in [L]})$ 27: return $(\pi_{\text{Rd}}, \{A_\varphi, B_\varphi\}_{\varphi \in [L]})$ 28: else return $\emptyset$

Analogously to $\text{Exp}_2^{\text{RUF}}, \text{Exp}_3^{\text{RUF}}, \text{Exp}_4^{\text{RUF}}$, and $\text{Exp}_5^{\text{RUF}}$, it can be shown that

$$\begin{aligned} |\Pr[\text{Exp}_2^{\text{UL}}(\lambda) = 1] - \Pr[\text{Exp}_1^{\text{UL}}(\lambda) = 1]| &\leq \text{negl}(\lambda) \\ |\Pr[\text{Exp}_3^{\text{UL}}(\lambda) = 1] - \Pr[\text{Exp}_2^{\text{UL}}(\lambda) = 1]| &\leq \text{negl}(\lambda) \\ |\Pr[\text{Exp}_4^{\text{UL}}(\lambda) = 1] - \Pr[\text{Exp}_3^{\text{UL}}(\lambda) = 1]| &\leq \text{negl}(\lambda) \\ |\Pr[\text{Exp}_5^{\text{UL}}(\lambda) = 1] - \Pr[\text{Exp}_4^{\text{UL}}(\lambda) = 1]| &\leq \text{negl}(\lambda). \end{aligned}$$

Simplify the request oracle. Analogously to $\text{Exp}_6^{\text{RUF}}$, we define game Exp_6^{UL} that simplifies the request oracle by using the proof system's simulator $\mathcal{S}_{\text{Rq}}$ to create request proofs instead of creating them honestly. Exp_6^{UL} is defined as follows:

Experiment: $\text{Exp}_6^{\text{UL}}(\lambda)$
... <u>$\mathcal{O}^{\text{Qu}}(i, j)$:</u> 8: if $T_i \neq \text{null} \vee j > j'$ then abort 9: Split bit representation of sk_j^{in} into $x_1^i, \dots, x_L^i \in [0, 2^\ell - 1]$ such that $\text{sk}_j^{\text{in}} = \sum_{\varphi \in [L]} 2^{(\varphi-1)\ell} x_\varphi$ without modulo reduction. 10: $r_i, \delta_i \xleftarrow{\$} \mathbb{Z}_p, C_i = G_0 + r_i H_0 + \sum_{\varphi \in [L]} x_\varphi^i H_\varphi, \text{pk}_i \leftarrow \text{pk}$ 11: $T_i \leftarrow \delta_i C_i, \pi_i \leftarrow \mathcal{S}_{\text{Rq}}(\text{pp}, \text{pk}_j^{\text{in}}, T_i)$ 12: return (T_i, π_i) ...

Analogously to $\text{Exp}_6^{\text{RUF}}$, it can be shown that

$$|\Pr[\text{Exp}_6^{\text{UL}}(\lambda) = 1] - \Pr[\text{Exp}_5^{\text{UL}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

based on the zero-knowledge property of the request proof system.

Simplification. In a bridging step, we simplify the experiment without changing the output distribution.

- We compute $C_i \xleftarrow{\$} \mathbb{G}_1$ instead of $C_i \leftarrow G_0 + r_i H_0 + \sum_{\varphi \in [L]} x_\varphi^i H_\varphi$. As the uniformly random sampled mask r_i acts as a one-time pad and is not used further, C_i is distributed uniformly random in both

experiments. This means that the change in the computation of C_i does not alter the output distribution.

- We remove the assignment of the token witness, i.e., the storage of variables $r_i, e_i, \{x_\varphi^i\}_{\varphi \in [L]}$ from the experiments. As these variables are used at most once (r_i and x_φ^i are not used at all), they can be discarded after being used without affecting the output distribution.

Exp_7^{UL} is defined as follows:

Experiment: $\text{Exp}_7^{\text{UL}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), j' \leftarrow 0, b \xleftarrow{\$} \{0, 1\}, \Gamma \leftarrow \emptyset$ 2: $(\text{pk}_s, \text{st}_0^A) \leftarrow \mathcal{A}^{\text{OKG}}(0, \text{pp})$ 3: $(\alpha, \beta, \text{st}_1^A) \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(1, \text{st}_0^A)$ 4: if $\exists k \in \{\alpha, \beta\}$ s.t. $\sigma_k \in \{\text{null}, \perp\} \vee k \in \Gamma$ then abort 5: if $b = 1$ then swap values of (σ_α) and (σ_β) 6: $b' \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(2, \text{st}_1^A)$ 7: if $b = b'$ then return 1 else return 0 $\mathcal{O}^{\text{KG}}()$: 8: $j' \leftarrow j' + 1, (\text{sk}_{j'}^{\text{ld}}, \text{pk}_{j'}^{\text{ld}}, \text{sk}_{j'}^{\text{ln}}, \text{pk}_{j'}^{\text{ln}}) \leftarrow \text{C.KeyGen}(\text{pp})$ 9: return $(\text{pk}_{j'}^{\text{ld}}, \text{sk}_{j'}^{\text{ln}}, \text{pk}_{j'}^{\text{ln}})$ $\mathcal{O}^{\text{Rq}}(i, j)$: 10: if $T_i \neq \text{null} \vee j > j'$ then abort 11: $\delta_i \xleftarrow{\$} \mathbb{Z}_p, C_i \xleftarrow{\$} \mathbb{G}_1, T_i \leftarrow \delta_i C_i,$ $\pi_i \leftarrow \mathcal{S}_{\text{Rq}}(\text{pp}, \text{pk}_C^{\text{ln}}, T_i)$, return (T_i, π_i) $\mathcal{O}^{\text{Fin}}(i, (e, S))$: 12: if $T_i = \text{null} \vee \sigma_i \neq \text{null}$ then return $\perp$ 13: if $e(S, \text{pk}_s + eG_2) \neq e(T_i, G_2)$ then $\sigma_i \leftarrow \perp$, return σ_i 14: else $\sigma_i \leftarrow \frac{1}{\delta_i} S, e_i \leftarrow e$, return 1 $\mathcal{O}^{\text{Rd}}(\text{sid}, m)$: 15: if $\text{st}_{\text{sid}} = \text{null}$ then 16: $(i, j) \leftarrow m$, if $\sigma_i \in \{\text{null}, \perp\} \vee \text{pk}_j^{\text{ld}} = \text{null}$ then return $\perp$ 17: $\Gamma \leftarrow \Gamma \cup \{i\}, \text{st}_{\text{sid}} \leftarrow (i, j)$, return $(\text{pk}_j^{\text{ld}}, \sigma_i)$ 18: elseif $\text{st}_{\text{sid}} \neq \perp$ then 19: $(i, j) \leftarrow \text{st}_{\text{sid}}, (K, \pi_{\text{Ch}}) \leftarrow m, \text{st}_{\text{sid}} \leftarrow \perp$ 20: if $\text{PK}_{\text{Ch}}.\text{Verify}((K, \text{pk}_j^{\text{ld}}), \pi_{\text{Ch}}) = 0$ then return $\perp$ 21: $k \leftarrow \mathcal{E}_{\text{Ch}}^A(\cdot)$, if $k = \perp$ then return $\perp$. 22: $\forall \varphi \in [L] : r_\varphi, \bar{x}_\varphi \xleftarrow{\$} \mathbb{Z}_p, A_\varphi \leftarrow r_\varphi G_1, B_\varphi \leftarrow r_\varphi K + \bar{x}_\varphi G_1$ 23: $\pi_{\text{Rd}}^{\text{sid}} \leftarrow \mathcal{S}_{\text{Rd}}(\text{pp}, \text{pk}_j^{\text{ld}}, K, \sigma_i, \{A_\varphi, B_\varphi\}_{\varphi \in [L]})$ 24: return $(\pi_{\text{Rd}}, \{A_\varphi, B_\varphi\}_{\varphi \in [L]})$ 25: else return $\emptyset$

As the modifications do not affect the output distribution, we conclude that

$$\Pr[\text{Exp}_7^{\text{UL}}(\lambda) = 1] = \Pr[\text{Exp}_6^{\text{UL}}(\lambda) = 1].$$

Randomize selected tokens. In the next experiment, we overwrite the selected tokens σ_α and σ_β with uniformly random sampled elements in $\mathbb{G}_1$, i.e., $\sigma_\alpha, \sigma_\beta \xleftarrow{\$} \mathbb{G}_1$. The game is defined as follows:

Experiment: $\text{Exp}_8^{\text{UL}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), j' \leftarrow 0, b \xleftarrow{\$} \{0, 1\}, \Gamma \leftarrow \emptyset$ 2: $(\text{pk}_s, \text{st}_0^A) \leftarrow \mathcal{A}^{\text{OKG}}(0, \text{pp})$ 3: $(\alpha, \beta, \text{st}_1^A) \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(1, \text{st}_0^A)$ 4: if $\exists k \in \{\alpha, \beta\}$ s.t. $\sigma_k \in \{\text{null}, \perp\} \vee k \in \Gamma$ then abort 5: if $b = 1$ then swap values of (σ_α) and (σ_β)

6: $\sigma_\alpha, \sigma_\beta \xleftarrow{\$} \mathbb{G}_1, b' \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(2, \text{st}_1^A)$
7: **if** $b = b'$ **then return 1 else return 0**
...

First, note that the request proofs (π_{Rq}) and redemption proofs (π_{Rd}) are simulated without using the witnesses and that the ciphertexts $\{A_\phi, B_\phi\}_{\phi \in [L]}$ are sampled uniformly random. These messages provide no information to the adversary. What remains are the blinded token requests T_i and tokens σ_i . In Exp_7^{UL} , it holds that $T_i = \delta_i C_i$ and $\sigma_i = \frac{1}{\text{sk}_s + e_i} C_i$ for randomly sampled $e_i, \delta_i \xleftarrow{\$} \mathbb{Z}$ (both e_i and δ_i are freshly sampled per i) and $C_i \xleftarrow{\$} \mathbb{G}_1$. As C_i is sampled uniformly random, it holds that σ_i is distributed uniformly random. Further, δ_i acts as a random one-time pad such that T_i is distributed uniformly random and independent of σ_i . Obviously, σ_i is the same every time it is queried (T_i can only be queried once). In Exp_8^{UL} , the only difference is that σ_i is sampled uniformly random from $\mathbb{G}_1$ once α, β are defined, i.e., if $i \in \alpha, \beta$ and $\{\alpha, \beta\} \not\subseteq \Gamma$. The latter implies that σ_α and σ_β have not been observed before they are re-sampled, which allows us to conclude that σ_α and σ_β are the same every time they are queried. As in both experiments all observed T_i are independent uniform random elements in $\mathbb{G}_1$ that can only be queried once and σ_i are independent uniform random elements in $\mathbb{G}_1$, that are the same every time they are queried, we can conclude that the view of the adversary is identical in both experiments and, hence, that

$$\Pr[\text{Exp}_8^{\text{UL}}(\lambda) = 1] = \Pr[\text{Exp}_7^{\text{UL}}(\lambda) = 1].$$

Do not swap the selected tokens. In the final experiment, we get rid of the swap and, therefore, of any utilization of the bit b that is to be guessed. The experiment is defined as follows:

Experiment: $\text{Exp}_9^{\text{UL}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), j' \leftarrow 0, b \xleftarrow{\$} \{0, 1\}, \Gamma \leftarrow \emptyset$ 2: $(\text{pk}_s, \text{st}_0^A) \leftarrow \mathcal{A}^{\text{OKG}}(0, \text{pp})$ 3: $(\alpha, \beta, \text{st}_1^A) \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(1, \text{st}_0^A)$ 4: if $\exists k \in \{\alpha, \beta\}$ s.t. $\sigma_k \in \{\text{null}, \perp\} \vee k \in \Gamma$ then abort 5: $\sigma_\alpha, \sigma_\beta \xleftarrow{\$} \mathbb{G}_1, b' \leftarrow \mathcal{A}^{\text{OKG}, \text{ORq}, \text{OFin}, \text{ORd}}(2, \text{st}_1^A)$ 6: if $b = b'$ then return 1 else return 0 ...

As σ_α and σ_β are overwritten after being swapped, the swap (or its removal) does not have any influence of the experiment's output distribution. We can conclude that:

$$\Pr[\text{Exp}_9^{\text{UL}}(\lambda) = 1] = \Pr[\text{Exp}_8^{\text{UL}}(\lambda) = 1].$$

Further, in Exp_9^{UL} , the randomly sampled bit b is never used. Therefore, we can conclude that

$$\Pr[\text{Exp}_9^{\text{UL}}(\lambda) = 1] = \frac{1}{2}.$$

Putting all steps together. As we have a polynomial number of game hops (in the number of oracle queries and the chunk size L), the difference in the success probability

of the adversary in any two adjacent experiments is at most negligible, and the adversary wins in the final game with probability $\frac{1}{2}$, we conclude that

$$|\Pr[\text{Exp}_{\mathcal{A}}^{\text{UL}}(\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda).$$

Appendix H. Proof of Theorem 6

This section provides a detailed proof of Theorem 6. To show betrayability, we have to show that for every pair of PPT adversaries $(\mathcal{A}_B, \mathcal{A}_V)$, there exists an extractor $\mathcal{E}$ and $n = n(\lambda) (n \in \mathbb{N})$ such that

$$\begin{aligned} \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 1}^{\text{BT}}(\lambda) = 1] &\geq \\ \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 0}^{\text{BT}}(\lambda) = 1] - \text{negl}(\lambda). \end{aligned}$$

We show betrayability via a sequence of games. Let $\mathcal{A}_B$ and $\mathcal{A}_V$ be arbitrary PPT adversaries, $\mathcal{E}$ be the extractor defined in Section 5.4, and $\text{Exp}_{0,b}^{\text{BT}} = \text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, b}^{\text{BT}}$ be the initial security game. The subsequent games are defined as follows.

Inline construction and extractor. In game Exp_1^{BT} , we inline the computation of our concrete instantiation as well as the actions of the extractor into the security game. To make the presentation more compact, we denote the interactive protocol between $\mathcal{A}_B$ and $\mathcal{A}_V$, in which the experiment passes the messages between the parties as

$$\langle \mathcal{A}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(2, \cdot, \cdot, \cdot), \mathcal{A}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(2, \cdot, \cdot, \cdot) \rangle.$$

The game is defined as follows:

Experiment: $\text{Exp}_{1,b}^{\text{BT}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ ,  $(\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp})$ ,  $b \xleftarrow{\$} \{0, 1\}$ 
2:  $(\mathcal{K}, \mathcal{T}, \mathcal{T}^*, \text{m}_0^V) \leftarrow \emptyset^4$ ,  $\text{st}_1 \leftarrow \{\text{pp}, n, \text{pk}_s\}$ 
3:  $(\sigma^*, \text{st}_{(2,0)}^B) \leftarrow \tilde{\mathcal{A}}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(1, \text{st}_1)$ ,  $\mathcal{K}^* \leftarrow \mathcal{K}$ 
4:  $(\text{pk}_*^{\text{ld}}, \text{sk}_*^{\text{ld}}, \text{st}_{(2,0)}^V) \leftarrow \mathcal{A}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(1, \sigma^*, \text{st}_1)$ 
5: if  $\text{KeyV}(\text{pp}, \text{sk}_*^{\text{ld}}, \text{pk}_*^{\text{ld}}) = 0$  then return 0
6:  $\langle \mathcal{A}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(2, \cdot, \cdot, \cdot), \mathcal{A}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(2, \cdot, \cdot, \cdot) \rangle$ 
7: if  $b = 0 \wedge \text{success} = \text{true}$  then return 1
8: if  $b = 1 \wedge (x^* G_1) \in \mathcal{K}^*$  then return 1
9: else return 0

```

$\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query} = (T, \pi_{\text{Rq}})) :$

```

10:  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{pk}\}$ 
11: if  $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}, T), \pi_{\text{Rq}}) = 0$  then return  $\emptyset$ 
12:  $e \xleftarrow{\$} \mathbb{Z}_p$ ,  $S \leftarrow \frac{1}{\text{sk}_s + e} T$ 
13: return  $(e, S)$ 

```

$\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, \text{m}) :$

```

14: if  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \in \{0, 1\}$  then return  $\emptyset$ 
15: elseif  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} = \text{null}$  then
16:    $(\sigma, \text{pk}) \leftarrow \text{m}$ ,  $k \xleftarrow{\$} \mathbb{Z}_p$ 
17:   if  $b = 1 \wedge \mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{ld}})$  then
18:      $K \leftarrow kG_1$ ,  $\pi \leftarrow \text{PK}_{\text{Ch}}.\text{Prove}((K, \text{pk}_*^{\text{ld}}), \text{sk}_*^{\text{ld}})$ 
19:      $\mathcal{T}^* \leftarrow \mathcal{T}^* \cup \{\text{sid}\}$ 
20:   else
21:      $K \leftarrow kG_0$ ,  $\pi = \text{PK}_{\text{Ch}}.\text{Prove}((K, \text{pk}), k)$ 
22:     if  $\mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{ld}})$  then  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{sid}\}$ 

```

```

23:  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow (k, K, \sigma, \text{pk})$ , return  $(K, \pi)$ 
24: else
25:    $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}$ ,  $(\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow \text{m}$ 
26:   if  $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$ 
      $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$  then  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0$ , return 0
27:   if  $b = 1 \wedge \mathcal{P} = B \wedge \text{sid} \in \mathcal{T}^* \wedge x^* = \text{null}$  then
28:      $\forall i \in [L] : D_i \leftarrow B_i - kA_i$ ,  $x_i \leftarrow \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$ 
29:     if  $\nexists i \in [L] : x_i = \perp$  then  $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i$ 
30:     if  $b = 0 \wedge \mathcal{P} = B \wedge \text{sid} \in \mathcal{T}$  then success  $\leftarrow \text{true}$ 
31:    $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1$ , return 1

```

This step does not affect the output distributions. It follows that

$$\Pr[\text{Exp}_{1,b}^{\text{BT}}(\lambda) = 1] = \Pr[\text{Exp}_{0,b}^{\text{BT}}(\lambda) = 1] \text{ for } b \in \{0, 1\}.$$

Simulate challenge proofs. First, we create the challenge proofs inside the redemption oracle with the challenge proof's simulator $\mathcal{S}_{\text{Ch}}$ instead of the proof system's witnesses. The simulator receives the proof's statement $(\text{pp}, K, \text{pk}_*^{\text{ld}})$ and has the capability of programming the random oracle. It creates a valid zero knowledge proof π_{Ch} , except with negligible probability. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_{2,b}^{\text{BT}}(\lambda)$

$\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, \text{m}) :$

```

14: if  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \in \{0, 1\}$  then return  $\emptyset$ 
15: elseif  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} = \text{null}$  then
16:    $(\sigma, \text{pk}) \leftarrow \text{m}$ ,  $k \xleftarrow{\$} \mathbb{Z}_p$ 
17:   if  $b = 1 \wedge \mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{ld}})$  then
18:      $K \leftarrow kG_1$ ,  $\pi \leftarrow \mathcal{S}_{\text{Ch}}(\text{pp}, K, \text{pk}_*^{\text{ld}})$ 
19:      $\mathcal{T}^* \leftarrow \mathcal{T}^* \cup \{\text{sid}\}$ 
20:   else
21:      $K \leftarrow kG_0$ ,  $\pi = \mathcal{S}_{\text{Ch}}(\text{pp}, K, \text{pk}_*^{\text{ld}})$ 
22:     if  $\mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{ld}})$  then  $\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{sid}\}$ 
23:    $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow (k, K, \sigma, \text{pk})$ , return  $(K, \pi)$ 
24: else ...

```

As the challenge proofs created in the redemption oracle of $\text{Exp}_{1,b}^{\text{BT}}$ are always valid, it follows that $\text{Exp}_{2,b}^{\text{BT}}$ and $\text{Exp}_{1,b}^{\text{BT}}$ only differ if the simulator is not successful, which happens with negligible probability due to the zero-knowledge property of the challenge proof system. We use the difference lemma to conclude that

$$|\Pr[\text{Exp}_{2,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{1,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda) \text{ (for } b \in \{0, 1\} \text{)}.$$

Coin independent oracles. Next, we include a bridging step to make the whole redemption oracle independent of the coin b . The following changes are only of conceptual nature and have no influence on the output distribution.

- We always include the trapdoor into the challenge message, i.e., always define $K = kG_1$. In both experiments, this and the previous, K is an independent random group element and the challenge proof is a simulated valid proof. Therefore, it follows that the adversary's view and, hence, the output distribution is not affected by this change.

- We merge $\mathcal{T}$ and $\mathcal{T}^*$ and utilize the same set $\mathcal{T}$ regardless of b . If $b = 1$, $\mathcal{T}$ is never populated. The same holds for $\mathcal{T}^*$ if $b = 0$. Therefore, it does not affect the experiment to use one set for $\mathcal{T}$ and $\mathcal{T}^*$.
- We try to compute x^* independent of b , i.e., also if $(b = 0 \wedge \mathcal{P} = B \wedge \text{sid} \in \mathcal{T})$. This does not change the output distribution as x^* is not further used if $b = 0$.
- We set $\text{success} \leftarrow \text{true}$ if $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$ but before the (potential) decryption of the ciphertexts $\{A_i, B_i\}$. As success is not further used if $b = 1$, defining success for $b = 1$ has no influence on the output distribution. Requiring $x^* = \text{null}$ for setting $\text{success} \leftarrow \text{true}$ does not influence the output distribution as, in the new experiment, x^* can only be defined to be not null if success has been set to true before.
- We merge redundant program flows instructions.

The resulting experiment is defined as follows:

Experiment: $\text{Exp}_{3,b}^{\text{BT}}(\lambda)$
$\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, m) :$ 14: if $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \in \{0, 1\}$ then return $\emptyset$ 15: elseif $\text{st}_{\text{sid}}^{(S, \mathcal{P})} = \text{null}$ then 16: $(\sigma, \text{pk}) \leftarrow m, k \xleftarrow{\$} \mathbb{Z}_p, K \leftarrow kG - 1, \pi \leftarrow \mathcal{S}_{\text{Ch}}(\text{pp}, K, \text{pk}_*^{\text{d}})$ 17: if $\mathcal{P} = B \wedge (\sigma, \text{pk}) = (\sigma^*, \text{pk}_*^{\text{d}})$ then $\mathcal{T} \leftarrow \mathcal{T} \cup \{\text{sid}\}$ 18: $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow (k, K, \sigma, \text{pk}),$ return (K, π) 19: else 20: $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}, (\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 21: if $\text{PK}_{\text{Rd}}. \text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$ $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0, \text{return } 0$ 22: if $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$ then 23: success $\leftarrow \text{true}$ 24: $\forall i \in [L] : D_i \leftarrow B_i - kA_i, x_i \leftarrow \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$ 25: if $\nexists i \in [L] : x_i = \perp$ then $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i$ 26: $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1,$ return 1

These changes do not affect the output distributions. It follows that

$$\Pr[\text{Exp}_{3,b}^{\text{BT}}(\lambda) = 1] = \Pr[\text{Exp}_{2,b}^{\text{BT}}(\lambda) = 1] \text{ for } b \in \{0, 1\}.$$

Extract redemption proofs. In the next game, we employ the extractor of the redemption proof system $\mathcal{E}_{\text{Rd}}$ to extract the redemption proof's witnesses if the proof has been validated correctly and $(\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null})$. The extractor has access to the adversaries $(\mathcal{A}_V, \mathcal{A}_B)$, receives the transcript trans of the experiment execution up to the extraction including the adversaries' random tapes, and has the power to program the random oracle. It extracts the witnesses $(y', r', e', \{x'_i, r'_i\}_{i \in [L]})$ that are either all $\perp$ or satisfy the proof system's relation PK_{Rd} . If the extraction is not successful, we abort the experiment with output 0.

Experiment: $\text{Exp}_{4,b}^{\text{BT}}(\lambda)$
$\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, m) :$ 15: ... 20: else 21: $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}, (\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 22: if $\text{PK}_{\text{Rd}}. \text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$

$e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0, \text{return } 0$ 23: if $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$ then 24: $(y', r', e', \{x'_i, r'_i\}_{i \in [L]}) \leftarrow \mathcal{E}_{\text{Rd}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 25: if $e' = \perp$ then abort experiment with output 0 26: success $\leftarrow \text{true}$ 27: $\forall i \in [L] : D_i \leftarrow B_i - kA_i, x_i \leftarrow \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$ 28: if $\nexists i \in [L] : x_i = \perp$ then $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i$ 29: $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1, \text{return } 1$
--

It is obvious that $\text{Exp}_{4,b}^{\text{BT}}$ and $\text{Exp}_{3,b}^{\text{BT}}$ only differ if the extractor is not successful, which happens with at most negligible probability due to the knowledge soundness property of the redemption proof system. This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_{4,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{3,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda) \text{ (for } b \in \{0, 1\}).$$

Extract request proofs and store issued tokens. Next, we adapt the request oracle to employ the request proof system's extractor $\mathcal{E}_{\text{Rq}}$ to extract the witnesses of the request proofs $(\{x_i\}_{i \in [L]}, r, \delta)$. The extractor has access to the adversaries $(\mathcal{A}_V, \mathcal{A}_B)$, receives the transcript trans of the experiment execution up to the extraction including the adversaries' random tapes, and has the power to program the random oracle. The extracted witnesses are either all $\perp$ or satisfy the proof system's relation. If the extraction is not successful, we abort the experiment with output 0.

Additionally, we compute the resulting token σ and if $\mathcal{K}^*$ is not defined, yet, i.e., if σ^* has not been submitted, store the token with the witnesses in an extra set Γ . The resulting experiment is defined as follows:

Experiment: $\text{Exp}_{5,b}^{\text{BT}}(\lambda)$
1: $\text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp}), b \xleftarrow{\$} \{0, 1\}$ 2: $(\mathcal{K}, \mathcal{T}, \mathcal{T}^*, \Gamma, m_0^5) \leftarrow \emptyset^5, \text{st}_1 \leftarrow \{\text{pp}, n, \text{pk}_s\}$ 3: ... $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query} = (T, \pi_{\text{Rq}})) :$ 10: $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{pk}\}$ 11: if $\text{PK}_{\text{Rq}}. \text{Verify}(\text{rpp}, (\text{pk}, T), \pi_{\text{Rq}}) = 0$ then return $\emptyset$ 12: $(\{x_i\}_{i \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 13: if $r = \perp$ then abort experiment with output 0 14: $e \xleftarrow{\$} \mathbb{Z}_p, S \leftarrow \frac{1}{\text{sk}_s + e} T$ 15: $\sigma \leftarrow \frac{1}{\text{sk}_s + e} (G_0 + rH_0 + \sum_{i \in [L]} x_i H_i)$ 16: if $\mathcal{K}^* = \text{null}$ then $\Gamma \leftarrow \Gamma \cup \{(\sigma, e, r, \{x_i\}_{i \in [L]})\}$ 17: return (e, S) ...

As Γ is only populated but not used, it is clearly evident that $\text{Exp}_{5,b}^{\text{BT}}$ and $\text{Exp}_{4,b}^{\text{BT}}$ only differ if the extractor is not successful, which happens with at most negligible probability due to the knowledge soundness property of the request proof system. This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_{5,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{4,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda) \text{ (for } b \in \{0, 1\}).$$

Abort if σ^ is re-issued.* Next, we check for every (blindly) issued token if it is equal to σ^* and abort the

experiment with output 0 if this is the case. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_{6,b}^{\text{BT}}(\lambda)$ $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query} = (T, \pi_{\text{Rq}})) :$ 10: $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{pk}\}$ 11: if $\text{PK}_{\text{Rq}}.\text{Verify}(\text{rpp}, (\text{pk}, T), \pi_{\text{Rq}}) = 0$ then return $\emptyset$ 12: $(\{x_i\}_{i \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 13: if $r = \perp$ then abort experiment with output 0 14: $e \leftarrow \mathbb{Z}_p, S \leftarrow \frac{1}{\text{sk}_s + e} T$ 15: $\sigma \leftarrow \frac{1}{\text{sk}_s + e} (G_0 + rH_0 + \sum_{i \in [L]} x_i H_i)$ 16: if $\mathcal{K}^* = \text{null}$ then $\Gamma \leftarrow \Gamma \cup \{(\sigma, e, r, \{x_i\}_{i \in [L]})\}$ 17: if $\sigma^* \neq \text{null} \wedge \sigma = \sigma^*$ then abort experiment with output 0 18: return (e, S) ...
--

It is evident that $\text{Exp}_{6,b}^{\text{BT}}$ and $\text{Exp}_{5,b}^{\text{BT}}$ only differ if any token computed within the request oracle is equal to σ^* after σ^* has already been defined. As e is sampled uniformly random, it holds that $(\text{sk}_s + e)$, $\frac{1}{\text{sk}_s + e}$ and, hence, σ is distributed uniformly random. Therefore, it can only happen with negligible probability that the request oracle (blindly) issues a token equal to σ^* (after σ^* has already been defined). We apply the difference lemma to conclude that

$$|\Pr[\text{Exp}_{6,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{5,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

(for $b \in \{0, 1\}$).

Abort if σ^ is a forgery.* Before setting success to **true**, we check if there is an element $(\sigma^*, \cdot, \cdot, \cdot)$ in Γ and abort the experiment with output 0, otherwise. The resulting game is defined as follows:

Experiment: $\text{Exp}_{7,b}^{\text{BT}}(\lambda)$ $\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, m) :$ 19: ... 24: else 25: $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}, (\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 26: if $\text{PK}_{\text{Rd}}.\text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$ $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0$, return 0 27: if $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$ then 28: $(y', r', e', \{x'_i, r'_i\}_{i \in [L]}) \leftarrow \mathcal{E}_{\text{Rd}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 29: if $e' = \perp$ then abort experiment with output 0 30: if $\nexists (\sigma^*, \cdot, \cdot, \cdot) \in \Gamma$ then abort experiment with output 0 31: success $\leftarrow \text{true}$ 32: $\forall i \in [L] : D_i \leftarrow B_i - kA_i, x_i \leftarrow \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$ 33: if $\nexists i \in [L] : x_i = \perp$ then $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i$ 34: $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1$, return 1
--

It is obvious that $\text{Exp}_{7,b}^{\text{BT}}$ and $\text{Exp}_{6,b}^{\text{BT}}$ only differ if the newly inserted abort is executed. Let's call this event F . For F to happen, the adversary $\mathcal{A}_V$ has to submit a token at the beginning of the experiment, that has not been issued by the request oracle, for which the other adversary $\mathcal{A}_B$ learns the corresponding witnesses (the extraction of the redemption proof needs to be successful for the newly inserted abort to be executed). This means that the token-witness pair constitutes a forgery if F happens. Formally, we can show that F cannot happen except with negligible probability via

a reduction to the BBS unforgeability game. We pick $L'(\lambda)$ such that, for $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e) \leftarrow \text{GGen}(1^\lambda)$, $G_1 \xleftarrow{\$} \mathbb{G}_1^*$, and $\ell = \lceil \frac{|p|}{L'(\lambda)} \rceil$, the computation of $\text{dlog}_{G_1}(G)[0, 2^\ell - 1]$ has complexity below $\mathcal{C}$ for all $G \in \mathbb{G}_1$. Then, we define an adversary $\mathcal{B}$ on security game $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}$ that internally executes $\text{Exp}_{7,b}^{\text{BT}}$ with $(\mathcal{A}_V, \mathcal{A}_B)$ and wins with the probability of F occurring in $\text{Exp}_{7,b}^{\text{BT}}$. We define $\mathcal{B}$ as follows:

- Receive $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, \{H_i\}_{i \in [L'+1]}, G_2, Z)$ from the $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}$ security game.
- Pick ℓ such that $\text{dlog}_{G_1}(G)[0, 2^{\ell-1}]$ can be computed with complexity of at most $\mathcal{C}$ for any $G \in \mathbb{G}_1$ and set $L \leftarrow \lceil \frac{|p|}{\ell} \rceil$. Based on the definition of L' it holds that $L \leq L'$.
- Define $H_0 \leftarrow H_{L+1}$ and compute $\text{rpp} = \text{RP.Setup}(p, \mathbb{G}_1, G_0, H_0, L, \ell)$.
- Define $\text{pp} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, G_0, G_1, H_0, H_1, \dots, H_L, L, \ell, \text{rpp})$.
- Execute $\text{Exp}_{6,b}^{\text{BT}}$ with $(\mathcal{A}_V, \mathcal{A}_B)$ based on public parameters pp , server public key $\text{pk}_s = Z$ and with unknown server secret key sk_s and answer oracle queries as follows:
 - To answer $\mathcal{O}^{\text{Rq}}$, execute the steps of the oracle as defined in $\text{Exp}_{6,b}^{\text{BT}}$ but instead of sampling e and calculating σ and S (which is not possible, as sk_s is not known), query $\mathcal{O}^{\text{Sign}}(\{x_i\}_{i \in [L]} \cup \{r\} \cup \{0\}_{i \in L' \setminus L})$ of the $\text{Exp}_{\mathcal{B}, L'+1}^{\text{P-BBS-SUF}}$ game to receive (e, σ) and compute $S = \delta\sigma$.
 - $\mathcal{O}^{\text{Rd}}$ can be executed as defined in $\text{Exp}_{6,b}^{\text{BT}}$ since it does not require knowledge of the server secret key. However, if the abort in line 30 is executed, i.e., if F occurs, return the forgery $(\{x'_i\}_{i \in [L]} \cup \{r'\} \cup \{0\}_{i \in L' \setminus L}, \sigma^*, e')$.
- Return $\perp$.

First note, that the view of the adversaries in the experiment simulated by $\mathcal{B}$ is identical to the view of the adversaries in $\text{Exp}_{7,b}^{\text{BT}}$. The public parameters are generated identically and oracle queries are answered identically in both experiments. Therefore, it also needs to hold that F occurs in the experiment simulated by $\mathcal{B}$ with the same probability as in $\text{Exp}_{7,b}^{\text{BT}}$. Further, we observe that (a) $\mathcal{B}$ outputs a tuple $\neq \perp$ if and only if F occurs, (b) every tuple $\neq \perp$ returned by $\mathcal{B}$ is a valid BBS signature, and (c) every output $\neq \perp$ has not been issued by the signing oracle. Observation (a) follows straight-forwardly from the definition of $\mathcal{B}$. Observation (b) follows from the fact that the non- $\perp$ output of $\mathcal{B}$ is only produced if F occurs and F can only occur if $e(\sigma^*, Z) = e(\sigma', G_2)$ and $\sigma' = G_0 + r'H_0 - e'\sigma^* + \sum_{i \in [L]} x'_i H_i$ (the extraction of the redemption proof needs to be successful), which implies that $\sigma^* = \frac{1}{x+e'}(G_0 + r'H_0 + \sum_{i \in [L]} x'_i H_i)$. Finally, observation (c) follows from the fact that $\text{Exp}_{7,b}^{\text{BT}}$ aborts if a signature with $\sigma = \sigma^*$ is issued after σ^* is defined, every token issued before σ^* is defined is added to Γ and F cannot occur if σ^* has been added to Γ . From observations (a), (b), and (c), we can conclude that $\mathcal{B}$ wins with the same probability as F occurs in $\text{Exp}_{7,b}^{\text{BT}}$. As the success probability of $\mathcal{B}$ is bound to be negligible by assumption (cf. Theorem 1), we can use the difference lemma to conclude that

$$|\Pr[\text{Exp}_{7,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{6,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda)$$

(for $b \in \{0, 1\}$).

Abort if σ^ is a collision.* In the next game, we check if the witnesses extracted in the redemption proof for token

σ^* , i.e., $(e', r', \{x'_i\}_{i \in [L]})$ match the witnesses stored in Γ for token σ^* . If this is the case, we abort the experiment. The resulting experiment is defined as follows:

Experiment: $\text{Exp}_{8,b}^{\text{BT}}(\lambda)$ $\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, m) :$ 19: ... 24: else 25: $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}, (\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 26: if $\text{PK}_{\text{Rd}}. \text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$ $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$ then $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0$, return 0 27: if $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$ then 28: $(y', r', e', \{x'_i\}_{i \in [L]}) \leftarrow \mathcal{E}_{\text{Rd}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 29: if $e' = \perp$ then abort experiment with output 0 30: if $\nexists (\sigma^*, \cdot, \cdot, \cdot) \in \Gamma$ then abort experiment with output 0 31: Let $(e^*, r^*, \{x_i^*\}_{i \in [L]})$ be such that $(\sigma^*, e^*, r^*, \{x_i^*\}_{i \in [L]}) \in \Gamma$ (first occurrence, if multiple) 32: if $(e', r', \{x'_i\}_{i \in [L]}) \neq (e^*, r^*, \{x_i^*\}_{i \in [L]})$ then abort experiment with output 0 33: success $\leftarrow \text{true}$ 34: $\forall i \in [L] : D_i \leftarrow B_i - kA_i, x_i \leftarrow \text{dlog}_{G_1}(D_i)[0, 2^\ell - 1]$ 35: if $\nexists i \in [L] : x_i = \perp$ then $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i$ 36: $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1$, return 1

Again, it is obvious that $\text{Exp}_{8,b}^{\text{BT}}$ and $\text{Exp}_{7,b}^{\text{BT}}$ only differ if the newly inserted abort is executed. Let's call this event F . We show that the probability of F is negligible by building a PPT discrete logarithm solver in $\mathbb{G}_1, \mathcal{B}$, that internally simulates $\text{Exp}_{8,b}^{\text{BT}}$ and solves discrete logarithm problems with probability

$$\Pr[\text{Exp}_{\mathcal{B}}^{\text{dlog}}(1^\lambda) = 1] = \Pr[F \text{ in } \text{Exp}_{8,b}^{\text{BT}}(1^\lambda)] \left(\frac{p-1}{p}\right)^{L+3}.$$

We define the solver $\mathcal{B}$ as follows:

- Execute $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and send $(\mathbb{G}_1, p)$ to the discrete logarithm challenger to receive (G, H). - Sample $\alpha_g, \alpha_0, \alpha_1, \dots, \alpha_L \xleftarrow{\\$} \mathbb{Z}_p$ and $\beta_g, \beta_0, \beta_1, \dots, \beta_L \xleftarrow{\\$} \mathbb{Z}_p$. - Compute $G_0 = \alpha_g G + \beta_g H$ and sample $G_1 \xleftarrow{\\$} \mathbb{G}_1^*$. - For $i \in [0, L]$ compute $H_i = \alpha_i G + \beta_i H$. - If $G_0 = 0$ or $\exists i \in [0, L] : H_i = 0$ then output $\perp$. - Sample $\text{rpp} \leftarrow \text{RP.Setup}(p, \mathbb{G}_1, G_0, H_0, L, \ell)$. - Define $\text{pp}' \leftarrow (p, \mathbb{G}_1, G_0, G_1, H_0, H_1, \dots, H_L, L, \ell, \text{rpp})$. - Execute experiment $\text{Exp}_{8,b}^{\text{BT}}$ with the following modifications: - Instead of computing $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, set $\text{pp} \leftarrow \text{pp}'$. - If the newly inserted abort condition is executed, i.e., if Line 32 is executed with $(e^*, r^*, \{x_i^*\}_{i \in [L]}) \neq (e', r', \{x'_i\}_{i \in [L]})$, then store $z = \text{sk}_s, (e^*, r^*, \{x_i^*\}_{i \in [L]})$ and $(e', r', \{x'_i\}_{i \in [L]})$ and abort the experiment execution. - If the experiment ends without being aborted by the solver then output $\perp$. - Compute: $u \leftarrow (z + e')(\alpha_g + \alpha_0 r^* + \sum_{i \in [L]} \alpha_i x_i^*) - (z + e^*)(\alpha_g + \alpha_0 r' + \sum_{i \in [L]} \alpha_i x'_i)$ $v \leftarrow (z + e')(\beta_g + \beta_0 r^* + \sum_{i \in [L]} \beta_i x_i^*) - (z + e^*)(\beta_g + \beta_0 r' + \sum_{i \in [L]} \beta_i x'_i)$ - If $v = 0$ then output $\perp$. Otherwise, output $h = -\frac{u}{v}$.

First, note that $\mathcal{B}$ perfectly simulates the experiment to the adversaries (until the abort). This is due to the fact that the solver's only change to the experiment is the way the generators $G_0, H_0, H_1, \dots, H_L$ are sampled. However, if $\mathcal{B}$

simulates the experiment, i.e., if none of the generators is equal to 0, the generators $G_0, H_0, H_1, \dots, H_L$ are uniformly random generators of $\mathbb{G}_1$ just as they are in a real experiment. Second, we note that the probability that none of the generators $G_0, H_0, H_1, \dots, H_L$ equals 0 is $(\frac{p-1}{p})^{L+2}$. Therefore, it follows that the probability that F occurs in the experiment simulated by the discrete logarithm solver is $(\Pr[F \text{ in } \text{Exp}_{8,b}^{\text{BT}}] \cdot (\frac{p-1}{p})^{L+2})$. Third, we note that if F occurs in the simulated experiment and $v \neq 0$ then it holds that $H = hG$. To see this, first recall that the occurrence of event F implies that

$$\begin{aligned} \sigma^* &= \frac{1}{z + e^*} (G_0 + r^* H_0 + \sum_{i \in [L]} x_i^* H_i) \\ &= \frac{1}{z + e'} (G_0 + r' H_0 + \sum_{i \in [L]} x'_i H_i). \end{aligned}$$

Therefore, it holds that

$$\begin{aligned} 0 &= (z + e')(G_0 + r^* H_0 + \sum_{i \in [L]} x_i^* H_i) \\ &\quad - (z + e^*)(G_0 + r' H_0 + \sum_{i \in [L]} x'_i H_i) \\ &= ((z + e')(\alpha_0 + \alpha_h r^* + \sum_{i \in [L]} \alpha_i x_i^*) \\ &\quad - (z + e^*)(\alpha_0 + \alpha_h r' + \sum_{i \in [L]} \alpha_i x'_i))G \\ &\quad + ((z + e')(\beta_0 + \beta_h r^* + \sum_{i \in [L]} \beta_i x_i^*) \\ &\quad - (z + e^*)(\beta_0 + \beta_h r' + \sum_{i \in [L]} \beta_i x'_i))H \\ &= uG + vH \Leftrightarrow H = \frac{-u}{v}G = hG \quad (\text{if } v \neq 0). \end{aligned}$$

Fourth, we observe that the probability that $v = 0$ is $\frac{1}{p}$. Let h be such that $H = hG$, $\gamma_0 = \alpha_g + h\beta_g$ and $\gamma_i = \alpha_i + h\beta_i$ for $i \in [0, L]$. By definition, it holds that $G_0 = \gamma_g G$ and $H_i = \gamma_i G$ for $i \in [0, L]$. Note that the only information about the α and β variables provided to the adversary are the generators $G_0, H_0, H_1, \dots, H_L$. During the execution of the simulated experiments, the α - and β -variables are no longer used. With the generators, the adversary gets some information on $\gamma_h, \gamma_0, \gamma_1, \dots, \gamma_L$ but the variables $\beta_g, \beta_0, \beta_1, \dots, \beta_L$ are independent from the γ -variables as the α -values act as one-time pads. Now, we re-write v as

$$\begin{aligned} v &= (e' - e^*)\beta_g + ((z + e')r^* - (z + e^*)r')\beta_0 \\ &\quad + \sum_{i \in [L]} ((z + e')x_i^* - (z + e^*)x'_i)\beta_i \end{aligned}$$

From this representation, it becomes evident that, if $r^* \neq r'$ or there exists $i \in [L]$ such that $x_i^* \neq x'_i$ (which is necessary for event F), we can conclude that the distribution of v conditioned on fixed generators is uniform. Therefore, it follows that $\Pr[v = 0 \mid \text{event } F] = \frac{1}{p}$. By combining these

observations, we can conclude that $\mathcal{E}$ indeed solves discrete logarithm problems with probability

$$\Pr[\text{Exp}_B^{\text{dlog}}(1^\lambda) = 1] = \Pr[F \text{ in } \text{Exp}_{8,b}^{\text{BT}}(1^\lambda)] \left(\frac{p-1}{p}\right)^{L+3}.$$

Finally, the discrete logarithm assumption states that the advantage of every PPT discrete logarithm solver is bound by $\text{Adv}_{\text{dlog}}(\lambda) \leq \text{negl}(\lambda)$. Therefore, we conclude that

$$\Pr[\text{Exp}_B^{\text{dlog}}(1^\lambda) = 1] \leq \text{Adv}_{\text{dlog}}(\lambda) \leq \text{negl}(\lambda) \\ \Leftrightarrow \Pr[F \text{ in } \text{Exp}_{8,b}^{\text{BT}}(\lambda)] \leq \text{Adv}_{\text{dlog}}(\lambda) \cdot \left(\frac{p}{p-1}\right)^{L+3} \leq \text{negl}(\lambda)$$

This allows us to use the difference lemma to conclude that

$$|\Pr[\text{Exp}_{8,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{7,b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda) \\ (\text{for } b \in \{0, 1\}).$$

Compute x^* based on the witnesses stored in Γ . In the next game, we do not decrypt the ciphertexts but compute x^* based on $\{x_i^*\}_{i \in [L]}$.

Experiment: $\text{Exp}_{9,b}^{\text{BT}}(\lambda)$

```

...
 $\mathcal{O}^{\text{Rd}}(\mathcal{P}, \text{sid}, m) :$ 
19: ...
24: else
25:  $(k, K, \sigma, \text{pk}) \leftarrow \text{st}_{\text{sid}}^{(S, \mathcal{P})}, (\pi, \sigma', \{A_i, B_i\}_{i \in [L]}) \leftarrow m$ 
26: if  $\text{PK}_{\text{Rd}}. \text{Verify}((\text{pk}, K, \sigma, \sigma', \{A_i, B_i\}_{i \in [L]}), \pi_{\text{Rd}}) = 0 \vee$ 
 $e(\sigma, \text{pk}_s) \neq e(\sigma', G_2)$  then  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 0$ , return 0
27: if  $\mathcal{P} = B \wedge \text{sid} \in \mathcal{T} \wedge x^* = \text{null}$  then
28:  $(y', r', e', \{x_i', r_i'\}_{i \in [L]}) \leftarrow \mathcal{E}_{\text{Rd}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 
29: if  $e' = \perp$  then abort experiment with output 0
30: if  $\nexists (\sigma^*, \cdot, \cdot, \cdot) \in \Gamma$  then abort experiment with output 0
31: Let  $(e^*, r^*, \{x_i^*\}_{i \in [L]})$  be such that  $(\sigma^*, e^*, r^*, \{x_i^*\}_{i \in [L]}) \in \Gamma$  (first occurrence, if multiple)
32: if  $(e', r', \{x_i'\}_{i \in [L]}) \neq (e^*, r^*, \{x_i^*\}_{i \in [L]})$  then abort
experiment with output 0
33: success  $\leftarrow \text{true}$ ,  $x^* \leftarrow \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i^*$ 
34:  $\text{st}_{\text{sid}}^{(S, \mathcal{P})} \leftarrow 1$ , return 1

```

First, we make the following observations with respect to $\text{Exp}_{8,b}^{\text{BT}}$. For $i \in [L]$ it needs to hold that $D_i = x_i' G_1$ (due to the successful extraction of the redemption proof), $x_i' = x_i^*$ (due to the previous checks), and $x_i^* \in [0, 2^\ell - 1]$ (due to the successful extraction of the request proofs). It follows that $D_i = x_i^* G_1$ (from the first two) and that $\text{dlog}_{G_1}(D_i) \neq \perp$ (from the third). From these, we can conclude that the check $(\nexists i \in [L] : x_i = \perp)$ is redundant, i.e., x^* is defined whenever the decryption is attempted, and that $x_i = x_i^*$ for $i \in [L]$. It follows that x^* is defined whenever success is set to **true** and that x^* is always equal to $\sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i^*$ in $\text{Exp}_{8,b}^{\text{BT}}$. Therefore, we can conclude that

$$\Pr[\text{Exp}_{9,b}^{\text{BT}}(\lambda) = 1] = \Pr[\text{Exp}_{8,b}^{\text{BT}}(\lambda) = 1] \text{ for } b \in \{0, 1\}.$$

Final game. Finally, we modify the success condition of the experiment. In particular, if $b = 0$, we only output 1 if $x^* G_1 \in \mathcal{K}^* \wedge \text{success} = \text{true}$. The former of the two conditions is new. The resulting final experiment is defined as follows:

Experiment: $\text{Exp}_{10}^{\text{BT}}(\lambda)$

```

1:  $\text{pp} \leftarrow \text{Setup}(1^\lambda), (\text{sk}_s, \text{pk}_s) \leftarrow \text{S.KeyGen}(\text{pp}), b \xleftarrow{\$} \{0, 1\}$ 
2:  $(\mathcal{K}, \mathcal{T}, \mathcal{T}^*, m_0^V) \leftarrow \emptyset^4, \text{st}_1 \leftarrow \{\text{pp}, n, \text{pk}_s\}$ 
3:  $(\sigma^*, \text{st}_{(2,0)}^B) \leftarrow \tilde{\mathcal{A}}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(1, \text{st}_1), \mathcal{K}^* \leftarrow \mathcal{K}$ 
4:  $(\text{pk}_*^{\text{ld}}, \text{sk}_*^{\text{ld}}, \text{st}_{(2,0)}^V) \leftarrow \mathcal{A}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(1, \sigma^*, \text{st}_1)$ 
5: if  $\text{KeyV}(\text{pp}, \text{sk}_*^{\text{ld}}, \text{pk}_*^{\text{ld}}) = 0$  then return 0
6:  $(\mathcal{A}_B^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(B, \cdot, \cdot)}(2, \cdot, \cdot, \cdot), \mathcal{A}_V^{\mathcal{O}^{\text{Rq}}, \mathcal{O}^{\text{Rd}}(V, \cdot, \cdot)}(2, \cdot, \cdot, \cdot))$ 
7: if  $b = 0 \wedge (x^* G_1) \in \mathcal{K}^* \wedge \text{success} = \text{true}$  then return 1
8: if  $b = 1 \wedge (x^* G_1) \in \mathcal{K}^*$  then return 1
9: else return 0
 $\mathcal{O}^{\text{Rq}}(\text{pk}, \text{query} = (T, \pi_{\text{Rq}})) :$ 
10:  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{pk}\}$ 
11: if  $\text{PK}_{\text{Rq}}. \text{Verify}(\text{rpp}, (\text{pk}, T), \pi_{\text{Rq}}) = 0$  then return  $\emptyset$ 
12:  $(\{x_i\}_{i \in [L]}, r, \delta) \leftarrow \mathcal{E}_{\text{Rq}}^{\mathcal{A}_V(\cdot), \mathcal{A}_B(\cdot)}(\text{trans})$ 
13: if  $r = \perp$  then abort experiment with output 0
14:  $e \xleftarrow{\$} \mathbb{Z}_p, S \leftarrow \frac{1}{\text{sk}_s + e} T$ 
15:  $\sigma \leftarrow \frac{1}{\text{sk}_s + e} (G_0 + r H_0 + \sum_{i \in [L]} x_i H_i)$ 
16: if  $\mathcal{K}^* = \text{null}$  then  $\Gamma \leftarrow \Gamma \cup \{(\sigma, e, r, \{x_i\}_{i \in [L]})\}$ 
17: if  $\sigma^* \neq \text{null} \wedge \sigma = \sigma^*$  then abort experiment with output 0
18: return  $(e, S)$ 
...
```

We observe that $x^* G_1 \in \mathcal{K}^*$ holds whenever success = **true**. To see this, first observe that for every $(\cdot, \cdot, \cdot, \{x_i\}_{i \in [L]}) \in \Gamma$, it needs to hold that $\text{pk} = (\sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i) G_1$ (due to the successful extraction) and that $\text{pk} \in \mathcal{K}^*$ as for every tuple added to Γ the corresponding public key pk is added to $\mathcal{K}$ before $\mathcal{K}^*$ is defined and, hence, also ends up in $\mathcal{K}^*$. Secondly, observe that if success is set to **true**, then x^* is defined to be $x^* = \sum_{i \in [L]} 2^{(i-1) \cdot \ell} x_i^*$ for $(\cdot, \cdot, \cdot, \{x_i^*\}_{i \in [L]}) \in \Gamma$. From these two observations, we can conclude that there exists $\text{pk}^* \in \mathcal{K}^*$ such that $\text{pk}^* = x^* G_1$ if success = **true**. As $x^* G_1 \in \mathcal{K}^*$ holds whenever success = **true**, inserting the additional success condition does not affect the output distribution of the experiment. Therefore, we can conclude that

$$\Pr[\text{Exp}_{10,b}^{\text{BT}}(\lambda) = 1] = \Pr[\text{Exp}_{9,b}^{\text{BT}}(\lambda) = 1] \text{ for } b \in \{0, 1\}.$$

Output distribution in the final game. In the final game, the experiment for $b = 0$ and $b = 1$ only differ in the condition for outputting 1. The condition for outputting 1 if $b = 1$ is contained in the one for outputting 1 if $b = 0$. Therefore, it follows that $\Pr[\text{Exp}_{10,1}^{\text{BT}}(\lambda) = 1] \geq \Pr[\text{Exp}_{10,0}^{\text{BT}}(\lambda) = 1]$.

Further, we have used a polynomial number of game hops and shown that the difference in the success probability of the adversary in any two adjacent experiments is at most negligible. Therefore, we can conclude that

$$|\Pr[\text{Exp}_{10,b}^{\text{BT}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, b}^{\text{BT}}(\lambda) = 1]| \leq \text{negl}(\lambda) \\ (\text{for } b \in \{0, 1\}).$$

By combining the above two, we can conclude that

$$\Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 1}^{\text{BT}}(\lambda) = 1] \geq \\ \Pr[\text{Exp}_{\mathcal{A}_B, \mathcal{A}_V, \mathcal{E}, n, 0}^{\text{BT}}(\lambda) = 1] - \text{negl}(\lambda).$$